ICPC Team Notebook

1 Template **1**
1.1 Contest Template 1

2 Data Structures **1**
2.1 Disjoint Set Union 1
2.2 Fenwick Tree 1
2.3 Sparse Table 2
2.4 Wavelet Tree 2
2.5 Short Segment Tree 3
2.6 Priority Queue with Delete 3
2.7 Lazy Segment Tree 4
2.8 Segment Tree Beats 4
2.9 Segment Tree Beats Invariants 5
2.10 Line Container 6
2.11 Implicit Treap 6
2.12 DSU on Segment Tree 7
2.13 Persistent Segment Tree 7
2.14 Monotonic Stack and Queue 8
2.15 Cartesian Tree 8
2.16 Sqrt Decomposition 9
2.17 Mo's Algorithm 9
2.18 Link-Cut Tree 10
2.19 Link-Cut Tree Operations 10

3 String **11**
3.1 KMP Prefix Function 11
3.2 Z Function 11
3.3 Aho-Corasick 11
3.4 Tries 12
3.5 Rolling Hash 13
3.6 Manacher 13
3.7 Palindromic Tree (Eertree) 14
3.8 Minimum Rotation 14
3.9 Suffix Array 15
3.10 Kasai LCP 15
3.11 Suffix Automaton 16

4 Graph **16**
4.1 Dijkstra 16
4.2 Bellman–Ford / Difference Constraints 17
4.3 Multi-source and 0-1 BFS 18
4.4 Topological Sort 18
4.5 Floyd–Warshall 18
4.6 Kruskal MST 19
4.7 Functional Graph 19
4.8 Strongly Connected Components . . . 19
4.9 LCA Binary Lifting 20
4.10 Deterministic Tree Canonization . . . 21
4.11 Bridges 21
4.12 Articulation Points 22
4.13 Block-Cut Tree 22
4.14 Dinic Max Flow 23
4.15 Maximum-Weight Closure 24
4.16 Min-Cut Modeling Recipes 24
4.17 Min Cost Flow 24
4.18 Kuhn Bipartite Matching 25
4.19 Hungarian Assignment 26
4.20 2-SAT 26
4.21 Euler Walk 27
4.22 Heavy Light Decomposition 27
4.23 Rerooting DP 28
4.24 Centroid Decomposition 29
4.25 Offline Dynamic Connectivity 29
4.26 CDQ 3D Dominance 30
4.27 DSU on Tree Sack 31
4.28 Virtual Tree 32
4.29 Kruskal Reconstruction Tree 32
4.30 Global Min Cut 32
4.31 Maximum Clique 33
4.32 Critical Cities Dominator 34

5 Math **36**
5.1 Number Theory Essentials 36
5.2 Sieve 36
5.3 CRT 36
5.4 Mod Inverse Table 37
5.5 Binomial (auto-extend) 37
5.6 Lucas Theorem 37

5.7 Lucas and Kummer 38
5.8 Matrix Power 38
5.9 Primitive Root 38
5.10 Miller Rabin and Pollard Rho 39
5.11 Gaussian Elimination mod p 39
5.12 Floating Simplex 40
5.13 Simplex Modeling 41
5.14 Catalan Numbers 41
5.15 Mobius Sieve 41
5.16 Mobius Inversion Notes 41
5.17 XOR Linear Basis 42
5.18 XOR FWHT 42
5.19 Discrete Log 43
5.20 Discrete kth Roots mod Prime 43
5.21 FFT 44
5.22 FFT Mod 45
5.23 NTT 45
5.24 Formal Power Series 46
5.25 FPS Newton Formulas 46
5.26 Fraction Finder 47
5.27 Zeta and Mobius Transforms 47
5.28 Inclusion–Exclusion 48
5.29 Inclusion–Exclusion Formulas 48
5.30 Sprague–Grundy 48
5.31 Floor Sum 49
5.32 Floor-Sum Notes 49
5.33 Lagrange Interpolation 49
5.34 Lagrange Formula 50

6 Geometry **50**
6.1 Geometry Base 50
6.2 Integer Geometry 51
6.3 Convex Hull and Geometry Routines . 51
6.4 Point in Convex Polygon 52
6.5 Closest Pair 53
6.6 Circle Tangents 53
6.7 Minimum Enclosing Circle 54
6.8 Half-Plane Intersection 54
6.9 Minkowski Sum 55
6.10 Rectangle Union Area 55
6.11 Sweep-Line Event Ordering 56

7 Dynamic Programming **56**
7.1 Fast Knapsack and Subset Sum 56
7.2 Fast Knapsack Formulas 57
7.3 Longest Increasing Subsequence 58
7.4 Meet in the Middle 58
7.5 Bitmask DP 58
7.6 Bitmask DP Formulas 59
7.7 Digit DP 59
7.8 Digit DP State Design 59
7.9 Slope Trick 60
7.10 Slope Trick Formulas 60
7.11 Lagrangian Relaxation (Aliens/WQS) 61
7.12 Lagrangian Relaxation Proof 61
7.13 Knuth Optimization 61
7.14 Divide and Conquer DP 62
7.15 Divide and Conquer DP Notes 62

8 Misc **63**
8.1 Parallel Binary Search 63
8.2 Quick Select 63
8.3 mt19937 RNG 63
8.4 mt19937_64 RNG 64
8.5 Random Hashing 64
8.6 Zobrist Hashing 64
8.7 Custom Hash for unordered_map . . . 64
8.8 Deep Recursion Stack 65
8.9 GNU Extensions 65
8.10 Simulated Annealing Skeleton 66
8.11 Debug Checklist 66

1 Template

1.1 Contest Template

Use: Default C++17 starting file.
Complexity: $O(1)$ setup.
Pitfalls: Uses `#define int long long`; remove multi-test loop if input has one case. Current return expression is intentional source text.
Assumptions/deps: Includes `bits/stdc++.h`, macros, pair printer, and `solve()`.

```cpp
#include<bits/stdc++.h>
using namespace std;
#define int long long
#define F first
#define S second
#define all(x) (x).begin(), (x).end()
// debug printers named dbg, NOT log: a log(x) macro shadows std::log and
// breaks any later math code with confusing errors.
#define dbg(x) cout << #x << " = " << x << " "
#define dbgl(x) cout << #x << " = " << x << endl
#define print(a) cout << #a << " = "; \
for(auto x: a) cout << x << " "; cout << endl;
template<typename T, typename U>
ostream &operator<<(ostream &os, const pair<T, U> &p)
{
    return os << "(" << p.F << "," << p.S << ")";
}

void solve()
{

}
int32_t main()
{
    ios::sync_with_stdio(0); cin.tie(NULL);
    int t; cin >> t; while(t--)
    solve();
    return 8 >> 13;
}
```

2 Data Structures

2.1 Disjoint Set Union

Use: Dynamic connectivity, Kruskal, components.
Complexity: Amortized $O(\alpha(n))$.
Pitfalls: Reinitialize per test case. `unite` returns false when already joined.
Assumptions/deps: 0-indexed; uses `vector`, `iota`.

```cpp
// Disjoint Set Union.
// Complexity: amortized O(alpha(n)).
// Pitfalls: init per test case; unite returns false if already same set.
// Usage: DSU d(n); d.unite(u, v); d.same(u, v); d.size(u);
struct DSU {
    vector<int> p, sz;
    DSU(int n = 0) { init(n); }
    void init(int n) {
        p.resize(n);
        iota(p.begin(), p.end(), 0);
        sz.assign(n, 1);
    }
    int find(int x) {
        return p[x] == x ? x : p[x] = find(p[x]);
    }
    bool unite(int a, int b) {
        a = find(a), b = find(b);
        if (a == b) return false;
        if (sz[a] < sz[b]) swap(a, b);
        p[b] = a;
        sz[a] += sz[b];
        return true;
    }
    bool same(int a, int b) {
        return find(a) == find(b);
    }
    int size(int x) {
        return sz[find(x)];
    }
};
```

2.2 Fenwick Tree

Use: Prefix sums, range sums, kth by prefix.
Complexity: All operations $O(\log n)$.
Pitfalls: External indices are 0-based; `lower_bound` needs nonnegative frequencies.
Assumptions/deps: Uses `long long`; half-open query `[l,r)`.

```cpp
// Fenwick tree for prefix sums.
// Complexity: add/sum/rangeSum/lower_bound are O(log n).
```

```cpp
// Pitfalls: external indices are 0-based; lower_bound assumes all values nonnegative.
// Usage: Fenwick fw(n); fw.add(i, x); fw.rangeSum(l, r); // [l,r)
struct Fenwick {
    int n;
    vector<long long> bit;
    Fenwick(int n = 0) { init(n); }
    void init(int n_) {
        n = n_;
        bit.assign(n + 1, 0);
    }
    void add(int idx, long long val) {
        for (++idx; idx <= n; idx += idx & -idx) bit[idx] += val;
    }
    long long sumPrefix(int r) {
        long long res = 0;
        for (; r > 0; r -= r & -r) res += bit[r];
        return res;
    }
    long long rangeSum(int l, int r) {
        return sumPrefix(r) - sumPrefix(l);
    }
    int lower_bound(long long need) {
        // first idx with sumPrefix(idx+1) >= need, or n if total < need
        if (need <= 0) return 0;
        int idx = 0;
        for (int step = 1 << __lg(max(1, n)); step; step >>= 1) {
            int next = idx + step;
            if (next <= n && bit[next] < need) {
                idx = next;
                need -= bit[next];
            }
        }
        return idx;
    }
};
```

2.3 Sparse Table

Use: Static idempotent range query (min default).
Complexity: Build $O(n \log n)$, query $O(1)$.
Pitfalls: Range is `[l,r)` and must be nonempty; op must be idempotent.
Assumptions/deps: EDIT ONLY `op()` (min/max/gcd/and/or).

```cpp
// Sparse table for static idempotent range query, [l, r) with l < r, O(1).
// Build O(n log n). EDIT ONLY op(): must be idempotent (min/max/gcd/and/or).
struct SparseTable {
    static long long op(long long a, long long b) { return min(a, b); }

    vector<vector<long long>> st;
    vector<int> lg;
    SparseTable() {}
    SparseTable(const vector<long long>& a) { build(a); }
    void build(const vector<long long>& a) {
        int n = (int)a.size();
        lg.assign(n + 1, 0);
        for (int i = 2; i <= n; i++) lg[i] = lg[i / 2] + 1;
        st.assign(lg[n] + 1, vector<long long>(n));
        st[0] = a;
        for (int j = 1; j < (int)st.size(); j++)
            for (int i = 0; i + (1 << j) <= n; i++)
                st[j][i] = op(st[j - 1][i], st[j - 1][i + (1 << (j - 1))]);
    }
    long long query(int l, int r) {
        int j = lg[r - l];
        return op(st[j][l], st[j][r - (1 << j)]);
    }
};
```

2.4 Wavelet Tree

Use: Static range kth/count-less/count-equal queries.
Complexity: Build $O(n \log \sigma)$; queries $O(\log \sigma)$.
Pitfalls: Half-open ranges; kth is 0-based and requires a nonempty range; static only.
Assumptions/deps: Values are internally compressed, including negatives and duplicates.

```cpp
// Static wavelet tree over compressed values. Index ranges are half-open [l,r).
// Complexity: build O(n log sigma); queries O(log sigma); memory O(n log sigma).
// Pitfalls: kth uses 0-based k and needs l<r; static only; values are compressed
// internally, so negatives and duplicates work.
// Usage: WaveletTree wt(a); wt.kth(l,r,k), countLess(l,r,x), countEqual(...).
struct WaveletTree {
    struct Node { int lo,hi,left=-1,right=-1; vector<int>pref; };
    vector<long long>values;
    vector<Node>nodes;
    int root=-1;

    WaveletTree(const vector<long long>& a={}) { init(a); }
    int build(const vector<int>& a,int lo,int hi) {
        int x=nodes.size();nodes.push_back({lo,hi,-1,-1,{}});
        nodes[x].pref.assign(a.size()+1,0);
        if(lo==hi || a.empty()) return x;
        int mid=(lo+hi)/2;
        vector<int>L,R;L.reserve(a.size());R.reserve(a.size());
```

```
        for(int i=0;i<(int)a.size();++i) {
            bool left=a[i]<=mid;
            nodes[x].pref[i+1]=nodes[x].pref[i]+left;
            (left?L:R).push_back(a[i]);
        }
        nodes[x].left=build(L,lo,mid);
        nodes[x].right=build(R,mid+1,hi);
        return x;
    }
    void init(const vector<long long>& a) {
        values=a;sort(values.begin(),values.end());values.erase(unique(values.begin()
            ,values.end()),values.end());
        nodes.clear();root=-1;
        if(values.empty())return;
        vector<int>rank(a.size());
        for(int i=0;i<(int)a.size();++i)rank[i]=lower_bound(values.begin(),values.end
            (),a[i])-values.begin();
        root=build(rank,0,values.size()-1);
    }
    int kthRank(int x,int l,int r,int k) const {
        auto& nd=nodes[x];if(nd.lo==nd.hi)return nd.lo;
        int before=nd.pref[l],inside=nd.pref[r]-before;
        if(k<inside)return kthRank(nd.left,before,nd.pref[r],k);
        return kthRank(nd.right,l-before,r-nd.pref[r],k-inside);
    }
    int countRank(int x,int l,int r,int rank) const {
        if(l>=r)return 0;
        auto& nd=nodes[x];
        if(rank<=nd.lo)return 0;
        if(nd.hi<rank)return r-l;
        int bl=nd.pref[l],br=nd.pref[r];
        return countRank(nd.left,bl,br,rank)+countRank(nd.right,l-bl,r-br,rank);
    }
    long long kth(int l,int r,int k) const {
        assert(root!=-1 && 0<=l && l<r && r<(int)nodes[root].pref.size() && 0<=k && k
            <r-l);
        return values[kthRank(root,l,r,k)];
    }
    int countLess(int l,int r,long long x) const {
        if(root==-1)return 0;
        int rank=lower_bound(values.begin(),values.end(),x)-values.begin();
        return countRank(root,l,r,rank);
    }
    int countEqual(int l,int r,long long x) const {
        if(root==-1)return 0;
        int rank=lower_bound(values.begin(),values.end(),x)-values.begin();
        if(rank==(int)values.size() || values[rank]!=x)return 0;
        return countRank(root,l,r,rank+1)-countRank(root,l,r,rank);
    }
};
```

2.5 Short Segment Tree

Use: Iterative point set and range query.
Complexity: Build $O(n)$, update/query $O(\log n)$.
Pitfalls: Query is half-open `[l,r)`; op must be commutative; `N` fixed at compile time.
Assumptions/deps: EDIT ONLY `op()`/`ID`. Fill `t[n+i]` before `build()`.

```
// Iterative segment tree: point set, range query [l, r). O(log n) per op.
// EDIT ONLY op() and ID. Op must be associative AND commutative
// (sum/min/max/gcd/xor ok; non-commutative like matrix mult needs a
// two-sided query variant).
const int N = 1e5; // limit for array size
int n; // array size
long long t[2 * N];

long long op(long long a, long long b) { return a + b; }
const long long ID = 0; // identity: op(x, ID) == x

void build() { // set t[n..n+i] = a[i] first
  for (int i = n - 1; i > 0; --i) t[i] = op(t[i<<1], t[i<<1|1]);
}

void modify(int p, long long value) { // set value at position p
  for (t[p += n] = value; p > 1; p >>= 1) t[p>>1] = op(t[p], t[p^1]);
}

long long query(int l, int r) { // op over [l, r)
  long long res = ID;
  for (l += n, r += n; l < r; l >>= 1, r >>= 1) {
    if (l&1) res = op(res, t[l++]);
    if (r&1) res = op(res, t[--r]);
  }
  return res;
}
```

2.6 Priority Queue with Delete

Use: Lazy deletion for max heap: `push/erase/top/pop/size`.
Complexity: Amortized $O(\log n)$.
Pitfalls: `top()`/`pop()` need `size()>0`; only erase values currently in the heap.
Assumptions/deps: Uses two `priority_queue<int>`.

```
// Heap with lazy deletion. Only erase values that are currently in the heap.
// top()/pop() require nonempty heap (size() > 0). All ops O(log n).
struct Heap {
  priority_queue<int> in, out;
```

```
  void push(int v) { in.push(v); }
  void erase(int v) { out.push(v); }
  int size() { return in.size() - out.size(); }
  int top() {
    while (out.size() && in.top() == out.top()) in.pop(), out.pop();
    return in.top();
  }
  void pop() { top(); in.pop(); }
};
```

2.7 Lazy Segment Tree

Use: Range update + range query; edit only the marked block for other ops.
Complexity: Build $O(n)$, operations $O(\log n)$.
Pitfalls: Inclusive 0-indexed bounds `[l,r]`. Default is range add / range sum; header lists assign and min variants.
Assumptions/deps: Public wrappers: `update(l,r,f)`, `query(l,r)`, `init(vec)`.

```
// Lazy segment tree. 0-indexed, inclusive ranges [l, r]. O(log n) per op.
// To reuse for another operation, edit ONLY the marked block below:
// types V (node) / L (tag), the two identities, and merge/applyVal/compose.
// Configured for: range-add update, range-sum query.
// range assign : L LID=LLONG_MIN; applyVal -> f*len; compose -> (f==LID?a:f)
// range add, range min : merge=min; applyVal -> a+f (ignore len); compose=a+f
// Usage: SegTree st(n); or st.init(a); st.update(l,r,v); st.query(l,r);
struct SegTree {
    // ---- customize for the operation ----------------------------------
    using V = long long; // node value type
    using L = long long; // lazy tag type
    static constexpr V VID = 0; // merge identity (returned for empty range)
    static constexpr L LID = 0; // "no pending update" tag
    static V merge(V a, V b) { return a + b; } // combine children
    static V applyVal(V a, L f, int len){ return a + f * len; } // tag over len cells
    static L compose(L a, L f) { return a + f; } // apply f on top of a
    // -------------------------------------------------------------------

    int n;
    vector<V> t;
    vector<L> lz;

    SegTree(int n = 0) { init(n); }
    void init(int n_) { n = max(1, n_); t.assign(4 * n, VID); lz.assign(4 * n, LID);
        }
    void init(const vector<V>& a) { init((int)a.size()); build(a, 1, 0, n - 1); }

    void apply(int x, L f, int len) {
        t[x] = applyVal(t[x], f, len);
        lz[x] = compose(lz[x], f);
    }
    void push(int x, int l, int r) {
        if (lz[x] == LID) return;
        int m = (l + r) / 2;
        apply(2 * x, lz[x], m - l + 1);
        apply(2 * x + 1, lz[x], r - m);
        lz[x] = LID;
    }
    void build(const vector<V>& a, int x, int l, int r) {
        if (l == r) { t[x] = a[l]; return; }
        int m = (l + r) / 2;
        build(a, 2 * x, l, m);
        build(a, 2 * x + 1, m + 1, r);
        t[x] = merge(t[2 * x], t[2 * x + 1]);
    }
    void update(int x, int l, int r, int ql, int qr, L f) {
        if (qr < l || r < ql) return;
        if (ql <= l && r <= qr) { apply(x, f, r - l + 1); return; }
        push(x, l, r);
        int m = (l + r) / 2;
        update(2 * x, l, m, ql, qr, f);
        update(2 * x + 1, m + 1, r, ql, qr, f);
        t[x] = merge(t[2 * x], t[2 * x + 1]);
    }
    V query(int x, int l, int r, int ql, int qr) {
        if (qr < l || r < ql) return VID;
        if (ql <= l && r <= qr) return t[x];
        push(x, l, r);
        int m = (l + r) / 2;
        return merge(query(2 * x, l, m, ql, qr), query(2 * x + 1, m + 1, r, ql, qr));
    }
    // public wrappers: call with just the range, no node/bounds by hand
    void update(int l, int r, L f) { update(1, 0, n - 1, l, r, f); }
    V query(int l, int r) { return query(1, 0, n - 1, l, r); }
};
```

2.8 Segment Tree Beats

Use: Range add/chmin/chmax plus sum/min/max.
Complexity: Add/query $O(\log n)$; clamps amortized $O(\log^2 n)$.
Pitfalls: Strict second extrema and strict fast-path comparisons are essential; overflow bounds.
Assumptions/deps: 0-indexed half-open ranges. High-risk; read invariant note below.

```
// Segment tree beats: range add/chmin/chmax and range sum/min/max on [l,r).
// Reference: Ji Ruyi's strict-second-extrema formulation.
```

```cpp
// Complexity: add/query O(log n); chmin/chmax amortized O(log^2 n).
// Pitfalls: mx2/mn2 are STRICT second extrema; comparisons in the fast-path
// conditions must stay strict. Keep values far inside +/-INF; sums
// and lazy additions can overflow long long.
// Usage: SegTreeBeats st(a); st.chmin(l,r,x); st.add(...); st.sum(l,r).
struct SegTreeBeats {
    using ll=long long;
    static constexpr ll INF=(1LL<<60),NINF=-INF;
    struct Node {
        ll sum=0,mx1=NINF,mx2=NINF,mn1=INF,mn2=INF,add=0;
        int mxc=0,mnc=0;
    };
    int n;
    vector<Node>t;
    SegTreeBeats(const vector<ll>&a={}){init(a);}
    Node mergeNode(const Node&a,const Node&b) {
        Node c;c.sum=a.sum+b.sum;
        if(a.mx1==b.mx1)c.mx1=a.mx1,c.mxc=a.mxc+b.mxc,c.mx2=std::max(a.mx2,b.mx2);
        else if(a.mx1>b.mx1)c.mx1=a.mx1,c.mxc=a.mxc,c.mx2=std::max(a.mx2,b.mx1);
        else c.mx1=b.mx1,c.mxc=b.mxc,c.mx2=std::max(a.mx1,b.mx2);
        if(a.mn1==b.mn1)c.mn1=a.mn1,c.mnc=a.mnc+b.mnc,c.mn2=std::min(a.mn2,b.mn2);
        else if(a.mn1<b.mn1)c.mn1=a.mn1,c.mnc=a.mnc,c.mn2=std::min(a.mn2,b.mn1);
        else c.mn1=b.mn1,c.mnc=b.mnc,c.mn2=std::min(a.mn1,b.mn2);
        return c;
    }
    void init(const vector<ll>&a){n=a.size();t.assign(4*std::max(1,n),{});if(n)build(1,0,n,a);}
    void build(int x,int l,int r,const vector<ll>&a){
        if(r-l==1){t[x].sum=t[x].mx1=t[x].mn1=a[l];t[x].mxc=t[x].mnc=1;return;}
        int m=(l+r)/2;build(2*x,l,m,a);build(2*x+1,m,r,a);t[x]=mergeNode(t[2*x],t[2*x+1]);
    }
    void applyAdd(int x,ll v,int len){auto&z=t[x];z.sum+=v*len;z.mx1+=v;if(z.mx2!=NINF)z.mx2+=v;z.mn1+=v;if(z.mn2!=INF)z.mn2+=v;z.add+=v;}
    void applyChmin(int x,ll v){auto&z=t[x];z.sum+=(v-z.mx1)*z.mxc;if(z.mn1==z.mx1)z.mn1=v;else if(z.mn2==z.mx1)z.mn2=v;z.mx1=v;}
    void applyChmax(int x,ll v){auto&z=t[x];z.sum+=(v-z.mn1)*z.mnc;if(z.mx1==z.mn1)z.mx1=v;else if(z.mx2==z.mn1)z.mx2=v;z.mn1=v;}
    void push(int x,int l,int r){
        int m=(l+r)/2;
        if(t[x].add){applyAdd(2*x,t[x].add,m-l);applyAdd(2*x+1,t[x].add,r-m);t[x].add=0;}
        if(t[2*x].mx1>t[x].mx1)applyChmin(2*x,t[x].mx1);
        if(t[2*x+1].mx1>t[x].mx1)applyChmin(2*x+1,t[x].mx1);
        if(t[2*x].mn1<t[x].mn1)applyChmax(2*x,t[x].mn1);
        if(t[2*x+1].mn1<t[x].mn1)applyChmax(2*x+1,t[x].mn1);
    }
    void add(int x,int l,int r,int ql,int qr,ll v){if(qr<=l||r<=ql)return;if(ql<=l&&r<=qr){applyAdd(x,v,r-l);return;}push(x,l,r);int m=(l+r)/2;add(2*x,l,m,ql,qr,v);add(2*x+1,m,r,ql,qr,v);t[x]=mergeNode(t[2*x],t[2*x+1]);}
    void chmin(int x,int l,int r,int ql,int qr,ll v){if(qr<=l||r<=ql||t[x].mx1<=v)return;if(ql<=l&&r<=qr&&t[x].mx2<v){applyChmin(x,v);return;}push(x,l,r);int m=(l+r)/2;chmin(2*x,l,m,ql,qr,v);chmin(2*x+1,m,r,ql,qr,v);t[x]=mergeNode(t[2*x],t[2*x+1]);}
    void chmax(int x,int l,int r,int ql,int qr,ll v){if(qr<=l||r<=ql||t[x].mn1>=v)return;if(ql<=l&&r<=qr&&t[x].mn2>v){applyChmax(x,v);return;}push(x,l,r);int m=(l+r)/2;chmax(2*x,l,m,ql,qr,v);chmax(2*x+1,m,r,ql,qr,v);t[x]=mergeNode(t[2*x],t[2*x+1]);}
    Node query(int x,int l,int r,int ql,int qr){if(ql<=l&&r<=qr)return t[x];push(x,l,r);int m=(l+r)/2;if(qr<=m)return query(2*x,l,m,ql,qr);if(m<=ql)return query(2*x+1,m,r,ql,qr);return mergeNode(query(2*x,l,m,ql,qr),query(2*x+1,m,r,ql,qr));}
    void add(int l,int r,ll v){if(l<r)add(1,0,n,l,r,v);}
    void chmin(int l,int r,ll v){if(l<r)chmin(1,0,n,l,r,v);}
    void chmax(int l,int r,ll v){if(l<r)chmax(1,0,n,l,r,v);}
    ll sum(int l,int r){return l<r?query(1,0,n,l,r).sum:0;}
    ll min(int l,int r){return l<r?query(1,0,n,l,r).mn1:INF;}
    ll max(int l,int r){return l<r?query(1,0,n,l,r).mx1:NINF;}
};
```

2.9 Segment Tree Beats Invariants

Use: Why strict extrema permit lazy clamps.
Complexity: Amortized $O(\log^2 n)$ per clamp.
Pitfalls: Do not change inequality ties without re-deriving invariants.
Assumptions/deps: Pairs with the implementation above.

Stored invariant

Each node stores its sum and two extrema in each direction:

- mx_1: maximum; mx_2: **strict** second maximum; mx_c: count of mx_1.
- mn_1: minimum; mn_2: **strict** second minimum; mn_c: count of mn_1.

Range `chmin(x)`

- $mx_1 \le x$: nothing changes.
- $mx_2 < x < mx_1$: only the mx_c maxima change, so update the node in $O(1)$.
- Otherwise descend; more than one value class would change.

Range `chmax` is symmetric. Push lazy addition first, then clamp each child to the parent's current mx_1 and mn_1. Pull recomputes strict extrema and their counts.

Every failed $O(1)$ clamp can be charged to a strict-extremum level change, giving the usual amortized $O(\log^2 n)$ bound. Never change `<` to `<=` in a fast-path condition without re-deriving the invariant.

2.10 Line Container

Use: Dynamic convex hull trick: add lines, query maximum at x.
Complexity: Amortized $O(\log n)$ add/query.
Pitfalls: Queries MAX only; for min use `add(-k,-m)` and `-get(x)`. Duplicate slopes handled.
Assumptions/deps: Snippet defines `#define int long long`; requires `multiset`, `LLONG_MAX`.

```cpp
// CHT: add lines y = k*x + m, query MAXIMUM at x. Amortized O(log n) per op.
// For MINIMUM: add(-k, -m) and use -get(x).
// Requires notebook-wide "#define int long long" before this snippet;
// without it slopes/intercepts/intersections overflow.
// (for doubles: use inf = 1/.0, div(a,b) = a/b)
#define int long long
struct Line {
  mutable int k, m, p;
  bool operator<(const Line& o) const { return k < o.k; }
  bool operator<(int x) const { return p < x; }
};
struct LineContainer : multiset<Line, less<>> {
  static const int inf = LLONG_MAX;
  int div(int a, int b) { // floored division
    return a / b - ((a ^ b) < 0 && a % b); }
  bool isect(iterator x, iterator y) {
    if (y == end()) return x->p = inf, 0;
    if (x->k == y->k) x->p = x->m > y->m ? inf : -inf;
    else x->p = div(y->m - x->m, x->k - y->k);
    return x->p >= y->p;
  }
  void add(int k, int m) {
    auto z = insert({k, m, 0}), y = z++, x = y;
    while (isect(y, z)) z = erase(z);
    if (x != begin() && isect(--x, y)) isect(x, y = erase(y));
    while ((y = x) != begin() && (--x)->p >= y->p)
      isect(x, erase(y));
  }
  int get(int x) { // max over lines; container must be nonempty
    assert(!empty());
    auto l = *lower_bound(x);
    return l.k * x + l.m;
  }
};
```

2.11 Implicit Treap

Use: Sequence split/merge/reverse by position.
Complexity: Expected $O(\log n)$ per split/merge.
Pitfalls: Must allocate nodes and set `prior,value,cnt,rev,l,r`. Recursion depth randomized but possible.
Assumptions/deps: 0-indexed positions; call `push` before descending.

```cpp
typedef struct item * pitem;
struct item {
    int prior, value, cnt;
    bool rev;
    pitem l, r;
};

int cnt (pitem it) {
    return it ? it->cnt : 0;
}

void upd_cnt (pitem it) {
    if (it)
        it->cnt = cnt(it->l) + cnt(it->r) + 1;
}

void push (pitem it) {
    if (it && it->rev) {
        it->rev = false;
        swap (it->l, it->r);
        if (it->l) it->l->rev ^= true;
        if (it->r) it->r->rev ^= true;
    }
}

void merge (pitem & t, pitem l, pitem r) {
    push (l);
    push (r);
    if (!l || !r)
        t = l ? l : r;
    else if (l->prior > r->prior)
        merge (l->r, l->r, r), t = l;
    else
        merge (r->l, l, r->l), t = r;
    upd_cnt (t);
}

void split (pitem t, pitem & l, pitem & r, int key, int add = 0) {
    if (!t)
        return void( l = r = 0 );
    push (t);
    int cur_key = add + cnt(t->l);
    if (key <= cur_key)
```

```
        split (t->l, l, t->l, key, add), r = t;
    else
        split (t->r, t->r, r, key, add + 1 + cnt(t->l)), l = t;
    upd_cnt (t);
}

void reverse (pitem t, int l, int r) {
    pitem t1, t2, t3;
    split (t, t1, t2, l);
    split (t2, t2, t3, r-l+1);
    t2->rev ^= true;
    merge (t, t1, t2);
    merge (t, t, t3);
}

void output (pitem t) {
    if (!t) return;
    push (t);
    output (t->l);
    printf ("%d ", t->value);
    output (t->r);
}
```

2.12 DSU on Segment Tree

Use: Mergeable frequency/count segment trees, usually per DSU/tree node.
Complexity: Update/query $O(\log V)$; merge proportional to touched nodes.
Pitfalls: Reset `tp=1` and roots between tests. Positions default to `[0, POSMAX]`.
Assumptions/deps: Roots are ints (0 = empty); `merge(u,v)` destroys `v`.

```
// Mergeable dynamic segment trees (sum) over positions [0, POSMAX].
// Each set is a root id (0 = empty). Call with defaulted bounds:
// upd(root[u], pos, +1); query(root[u], l, r); root[u] = merge(root[u], root[u]);
// merge destroys v's tree and returns the combined root. Total O(inserts*logP).
// Pool: 40*maxn nodes fits ~maxn inserts for POSMAX ~ 1e9; shrink to fit ML.
// Pitfalls: reset tp = 1 and all roots to 0 between test cases.
const int maxn = 2e5 + 5, POSMAX = 1e9;
struct node {
  int s, l, r;
  node() : s(0), l(0), r(0) {}
} tree[40 * maxn];
int tp = 1;
void upd(int &u, int pos, int delta, int tl = 0, int tr = POSMAX)
{
  if(!u) { u = tp++; tree[u] = node(); } // fresh node: makes tp reset safe
  tree[u].s += delta;
  if(tl < tr) {
    int tm = tl + (tr - tl) / 2;
    if(pos <= tm) upd(tree[u].l, pos, delta, tl, tm);
    else upd(tree[u].r, pos, delta, tm + 1, tr);
  }
}
int query(int u, int l, int r, int tl = 0, int tr = POSMAX)
{
  if(l > r || u == 0) return 0;
  if(l == tl && r == tr) return tree[u].s;
  int tm = tl + (tr - tl) / 2;
  return query(tree[u].l, l, min(r, tm), tl, tm) +
      query(tree[u].r, max(l, tm + 1), r, tm + 1, tr);
}
int merge(int u, int v, int tl = 0, int tr = POSMAX)
{
  if(!u || !v) return u | v;
  tree[u].s += tree[v].s;
  if(tl < tr) {
    int tm = tl + (tr - tl) / 2;
    tree[u].l = merge(tree[u].l, tree[v].l, tl, tm);
    tree[u].r = merge(tree[u].r, tree[v].r, tm + 1, tr);
  }
  return u;
}
```

2.13 Persistent Segment Tree

Use: Kth smallest / value counts on subarray `a[l..r)`.
Complexity: $O(\log M)$ per op; $O(n \log M)$ memory.
Pitfalls: Coordinate-compress values into `[0,M)` first; `kth` is 1-based.
Assumptions/deps: Usage block in header: build prefix roots, then `kth(root[l],root[r],k)`.

```
// Persistent segment tree: k-th smallest value in a[l..r) in O(log M).
// Values must be in [0, M) (coordinate-compress first). Memory O(n log M).
// Usage:
// PST st(M); vector<int> root(n + 1); root[0] = 0;
// for (i = 0..n-1) root[i+1] = st.upd(root[i], a[i]);
// st.kth(root[l], root[r], k); // k-th smallest (1-based) in a[l..r)
// count of values in [lo,hi) on a[l..r): cnt(root[r],lo,hi)-cnt(root[l],lo,hi).
struct PST {
    struct N { int l, r, cnt; };
    vector<N> t{{0, 0, 0}}; // node 0 = empty tree, usable as initial root
    int M;
    PST(int M) : M(M) { t.reserve(1 << 20); }
    int upd(int u, int pos) { return upd(u, 0, M, pos); }
    int upd(int u, int lo, int hi, int pos) {
        t.push_back(t[u]);
        int v = (int)t.size() - 1;
```

```
        t[v].cnt++;
        if (hi - lo == 1) return v;
        int mid = (lo + hi) / 2;
        if (pos < mid) { int c = upd(t[v].l, lo, mid, pos); t[v].l = c; }
        else { int c = upd(t[v].r, mid, hi, pos); t[v].r = c; }
        return v;
    }
    int kth(int a, int b, int k) { return kth(a, b, 0, M, k); } // roots a < b
    int kth(int a, int b, int lo, int hi, int k) {
        if (hi - lo == 1) return lo;
        int mid = (lo + hi) / 2, cl = t[t[b].l].cnt - t[t[a].l].cnt;
        if (k <= cl) return kth(t[a].l, t[b].l, lo, mid, k);
        return kth(t[a].r, t[b].r, mid, hi, k - cl);
    }
    int cnt(int u, int lo, int hi) { return cnt(u, 0, M, lo, hi); } // vals [lo,hi)
    int cnt(int u, int l, int r, int lo, int hi) {
        if (!u || hi <= l || r <= lo) return 0;
        if (lo <= l && r <= hi) return t[u].cnt;
        int mid = (l + r) / 2;
        return cnt(t[u].l, l, mid, lo, hi) + cnt(t[u].r, mid, r, lo, hi);
    }
};
```

2.14 Monotonic Stack and Queue

Use: Nearest-smaller boundaries and sliding-window maximum.
Complexity: $O(n)$.
Pitfalls: Duplicate tie rule is left $<$ and right $\leq$; change comparisons deliberately.
Assumptions/deps: 0-indexed; deque stores indices.

```
// Monotonic stack boundaries and sliding-window maximum.
// Complexity: O(n); every index enters and leaves once.
// Pitfalls: duplicate tie rule is asymmetric: left < a[i], right <= a[i].
// Usage: auto [left,right]=smaller_bounds(a); auto mx=sliding_max(a,k);
pair<vector<int>, vector<int>> smaller_bounds(const vector<long long>& a) {
    int n = (int)a.size();
    vector<int> left(n, -1), right(n, n), st;
    for (int i = 0; i < n; ++i) {
        while (!st.empty() && a[st.back()] >= a[i]) {
            right[st.back()] = i;
            st.pop_back();
        }
        if (!st.empty()) left[i] = st.back();
        st.push_back(i);
    }
    return {left, right};
}

vector<long long> sliding_max(const vector<long long>& a, int k) {
    if (k <= 0 || k > (int)a.size()) return {};
    deque<int> q;
    vector<long long> ans;
    for (int i = 0; i < (int)a.size(); ++i) {
        while (!q.empty() && q.front() <= i - k) q.pop_front();
        while (!q.empty() && a[q.back()] <= a[i]) q.pop_back();
        q.push_back(i);
        if (i + 1 >= k) ans.push_back(a[q.front()]);
    }
    return ans;
}
```

2.15 Cartesian Tree

Use: Linear construction of min-heap tree whose inorder is the array.
Complexity: $O(n)$.
Pitfalls: Strict pop makes the earlier equal value the ancestor; duplicate policy matters.
Assumptions/deps: Returns root, parent, left, and right arrays.

```
// Min Cartesian tree: inorder is the array; parent value <= child value.
// Complexity: O(n) time/memory using a monotonic stack.
// Pitfalls: strict '>' makes the EARLIER equal value the ancestor; changing
// the comparison changes duplicate tie behavior.
// Usage: auto t=cartesian_tree(a); t.root,t.parent,t.left,t.right.
struct CartesianTree {
    int root=-1;
    vector<int>parent,left,right;
};
CartesianTree cartesian_tree(const vector<long long>& a) {
    int n=a.size();
    CartesianTree t;
    t.parent.assign(n,-1);t.left.assign(n,-1);t.right.assign(n,-1);
    vector<int>st;
    for(int i=0;i<n;++i) {
        int last=-1;
        while(!st.empty() && a[st.back()]>a[i]) last=st.back(),st.pop_back();
        if(!st.empty()) t.right[st.back()]=i,t.parent[i]=st.back();
        if(last!=-1) t.left[i]=last,t.parent[last]=i;
        st.push_back(i);
    }
    if(n) t.root=st.front();
    return t;
}
```

2.16 Sqrt Decomposition

Use: Range update/query with an editable operation block.
Complexity: Build $O(n)$; operations $O(\sqrt{n})$.
Pitfalls: Half-open ranges `[l,r)`; partial blocks push lazy tags.
Assumptions/deps: Default range add/range sum; edit only marked types/operations.

```
// Sqrt decomposition. 0-indexed, half-open ranges [l,r). O(sqrt(n)) per op.
// To reuse for another operation, edit ONLY the marked block below.
// Configured for: range-add update, range-sum query.
// Pitfalls: applyVal must work for both one cell and a whole block;
// L needs equality with LID. Partial operations push the block tag.
// Usage: SqrtDecomp s(a); s.update(l,r,x); long long sum=s.query(l,r);
struct SqrtDecomp {
    // ---- customize for the operation ----------------------------------
    using V = long long;
    using L = long long;
    static constexpr V VID = 0;
    static constexpr L LID = 0;
    static V merge(V a, V b) { return a + b; }
    static V applyVal(V a, L f, int len) { return a + f * len; }
    static L compose(L a, L f) { return a + f; }
    // -------------------------------------------------------------------

    int n, B, blocks;
    vector<V> a, aggregate;
    vector<L> lazy;

    SqrtDecomp(const vector<V>& values) : n(values.size()), a(values) {
        B = max(1, (int)sqrt(max(1, n)) + 1);
        blocks = (n + B - 1) / B;
        aggregate.assign(blocks, VID);
        lazy.assign(blocks, LID);
        for (int i = 0; i < n; ++i)
            aggregate[i / B] = merge(aggregate[i / B], a[i]);
    }
    int block_end(int b) const { return min(n, (b + 1) * B); }

    void rebuild(int b) {
        aggregate[b] = VID;
        for (int i = b * B; i < block_end(b); ++i)
            aggregate[b] = merge(aggregate[b], a[i]);
    }
    void push(int b) {
        if (lazy[b] == LID) return;
        for (int i = b * B; i < block_end(b); ++i)
            a[i] = applyVal(a[i], lazy[b], 1);
        lazy[b] = LID;
    }
    void update(int l, int r, L f) {
        while (l < r) {
            int b = l / B, e = block_end(b);
            if (l == b * B && e <= r) {
                aggregate[b] = applyVal(aggregate[b], f, e - l);
                lazy[b] = compose(lazy[b], f);
                l = e;
            } else {
                push(b);
                int stop = min(r, e);
                while (l < stop) a[l] = applyVal(a[l], f, 1), ++l;
                rebuild(b);
            }
        }
    }
    V query(int l, int r) {
        V ans = VID;
        while (l < r) {
            int b = l / B, e = block_end(b);
            if (l == b * B && e <= r) {
                ans = merge(ans, aggregate[b]);
                l = e;
            } else {
                push(b);
                int stop = min(r, e);
                while (l < stop) ans = merge(ans, a[l++]);
            }
        }
        return ans;
    }
};
```

2.17 Mo's Algorithm

Use: Offline half-open array queries with callback state.
Complexity: $O((n+q)\sqrt{n})$ state moves.
Pitfalls: Add/remove must be exact inverses; state must start empty; basic array version only.
Assumptions/deps: Generic callbacks `add/remove/get`; empty ranges supported.

```
// Mo's algorithm for offline half-open array queries [l,r).
// Complexity: O((n+q)sqrt(n) * add/remove cost), memory O(q).
// Pitfalls: add/remove must be exact inverses; reset state before the call;
// empty ranges are allowed. This is the basic array version only.
// Usage: ans=mo_solve(n,queries,add,remove,get_answer);
template<class Add,class Remove,class Get>
auto mo_solve(int n,const vector<pair<int,int>>& queries,Add add,Remove remove,Get
    get) {
```

```
    using Answer=decltype(get());
    int q=queries.size(),block=max(1,(int)sqrt(max(1,n)));
    vector<int>order(q);iota(order.begin(),order.end(),0);
    sort(order.begin(),order.end(),[&](int i,int j) {
        int bi=queries[i].first/block,bj=queries[j].first/block;
        if(bi!=bj) return bi<bj;
        return (bi&1)?queries[i].second>queries[j].second:
                    queries[i].second<queries[j].second;
    });
    vector<Answer>ans(q);
    int l=0,r=0;
    for(int id:order) {
        auto [ql,qr]=queries[id];
        while(l>ql) add(--l);
        while(r<qr) add(r++);
        while(l<ql) remove(l++);
        while(r>qr) remove(--r);
        ans[id]=get();
    }
    return ans;
}
```

2.18 Link-Cut Tree

Use: Dynamic-forest link/cut/connectivity and path sum.
Complexity: Amortized $O(\log n)$ per operation.
Pitfalls: Cut requires a direct edge; link requires separate trees; splay invariants are high-risk.
Assumptions/deps: Configured for commutative path sum; point-set updates only.

```
// Link-cut tree for a dynamic forest: link/cut/connectivity and path sum.
// Reference: standard Sleator-Tarjan access/makeroot, KACTL-style interface.
// Complexity: amortized O(log n) per operation, memory O(n).
// Pitfalls: link must join different trees; cut removes a DIRECT edge; path
// aggregate is configured as commutative sum. Splay invariants are risky.
// Usage: LinkCut lct(values); lct.link(u,v); lct.pathQuery(u,v); lct.cut(u,v).
struct LinkCut {
    // ---- customize commutative path aggregate here ----------------------
    using V=long long;
    static V identity(){return 0;}
    static V merge(V a,V b){return a+b;}
    // ---------------------------------------------------------------------
    struct Node {int ch[2]={-1,-1},p=-1;bool rev=false;V val=0,agg=0;};
    vector<Node>t;
    LinkCut(int n=0):t(n){for(int i=0;i<n;++i)pull(i);}
    LinkCut(const vector<V>&a):t(a.size()){for(int i=0;i<(int)a.size();++i)t[i].val=
        a[i],pull(i);}
    bool isRoot(int x) const {int p=t[x].p;return p==-1||(t[p].ch[0]!=x&&t[p].ch[1]!=
        x);}
    V aggregate(int x) const{return x==-1?identity():t[x].agg;}
    void pull(int x){t[x].agg=merge(merge(aggregate(t[x].ch[0]),t[x].val),aggregate(
        t[x].ch[1]));}
    void applyReverse(int x){if(x==-1)return;swap(t[x].ch[0],t[x].ch[1]);t[x].rev
        ^=1;}
    void push(int x){if(t[x].rev){applyReverse(t[x].ch[0]);applyReverse(t[x].ch[1]);
        t[x].rev=false;}}
    void pushPath(int x){if(!isRoot(x))pushPath(t[x].p);push(x);}
    void rotate(int x){int p=t[x].p,g=t[p].p,d=t[p].ch[1]==x,b=t[x].ch[d^1];if(!
        isRoot(p))t[g].ch[t[g].ch[1]==p]=x;t[x].p=g;t[x].ch[d^1]=p;t[p].p=x;t[p].ch[
        d]=b;if(b!=-1)t[b].p=p;pull(p);pull(x);}
    void splay(int x){pushPath(x);while(!isRoot(x)){int p=t[x].p,g=t[p].p;if(!isRoot(
        p))rotate((t[p].ch[1]==x)==(t[g].ch[1]==p)?p:x);rotate(x);}pull(x);}
    int access(int x){int last=-1;for(int y=x;y!=-1;y=t[y].p){splay(y);t[y].ch[1]=
        last;pull(y);last=y;}splay(x);return last;}
    void makeRoot(int x){access(x);applyReverse(x);}
    int findRoot(int x){access(x);while(push(x),t[x].ch[0]!=-1)x=t[x].ch[0];splay(x)
        ;return x;}
    bool connected(int u,int v){return u==v||findRoot(u)==findRoot(v);}
    bool link(int u,int v){makeRoot(u);if(findRoot(v)==u)return false;t[u].p=v;return
         true;}
    bool cut(int u,int v){makeRoot(u);access(v);if(t[v].ch[0]!=u||t[u].ch[1]!=-1)
        return false;t[v].ch[0]=-1;t[u].p=-1;pull(v);return true;}
    void setValue(int v,V value){access(v);t[v].val=value;pull(v);}
    V pathQuery(int u,int v){assert(connected(u,v));makeRoot(u);access(v);return t[v
        ].agg;}
};
```

2.19 Link-Cut Tree Operations

Use: Preferred paths, access, makeroot, link, cut, and path query.
Complexity: Amortized $O(\log n)$.
Pitfalls: Push reversals before rotations and pull after child changes.
Assumptions/deps: Explains the narrow implementation above.

Mental model

The represented forest is split into preferred paths, each stored as a splay tree. A splay parent may also be a virtual/path parent when the node is not its actual splay child.

Operations

- `access(v)`: expose the represented-tree path from its root to `v`.
- `makeRoot(v)`: `access(v)`, then lazily reverse the exposed path.
- `link(u,v)`: make `u` a represented-tree root, then attach it under `v`.

- `cut(u,v)`: `makeRoot(u)`, `access(v)`, then verify and remove the direct edge.
- `pathQuery(u,v)`: `makeRoot(u)`, `access(v)`; `v` stores the path aggregate.

Before each rotation, push reversals from ancestors; after changing children, pull aggregates. This version uses commutative path sum. Noncommutative aggregates need forward and reverse values; path updates need another lazy tag.

3 String

3.1 KMP Prefix Function

Use: Borders and single-pattern matching helper.
Complexity: $O(n)$.
Pitfalls: For matching, run on `pattern + sep + text`; choose sep absent from alphabet.
Assumptions/deps: 0-indexed prefix lengths.

```cpp
vector<int> prefix_function(string s) {
    int n = (int)s.length();
    vector<int> pi(n);
    for (int i = 1; i < n; i++) {
        int j = pi[i-1];
        while (j > 0 && s[i] != s[j])
            j = pi[j-1];
        if (s[i] == s[j])
            j++;
        pi[i] = j;
    }
    return pi;
}
```

3.2 Z Function

Use: Longest prefix match starting at each position.
Complexity: $O(n)$.
Pitfalls: For pattern matching, use separator not in alphabet. `z[0]=0`.
Assumptions/deps: 0-indexed string.

```cpp
vector<int> z_function(string s) {
    int n = s.size();
    vector<int> z(n);
    int l = 0, r = 0;
    for(int i = 1; i < n; i++) {
        if(i < r) {
            z[i] = min(r - i, z[i - l]);
        }
        while(i + z[i] < n && s[z[i]] == s[i + z[i]]) {
            z[i]++;
        }
        if(i + z[i] > r) {
            l = i;
            r = i + z[i];
        }
    }
    return z;
}
```

3.3 Aho-Corasick

Use: Count matches of many patterns in text.
Complexity: Build $O(\textit{total length} \cdot A)$, scan $O(n)$.
Pitfalls: Call `build()` after all patterns; out-of-alphabet chars are UB.
Assumptions/deps: EDIT ONLY `A`/`idx()` for other alphabets. Stores counts, not ids.

```cpp
struct Aho {
    // EDIT ONLY A and idx() for other alphabets; out-of-range chars = UB.
    static const int A = 26;
    static int idx(char ch) { return ch - 'a'; }

    struct Node {
        int nxt[A];
        int link = 0;
        int out = 0;

        Node() {
            memset(nxt, -1, sizeof nxt);
        }
    };

    vector<Node> t;

    Aho() {
        t.push_back(Node());
    }

    void add(const string& s) {
        int v = 0;
        for (char ch : s) {
```

```
            int c = idx(ch);
            if (t[v].nxt[c] == -1) {
                t[v].nxt[c] = t.size();
                t.push_back(Node());
            }
            v = t[v].nxt[c];
        }
        t[v].out++;
    }

    void build() {
        queue<int> q;

        for (int c = 0; c < A; c++) {
            int u = t[0].nxt[c];
            if (u == -1) t[0].nxt[c] = 0;
            else q.push(u);
        }

        while (!q.empty()) {
            int v = q.front();
            q.pop();

            t[v].out += t[t[v].link].out;

            for (int c = 0; c < A; c++) {
                int u = t[v].nxt[c];
                if (u == -1) {
                    t[v].nxt[c] = t[t[v].link].nxt[c];
                } else {
                    t[u].link = t[t[v].link].nxt[c];
                    q.push(u);
                }
            }
        }
    }

    long long countMatches(const string& s) {
        int v = 0;
        long long ans = 0;
        for (char ch : s) {
            int c = idx(ch);
            v = t[v].nxt[c];
            ans += t[v].out;
        }
        return ans;
    }
};
```

3.4 Tries

Use: Word/prefix counts and a 63-bit maximum-xor trie.
Complexity: $O(|s|)$ or $O(63)$ per operation.
Pitfalls: Erase only present binary values; word trie default alphabet is lowercase.
Assumptions/deps: Edit marked alphabet block or `MAX_BIT`; reusable public APIs.

```
// Lowercase trie and nonnegative 63-bit xor trie.
// Complexity: O(length) / O(63) per insert, erase, query.
// Pitfalls: lowercase alphabet only; erase only values currently present.
// Usage: Trie t; t.insert(s); BinaryTrie b; b.insert(x); b.max_xor(x).
struct Trie {
    // ---- customize alphabet here ----------------------------------------
    static const int A = 26;
    static int id(char c) { return c-'a'; }
    // ---------------------------------------------------------------------
    struct Node {
        array<int, A> next;
        int pass = 0, end = 0;
        Node() { next.fill(-1); }
    };
    vector<Node> tr{Node()};
    void insert(const string& s) {
        int v = 0; ++tr[v].pass;
        for (char c : s) {
            int x = id(c);
            if (tr[v].next[x] == -1) tr[v].next[x] = tr.size(), tr.emplace_back();
            v = tr[v].next[x]; ++tr[v].pass;
        }
        ++tr[v].end;
    }
    int count_word(const string& s) const {
        int v = 0;
        for (char c : s) {
            int x = id(c);
            if (tr[v].next[x] == -1) return 0;
            v = tr[v].next[x];
        }
        return tr[v].end;
    }
    int count_prefix(const string& s) const {
        int v = 0;
        for (char c : s) {
            int x = id(c);
            if (tr[v].next[x] == -1) return 0;
            v = tr[v].next[x];
        }
        return tr[v].pass;
    }
};
```

```

struct BinaryTrie {
    static const int MAX_BIT = 62;
    struct Node { int ch[2] = {-1, -1}, cnt = 0; };
    vector<Node> tr{Node()};
    void add(unsigned long long x, int delta) {
        int v = 0; tr[v].cnt += delta;
        for (int b = MAX_BIT; b >= 0; --b) {
            int bit = x >> b & 1;
            if (tr[v].ch[bit] == -1) tr[v].ch[bit] = tr.size(), tr.push_back(Node());
            v = tr[v].ch[bit]; tr[v].cnt += delta;
        }
    }
    void insert(unsigned long long x) { add(x, 1); }
    void erase(unsigned long long x) { add(x, -1); }
    unsigned long long max_xor(unsigned long long x) const {
        assert(tr[0].cnt > 0);
        int v = 0; unsigned long long ans = 0;
        for (int b = MAX_BIT; b >= 0; --b) {
            int bit = x >> b & 1, want = bit ^ 1, to = tr[v].ch[want];
            if (to != -1 && tr[to].cnt > 0) ans |= 1ULL << b, v = to;
            else v = tr[v].ch[bit];
        }
        return ans;
    }
};
```

3.5 Rolling Hash

Use: Polynomial substring hash mod $2^{61}-1$, runtime-random base.
Complexity: Build $O(n)$, query $O(1)$.
Pitfalls: NEVER hash mod 2^{64} by overflow: Thue-Morse pairs collide for every base. Half-open `get(l,r)`.
Assumptions/deps: All objects share global base `HB`; uses `__int128`.

```
// Polynomial rolling hash mod 2^61-1 with runtime-random base.
// get(l, r) hashes s[l..r) (HALF-OPEN) in O(1) after O(n) build.
// NEVER hash mod 2^64 via overflow: Thue-Morse strings of length 2048
// collide for EVERY base, and setters include such tests.
// All Hash objects share base HB, so hashes compare across strings.
// Collision prob per pair ~ n/2^61: single hash is fine at ICPC sizes.
const uint64_t HMOD = (1ULL << 61) - 1;
const uint64_t HB =
    mt19937_64(chrono::steady_clock::now().time_since_epoch().count())()
    % (HMOD - 300) + 257;

struct Hash {
    vector<uint64_t> h, p;
    static uint64_t mul(uint64_t a, uint64_t b) {
        __uint128_t c = (__uint128_t)a * b;
        uint64_t r = (uint64_t)(c >> 61) + ((uint64_t)c & HMOD);
        return r >= HMOD ? r - HMOD : r;
    }

    Hash(const string& s) {
        int n = s.size();
        h.assign(n + 1, 0);
        p.assign(n + 1, 1);
        for (int i = 0; i < n; i++) {
            h[i + 1] = mul(h[i], HB) + (unsigned char)s[i] + 1;
            if (h[i + 1] >= HMOD) h[i + 1] -= HMOD;
            p[i + 1] = mul(p[i], HB);
        }
    }

    // hash of s[l..r)
    uint64_t get(int l, int r) {
        uint64_t x = h[r] + HMOD - mul(h[l], p[r - l]);
        return x >= HMOD ? x - HMOD : x;
    }
};
```

3.6 Manacher

Use: Palindrome radii (odd; even via separator trick in header).
Complexity: $O(n)$.
Pitfalls: `p[i]` counts odd palindromes centered at i; longest has length `2p[i]-1`. Sentinels are bytes 1/2, safe for printable input.
Assumptions/deps: For both parities run on `#a#b#c#`; length is `p[i]-1` there.

```
// p[i] = # odd palindromes centered at i; longest has length 2*p[i]-1.
// For BOTH odd and even: run on t = "#a#b#c#" (insert '#' between chars and at
// both ends); then longest palindrome centered in t at i has length p[i]-1.
// Sentinels are bytes 1 and 2, so any printable input is safe (using '$'/'^'
// as sentinels breaks on inputs that contain those characters).
vector<int> manacher_odd(string s) {
    int n = s.size();
    s = char(1) + s + char(2);
    vector<int> p(n + 2);
    int l = 0, r = 1;
    for(int i = 1; i <= n; i++) {
        p[i] = min(r - i, p[l + (r - i)]);
        while(s[i - p[i]] == s[i + p[i]]) {
            p[i]++;
        }
```

```
        if(i + p[i] > r) {
            l = i - p[i], r = i + p[i];
        }
    }
    return vector<int>(begin(p) + 1, end(p) - 1);
}
```

3.7 Palindromic Tree (Eertree)

Use: Online distinct palindromes and occurrence counts.
Complexity: $O(n)$ build for fixed alphabet.
Pitfalls: Add all characters before propagating occurrences; lowercase by default.
Assumptions/deps: Nodes expose length, suffix link, first endpoint, and count.

```
// Palindromic tree (eertree): online distinct palindromes and occurrences.
// Complexity: O(n*A) memory here, O(n) build for fixed alphabet.
// Pitfalls: lowercase only; add all characters before finishOccurrences();
// do not extend after finishing. Nodes 0/1 are roots of length -1/0.
// Usage: Eertree e; int suffix=e.add(c); e.distinct(); e.finishOccurrences().
struct Eertree {
    // ---- customize alphabet here ----------------------------------------
    static const int A=26;
    static int id(char c){return c-'a';}
    // ---------------------------------------------------------------------
    struct Node {
        int len=0,link=0,firstEnd=-1;
        long long occ=0;
        array<int,A>next;
        Node(int length=0):len(length){next.fill(-1);}
    };
    vector<Node>tree;
    vector<int>s;
    string text;
    int suffix;
    bool ready;

    Eertree(const string& str=""){init();for(char c:str)add(c);}
    void init(){tree={Node(-1),Node(0)};tree[0].link=0;tree[1].link=0;s={-1};text.
        clear();suffix=1;ready=false;}
    int getLink(int v,int pos,int c) const {
        while(s[pos-1-tree[v].len]!=c)v=tree[v].link;
        return v;
    }
    int add(char ch) {
        assert(!ready);
        int c=id(ch);assert(0<=c&&c<A);
        text.push_back(ch);s.push_back(c);
        int pos=s.size()-1,cur=getLink(suffix,pos,c);
        if(tree[cur].next[c]==-1) {
            int now=tree.size();tree.push_back(Node(tree[cur].len+2));
            tree[now].firstEnd=text.size()-1;
            if(tree[now].len==1)tree[now].link=1;
            else {int p=getLink(tree[cur].link,pos,c);tree[now].link=tree[p].next[c];}
            tree[cur].next[c]=now;
        }
        suffix=tree[cur].next[c];++tree[suffix].occ;
        return suffix;
    }
    int distinct() const{return max(0,(int)tree.size()-2);}
    int longestSuffix() const{return suffix;}
    string palindrome(int node) const {
        int r=tree[node].firstEnd;
        return text.substr(r-tree[node].len+1,tree[node].len);
    }
    void finishOccurrences() {
        if(ready)return;
        vector<int>order(tree.size());iota(order.begin(),order.end(),0);
        sort(order.begin(),order.end(),[&](int a,int b){return tree[a].len>tree[b].
            len;});
        for(int v:order)if(v>=2)tree[tree[v].link].occ+=tree[v].occ;
        ready=true;
    }
};
```

3.8 Minimum Rotation

Use: Lexicographically minimum cyclic rotation index.
Complexity: $O(n)$.
Pitfalls: For periodic strings returns one minimum index. Empty string not handled.
Assumptions/deps: 0-indexed index in original string.

```
int min_rotation(string s) {
    s += s;
    int n = s.size(), i = 0, ans = 0;

    while (i < n / 2) {
        ans = i;
        int j = i + 1, k = i;

        while (j < n && s[k] <= s[j]) {
            if (s[k] < s[j]) k = i;
            else k++;
            j++;
        }

```

```
        while (i <= k) i += j - k;
    }

    return ans;
}
```

3.9 Suffix Array

Use: Suffix array via cyclic shift sort; wrapper included.
Complexity: $O(n \log n)$.
Pitfalls: `suffix_array(s)` handles the sentinel; input must not contain byte 1. Empty handled.
Assumptions/deps: Alphabet bytes 0..255; pair with Kasai LCP.

```
// Sorts cyclic shifts in O(n log n); suffix_array(s) wrapper below gives the
// suffix array (input must not contain '\1').
vector<int> sort_cyclic_shifts(string const& s) {
    int n = s.size();
    if (n == 0) return {};
    const int alphabet = 256;
    auto code = [&](int i) { return (unsigned char)s[i]; };
    vector<int> p(n), c(n), cnt(max(alphabet, n), 0);
    for (int i = 0; i < n; i++)
        cnt[code(i)]++;
    for (int i = 1; i < alphabet; i++)
        cnt[i] += cnt[i-1];
    for (int i = 0; i < n; i++)
        p[--cnt[code(i)]] = i;
    c[p[0]] = 0;
    int classes = 1;
    for (int i = 1; i < n; i++) {
        if (code(p[i]) != code(p[i-1]))
            classes++;
        c[p[i]] = classes - 1;
    }
    vector<int> pn(n), cn(n);
    for (int h = 0; (1 << h) < n; ++h) {
        for (int i = 0; i < n; i++) {
            pn[i] = p[i] - (1 << h);
            if (pn[i] < 0)
                pn[i] += n;
        }
        fill(cnt.begin(), cnt.begin() + classes, 0);
        for (int i = 0; i < n; i++)
            cnt[c[pn[i]]]++;
        for (int i = 1; i < classes; i++)
            cnt[i] += cnt[i-1];
        for (int i = n-1; i >= 0; i--)
            p[--cnt[c[pn[i]]]] = pn[i];
        cn[p[0]] = 0;
        classes = 1;
        for (int i = 1; i < n; i++) {
            pair<int, int> cur = {c[p[i]], c[(p[i] + (1 << h)) % n]};
            pair<int, int> prev = {c[p[i-1]], c[(p[i-1] + (1 << h)) % n]};
            if (cur != prev)
                ++classes;
            cn[p[i]] = classes - 1;
        }
        c.swap(cn);
    }
    return p;
}

// sa[i] = start of i-th smallest suffix. Use with lcp_array (Kasai) from lcp.cpp.
vector<int> suffix_array(string s) {
    s += '\1'; // sentinel smaller than any input char
    vector<int> p = sort_cyclic_shifts(s);
    return vector<int>(p.begin() + 1, p.end());
}
```

3.10 Kasai LCP

Use: LCP array from string and suffix array.
Complexity: $O(n)$.
Pitfalls: Requires valid suffix array for same string. Output size is `n-1`.
Assumptions/deps: Pairs with suffix array snippet.

```
// Kasai LCP array from string s and suffix array sa.
// lcp[i] = LCP(sa[i], sa[i+1]), size max(0, n-1).
// Time: O(n). Source: standard Kasai algorithm (cp-algorithms/USACO Guide).
vector<int> lcp_array(const string& s, const vector<int>& sa) {
    int n = (int)s.size();
    if (n == 0) return {};
    vector<int> rank(n), lcp(max(0, n - 1));
    for (int i = 0; i < n; i++) rank[sa[i]] = i;
    int h = 0;
    for (int i = 0; i < n; i++) {
        int r = rank[i];
        if (r == n - 1) {
            h = 0;
            continue;
        }
        int j = sa[r + 1];
        while (i + h < n && j + h < n && s[i + h] == s[j + h]) h++;
        lcp[r] = h;
        if (h) h--;
```

```
    }
    return lcp;
}
```

3.11 Suffix Automaton

Use: Substring membership/counts and number of distinct substrings.
Complexity: $O(|s|)$ build for fixed alphabet.
Pitfalls: Add all characters before occurrence propagation; lowercase by default.
Assumptions/deps: Edit marked alphabet block; clones and occurrence propagation included.

```
// Suffix automaton: substring membership, occurrences, distinct substrings.
// Complexity: O(|s|*A) memory here, O(|s|) build/finish for fixed alphabet.
// Pitfalls: lowercase only; add all characters before occurrence queries.
// Usage: SuffixAutomaton sam(s); sam.contains(t); sam.occurrences(t);
struct SuffixAutomaton {
    // ---- customize alphabet here ----------------------------------------
    static const int A = 26;
    static int id(char c) { return c-'a'; }
    // ---------------------------------------------------------------------
    struct State {
        array<int,A> next;
        int link=-1, len=0;
        long long occ=0;
        State() { next.fill(-1); }
    };
    vector<State> st;
    int last;
    bool ready;

    SuffixAutomaton(const string& s="") { init(); for(char c:s) extend(c); }
    void init() { st.assign(1,State()); last=0; ready=false; }
    void extend(char ch) {
        assert(!ready); // occurrence counts have not been propagated yet
        int c=id(ch), cur=st.size(); st.push_back(State());
        st[cur].len=st[last].len+1; st[cur].occ=1;
        int p=last;
        while(p!=-1 && st[p].next[c]==-1) st[p].next[c]=cur,p=st[p].link;
        if(p==-1) st[cur].link=0;
        else {
            int q=st[p].next[c];
            if(st[p].len+1==st[q].len) st[cur].link=q;
            else {
                int clone=st.size(); st.push_back(st[q]);
                st[clone].len=st[p].len+1; st[clone].occ=0;
                while(p!=-1 && st[p].next[c]==q) st[p].next[c]=clone,p=st[p].link;
                st[q].link=st[cur].link=clone;
            }
        }
        last=cur;
    }
    void finish() {
        if(ready) return;
        int mx=0; for(auto& s:st) mx=max(mx,s.len);
        vector<int> cnt(mx+1),order(st.size());
        for(auto& s:st) ++cnt[s.len];
        partial_sum(cnt.begin(),cnt.end(),cnt.begin());
        for(int i=(int)st.size()-1;i>=0;--i) order[--cnt[st[i].len]]=i;
        for(int i=(int)order.size()-1;i>0;--i) {
            int v=order[i]; st[st[v].link].occ+=st[v].occ;
        }
        ready=true;
    }
    int state(const string& s) const {
        int v=0;
        for(char ch:s) { int c=id(ch); if(c<0||c>=A||st[v].next[c]==-1)return -1; v=
            st[v].next[c]; }
        return v;
    }
    bool contains(const string& s) const { return state(s)!=-1; }
    long long occurrences(const string& s) {
        if(s.empty()) return st[last].len+1;
        finish(); int v=state(s); return v==-1?0:st[v].occ;
    }
    long long distinctSubstrings() const {
        long long ans=0;
        for(int v=1;v<(int)st.size();++v) ans+=st[v].len-st[st[v].link].len;
        return ans;
    }
};
```

4 Graph

Checklist: directed or undirected? weights negative? multiple edges/self-loops? clear globals between tests? recursion depth? 0-index input conversion?

4.1 Dijkstra

Use: Single-source shortest paths with nonnegative weights.
Complexity: $O((n+m)\log n)$.
Pitfalls: No negative edges. `INFLL` must exceed max path; `d+w` can overflow if too

close.

Assumptions/deps: Graph as `vector<vector<pair<int,ll>>>`.

```cpp
// Dijkstra shortest paths for nonnegative edge weights.
// Complexity: O((n + m) log n).
// Pitfalls: no negative edges; INF must exceed any possible distance.
// Usage: vector<vector<pair<int,ll>>> g(n); auto d = dijkstra(g, s);
// Path reconstruction: add vector<int> par(n,-1); set par[to]=v next to
// dist[to]=d+w; walk par[] back from t.
using ll = long long;
const ll INFLL = (1LL << 62);

vector<ll> dijkstra(const vector<vector<pair<int, ll>>>& g, int s) {
    int n = (int)g.size();
    vector<ll> dist(n, INFLL);
    priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll, int>>> pq;
    dist[s] = 0;
    pq.push({0, s});
    while (!pq.empty()) {
        auto [d, v] = pq.top();
        pq.pop();
        if (d != dist[v]) continue;
        for (auto [to, w] : g[v]) {
            if (dist[to] > d + w) {
                dist[to] = d + w;
                pq.push({dist[to], to});
            }
        }
    }
    return dist;
}
```

4.2 Bellman–Ford / Difference Constraints

Use: Negative-edge paths, reachable negative cycles, and systems $x_v - x_u \le w$.
Complexity: $O(nm)$.
Pitfalls: Only source-reachable cycles affect shortest paths; feasible assignments must fit `long long`.
Assumptions/deps: Difference constraints return `nullopt` when infeasible.

```cpp
// Bellman-Ford shortest paths with reachable negative-cycle propagation.
// Complexity: O(nm), memory O(n+m).
// Pitfalls: INF/-INF are sentinels; sums are clamped with i128; only cycles
// reachable from source make neg[v] true.
// Usage: auto r=bellman_ford(n,edges,s); r.dist[v], r.neg[v], r.parent[v].
struct BFEdge { int from,to; long long weight; };
struct BFResult {
    static constexpr long long INF=(1LL<<62);
    vector<long long> dist;
    vector<int> parent;
    vector<char> neg;
};
__extension__ typedef __int128 BFInt;

BFResult bellman_ford(int n,const vector<BFEdge>& edges,int source) {
    const long long INF=BFResult::INF;
    const BFInt WINF=(BFInt)1<<120;
    vector<BFInt>d(n,WINF);
    vector<int>parent(n,-1),changed;
    vector<vector<int>>g(n);
    for(auto e:edges) g[e.from].push_back(e.to);
    d[source]=0;
    for(int it=0;it<n;++it) {
        changed.clear();
        for(auto e:edges) if(d[e.from]!=WINF) {
            BFInt cand=d[e.from]+e.weight;
            if(cand<d[e.to]) d[e.to]=cand,parent[e.to]=e.from,changed.push_back(e.to);
        }
        if(changed.empty()) break;
    }
    vector<char>neg(n);
    queue<int>q;
    for(int v:changed) if(!neg[v]) neg[v]=1,q.push(v);
    while(!q.empty()) {int v=q.front();q.pop();for(int u:g[v])if(!neg[u])neg[u]=1,q.
        push(u);}
    vector<long long>dist(n,INF);
    for(int v=0;v<n;++v) {
        if(neg[v])dist[v]=-INF;
        else if(d[v]!=WINF)dist[v]=(long long)max((BFInt)-INF,min((BFInt)INF,d[v]));
    }
    return {dist,parent,neg};
}

// Constraints are x[to] - x[from] <= weight. nullopt means infeasible.
optional<vector<long long>> difference_constraints(int n,const vector<BFEdge>&
    constraints) {
    vector<BFInt>x(n);
    for(int it=0;it<n;++it) {
        bool changed=false;
        for(auto e:constraints) {
            BFInt value=x[e.from]+e.weight;
            if(x[e.to]>value) x[e.to]=value,changed=true;
        }
        if(!changed) {
            vector<long long>answer(n);
            for(int v=0;v<n;++v){assert(LLONG_MIN<=x[v]&&x[v]<=LLONG_MAX);answer[v]=(
                long long)x[v];}
            return answer;
        }
        if(it==n-1) return nullopt;
```

```
    }
    vector<long long>answer(n);return answer;
}
```

4.3 Multi-source and 0-1 BFS

Use: Nearest-source distances and shortest paths with weights 0/1.
Complexity: $O(n+m)$.
Pitfalls: Every weighted edge must be exactly 0 or 1; unreachable values differ by function.
Assumptions/deps: Adjacency-list functions; duplicate sources handled.

```
// Multi-source BFS and 0-1 BFS shortest paths.
// Complexity: O(n+m); -1/INF means unreachable.
// Pitfalls: 0-1 BFS requires every edge weight to be exactly 0 or 1.
// Usage: bfs_multi(g,sources); bfs01(weighted_graph,source);
vector<int> bfs_multi(const vector<vector<int>>& g, const vector<int>& sources) {
    vector<int> d(g.size(), -1);
    queue<int> q;
    for (int s : sources) if (0 <= s && s < (int)g.size() && d[s] == -1)
        d[s] = 0, q.push(s);
    while (!q.empty()) {
        int v = q.front(); q.pop();
        for (int to : g[v]) if (d[to] == -1) d[to] = d[v] + 1, q.push(to);
    }
    return d;
}

vector<long long> bfs01(const vector<vector<pair<int, int>>>& g, int s) {
    const long long INF = (1LL << 62);
    vector<long long> d(g.size(), INF);
    deque<pair<long long, int>> q;
    d[s] = 0; q.push_front({0, s});
    while (!q.empty()) {
        auto [dv, v] = q.front(); q.pop_front();
        if (dv != d[v]) continue;
        for (auto [to, w] : g[v]) if (d[to] > dv + w) {
            d[to] = dv + w;
            if (w == 0) q.push_front({d[to], to});
            else q.push_back({d[to], to});
        }
    }
    return d;
}
```

4.4 Topological Sort

Use: Kahn order and directed-cycle detection.
Complexity: $O(n+m)$.
Pitfalls: An empty result also represents an empty graph; check input size separately if needed.
Assumptions/deps: Parallel edges are handled through indegrees.

```
// Kahn topological sort; empty result means a cycle (or an empty graph).
// Complexity: O(n+m).
// Pitfalls: parallel edges each contribute to indegree; check size for cycles.
// Usage: auto order=topological_sort(g); bool dag=order.size()==g.size();
vector<int> topological_sort(const vector<vector<int>>& g) {
    vector<int> indeg(g.size());
    for (auto& e : g) for (int to : e) ++indeg[to];
    queue<int> q;
    for (int i = 0; i < (int)g.size(); ++i) if (!indeg[i]) q.push(i);
    vector<int> order;
    while (!q.empty()) {
        int v = q.front(); q.pop(); order.push_back(v);
        for (int to : g[v]) if (--indeg[to] == 0) q.push(to);
    }
    if (order.size() != g.size()) return {};
    return order;
}
```

4.5 Floyd–Warshall

Use: All-pairs shortest paths, including negative edges.
Complexity: $O(n^3)$ time, $O(n^2)$ memory.
Pitfalls: Guard INF addition; negative diagonal after the run indicates a negative cycle.
Assumptions/deps: Initialize diagonal to zero and absent edges to INF.

```
// Floyd-Warshall all-pairs shortest paths.
// Complexity: O(n^3) time, O(n^2) memory.
// Pitfalls: guard INF before addition; d[v][v] < 0 means a negative cycle.
// Usage: floyd_warshall(d, INF); initialize diagonal 0 and absent edges INF.
void floyd_warshall(vector<vector<long long>>& d, long long INF) {
    int n = (int)d.size();
    for (int k = 0; k < n; ++k)
        for (int i = 0; i < n; ++i) if (d[i][k] < INF)
            for (int j = 0; j < n; ++j) if (d[k][j] < INF)
                d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
}
```

4.6 Kruskal MST

Use: Minimum spanning forest.
Complexity: $O(m \log m + m\alpha(n))$.
Pitfalls: Disconnected input returns a forest; total weight may overflow.
Assumptions/deps: Requires the DSU snippet.

```cpp
// Kruskal minimum spanning forest; requires the notebook DSU snippet.
// Complexity: O(m log m + m alpha(n)).
// Pitfalls: disconnected input returns a forest; connected iff used == n-1.
// Usage: auto [weight,used]=kruskal_mst(n,edges);
struct MSTEdge { int u, v; long long w; };

pair<long long, int> kruskal_mst(int n, vector<MSTEdge> edges) {
    sort(edges.begin(), edges.end(), [](const MSTEdge& a, const MSTEdge& b) {
        return a.w < b.w;
    });
    DSU dsu(n);
    long long weight = 0;
    int used = 0;
    for (auto e : edges) if (dsu.unite(e.u, e.v)) weight += e.w, ++used;
    return {weight, used};
}
```

4.7 Functional Graph

Use: 64-bit successor jumps, cycle id, entry, and distance to cycle.
Complexity: Build $O(64n)$; jump $O(64)$.
Pitfalls: Every successor must be valid; cycle ids are arbitrary and reset per object.
Assumptions/deps: 0-indexed; supports full unsigned 64-bit jump count.

```cpp
// Functional graph: 64-bit successor jumps plus cycle metadata.
// Complexity: O(n log K) build, O(log K) jump, O(1) metadata queries.
// Pitfalls: next[v] must be valid; dist=0 exactly on a cycle; reset per test.
// Usage: FunctionalGraph f(next); f.go(v,k), f.cycle[v], f.dist[v], f.entry[v].
struct FunctionalGraph {
    static constexpr int LOG = 64;
    vector<vector<int>> up;
    vector<int> cycle, dist, entry;

    FunctionalGraph(const vector<int>& next) {
        int n = (int)next.size();
        up.assign(LOG, vector<int>(n));
        if (n == 0) return;
        up[0] = next;
        for (int b = 1; b < LOG; ++b) for (int v = 0; v < n; ++v)
            up[b][v] = up[b - 1][up[b - 1][v]];

        vector<int> indeg(n), alive(n, 1), order;
        for (int to : next) ++indeg[to];
        queue<int> q;
        for (int v = 0; v < n; ++v) if (!indeg[v]) q.push(v);
        while (!q.empty()) {
            int v = q.front(); q.pop(); alive[v] = 0; order.push_back(v);
            if (--indeg[next[v]] == 0) q.push(next[v]);
        }
        cycle.assign(n, -1); dist.assign(n, -1); entry.assign(n, -1);
        int cid = 0;
        for (int s = 0; s < n; ++s) if (alive[s] && cycle[s] == -1) {
            int v = s;
            do { cycle[v] = cid; dist[v] = 0; entry[v] = v; v = next[v]; } while (v !=
                s);
            ++cid;
        }
        reverse(order.begin(), order.end());
        for (int v : order) {
            int to = next[v];
            cycle[v] = cycle[to]; dist[v] = dist[to] + 1; entry[v] = entry[to];
        }
    }

    int go(int v, unsigned long long k) const {
        for (int b = 0; k; ++b, k >>= 1) if (k & 1) v = up[b][v];
        return v;
    }
};
```

4.8 Strongly Connected Components

Use: Kosaraju SCC + condensation DAG.
Complexity: $O(n + m)$; `condense` $O(m \log m)$.
Pitfalls: Recursive DFS may overflow. Comp ids are topological (edges go low to high id): DP in id order.
Assumptions/deps: Directed graph; call `init` per test case.

```cpp
// Strongly connected components, Kosaraju.
// Complexity: O(n + m); condense() O(m log m).
// Comp ids are in topological order (every edge has comp[u] <= comp[v]),
// so DP over the DAG can just loop components in id order.
// Usage: SCC s(n); s.addEdge(u,v); int c = s.build();
// auto dag = s.condense(c); // deduped SCC DAG, dag[comp] = successors
struct SCC {
    int n;
    vector<vector<int>> g, rg;
```

```
    vector<int> comp;
    SCC(int n = 0) { init(n); }
    void init(int n_) {
        n = n_;
        g.assign(n, {});
        rg.assign(n, {});
        comp.assign(n, -1);
    }
    void addEdge(int u, int v) {
        g[u].push_back(v);
        rg[v].push_back(u);
    }
    int build() {
        vector<int> order, vis(n);
        function<void(int)> dfs1 = [&](int v) {
            vis[v] = 1;
            for (int to : g[v]) if (!vis[to]) dfs1(to);
            order.push_back(v);
        };
        function<void(int, int)> dfs2 = [&](int v, int c) {
            comp[v] = c;
            for (int to : rg[v]) if (comp[to] == -1) dfs2(to, c);
        };
        for (int i = 0; i < n; i++) if (!vis[i]) dfs1(i);
        reverse(order.begin(), order.end());
        int cnt = 0;
        for (int v : order) if (comp[v] == -1) dfs2(v, cnt++);
        return cnt;
    }
    vector<vector<int>> condense(int c) { // call after build()
        vector<vector<int>> dag(c);
        set<pair<int, int>> seen;
        for (int u = 0; u < n; u++)
            for (int v : g[u])
                if (comp[u] != comp[v] && seen.insert({comp[u], comp[v]}).second)
                    dag[comp[u]].push_back(comp[v]);
        return dag;
    }
};
```

4.9 LCA Binary Lifting

Use: LCA, distance, kth ancestor, dfs order `tin`.
Complexity: Build $O(n \log n)$, query $O(\log n)$.
Pitfalls: Call `build(root)`; recursion depth can overflow on large chain.
Assumptions/deps: Graph must be a tree; 0-indexed. `tin` feeds Virtual Tree.

```
// Binary lifting LCA for rooted trees. Also provides dfs-order tin
// (used by virtual_tree.cpp) and kth-ancestor lift.
// Complexity: build O(n log n), query O(log n).
// Pitfalls: call build(root); graph must be a tree; recursion depth on large chains.
// Usage: LCA lca(n); lca.addEdge(u,v)...; lca.build(0); lca.lca(u,v); lca.dist(u,v).
struct LCA {
    int n, LOG, timer = 0;
    vector<vector<int>> g, up;
    vector<int> depth, tin;
    LCA(int n = 0) { init(n); }
    void init(int n_) {
        n = n_;
        LOG = 1;
        while ((1 << LOG) <= max(1, n)) LOG++;
        g.assign(n, {});
        up.assign(LOG, vector<int>(n));
        depth.assign(n, 0);
        tin.assign(n, 0);
        timer = 0;
    }
    void addEdge(int u, int v) {
        g[u].push_back(v);
        g[v].push_back(u);
    }
    void dfs(int v, int p) {
        tin[v] = timer++;
        up[0][v] = p;
        for (int j = 1; j < LOG; j++) up[j][v] = up[j - 1][up[j - 1][v]];
        for (int to : g[v]) if (to != p) {
            depth[to] = depth[v] + 1;
            dfs(to, v);
        }
    }
    void build(int root = 0) {
        depth[root] = 0;
        dfs(root, root);
    }
    int lift(int v, int k) {
        for (int j = 0; j < LOG; j++) if (k >> j & 1) v = up[j][v];
        return v;
    }
    int lca(int a, int b) {
        if (depth[a] < depth[b]) swap(a, b);
        a = lift(a, depth[a] - depth[b]);
        if (a == b) return a;
        for (int j = LOG - 1; j >= 0; j--) if (up[j][a] != up[j][b]) {
            a = up[j][a];
            b = up[j][b];
        }
        return up[0][a];
    }
    int dist(int a, int b) {
        int c = lca(a, b);
```

```
        return depth[a] + depth[b] - 2 * depth[c];
    }
};
```

4.10 Deterministic Tree Canonization

Use: Rooted/unrooted tree-isomorphism ids without hash collisions.
Complexity: $O(n \log n)$.
Pitfalls: Compare trees with the same `TreeCanon` dictionary; recursion depth.
Assumptions/deps: AHU child-id sorting; handles one or two tree centers.

```
// Deterministic AHU-style canonical ids for rooted/unrooted tree isomorphism.
// Complexity: O(n log n) from sorting child ids, memory O(n).
// Pitfalls: compare trees using the SAME TreeCanon object; recursive DFS depth;
// inputs must be trees. No probabilistic collisions.
// Usage: TreeCanon canon; canon.unrooted(g1)==canon.unrooted(g2).
struct TreeCanon {
    map<vector<int>,int> id;
    int get(vector<int> children) {
        sort(children.begin(),children.end());
        auto [it,inserted]=id.emplace(children,id.size());
        return it->second;
    }
    int rootedDfs(const vector<vector<int>>& g,int v,int p) {
        vector<int>children;
        for(int u:g[v]) if(u!=p) children.push_back(rootedDfs(g,u,v));
        return get(children);
    }
    int rooted(const vector<vector<int>>& g,int root) { return rootedDfs(g,root,-1);
         }
    vector<int> centers(const vector<vector<int>>& g) {
        int n=g.size();
        if(n<=2) {vector<int>c(n);iota(c.begin(),c.end(),0);return c;}
        vector<int>deg(n);queue<int>q;
        for(int v=0;v<n;++v) if((deg[v]=g[v].size())<=1)q.push(v);
        int left=n;
        while(left>2) {
            int take=q.size();left-=take;
            while(take--) {int v=q.front();q.pop();for(int u:g[v])if(--deg[u]==1)q.
                 push(u);}
        }
        vector<int>c;while(!q.empty())c.push_back(q.front()),q.pop();return c;
    }
    int unrooted(const vector<vector<int>>& g) {
        if(g.empty()) return -1;
        auto c=centers(g);
        if(c.size()==1) return rooted(g,c[0]);
        int a=rootedDfs(g,c[0],c[1]),b=rootedDfs(g,c[1],c[0]);
        return get({min(a,b),max(a,b)}); // virtual root between the centers
    }
};
```

4.11 Bridges

Use: Find bridges in undirected graph, including multiedges.
Complexity: $O(n + m)$.
Pitfalls: Store each parallel edge separately in `adj`; recursion depth on chains.
Assumptions/deps: Fill `n,adj`; call `find_bridges()`; result in `bridges`.

```
// Bridges of an undirected graph. Fill n and adj, call find_bridges();
// result is in `bridges`. Handles multiedges (a doubled edge is no bridge)
// and disconnected graphs. O(n + m).
// Pitfalls: recursive dfs (~2e5 depth); store each parallel edge separately.
int n; // number of nodes
vector<vector<int>> adj; // adjacency list of graph

vector<pair<int, int>> bridges;
vector<bool> visited;
vector<int> tin, low;
int timer;

void dfs(int v, int p = -1) {
    visited[v] = true;
    tin[v] = low[v] = timer++;
    bool parent_skipped = false;
    for (int to : adj[v]) {
        if (to == p && !parent_skipped) {
            parent_skipped = true;
            continue;
        }
        if (visited[to]) {
            low[v] = min(low[v], tin[to]);
        } else {
            dfs(to, v);
            low[v] = min(low[v], low[to]);
            if (low[to] > tin[v])
                bridges.push_back({v, to});
        }
    }
}

void find_bridges() {
    timer = 0;
    bridges.clear();
    visited.assign(n, false);
    tin.assign(n, -1);
```

```
    low.assign(n, -1);
    for (int i = 0; i < n; ++i) {
        if (!visited[i])
            dfs(i);
    }
}
```

4.12 Articulation Points

Use: Find cut vertices in undirected graph.
Complexity: $O(n+m)$.
Pitfalls: Root is a cutpoint iff it has > 1 DFS children; recursion depth on chains.
Assumptions/deps: Fill `n,adj`; call `find_cutpoints()`; results in `is_cut[]` and `cutpoints`.

```
// Articulation points of an undirected graph. Fill n and adj, call
// find_cutpoints(); result: is_cut[v] flags + `cutpoints` list (no dups).
// Handles disconnected graphs. O(n + m).
// Pitfalls: recursive dfs (~2e5 depth); root is a cutpoint iff children > 1.
int n; // number of nodes
vector<vector<int>> adj; // adjacency list of graph

vector<bool> visited, is_cut;
vector<int> tin, low, cutpoints;
int timer;

void dfs(int v, int p = -1) {
    visited[v] = true;
    tin[v] = low[v] = timer++;
    int children = 0;
    for (int to : adj[v]) {
        if (to == p) continue;
        if (visited[to]) {
            low[v] = min(low[v], tin[to]);
        } else {
            dfs(to, v);
            low[v] = min(low[v], low[to]);
            if (low[to] >= tin[v] && p != -1)
                is_cut[v] = true;
            ++children;
        }
    }
    if (p == -1 && children > 1)
        is_cut[v] = true;
}

void find_cutpoints() {
    timer = 0;
    visited.assign(n, false);
    is_cut.assign(n, false);
    tin.assign(n, -1);
    low.assign(n, -1);
    cutpoints.clear();
    for (int i = 0; i < n; ++i) {
        if (!visited[i])
            dfs(i);
    }
    for (int i = 0; i < n; ++i)
        if (is_cut[i]) cutpoints.push_back(i);
}
```

4.13 Block-Cut Tree

Use: Vertex-biconnected blocks and their incidence forest.
Complexity: $O(n+m)$ excluding vertex-list sorting.
Pitfalls: Use edge ids; parallel edges work, self-loops do not; recursion depth.
Assumptions/deps: Original nodes first, then block nodes; isolated vertices remain isolated.

```
// Vertex-biconnected components and block-cut forest of an undirected graph.
// Original vertices are 0..n-1; block nodes are n..n+blocks.size()-1.
// Complexity: O(n+m) plus sorting vertices inside blocks, O(n+m) memory.
// Pitfalls: use edge ids (parallel edges work); self-loops are not supported;
// recursive DFS can overflow. Isolated vertices stay isolated.
// Usage: BlockCutTree bc(n); bc.addEdge(u,v); bc.build();
struct BlockCutTree {
    int n, timer;
    vector<pair<int,int>> edges;
    vector<vector<pair<int,int>>> g;
    vector<int> tin, low, edgeStack;
    vector<vector<int>> blocks, tree;

    BlockCutTree(int n = 0) { init(n); }
    void init(int n_) {
        n=n_; timer=0; edges.clear(); g.assign(n,{});
        tin.assign(n,-1); low.resize(n); edgeStack.clear();
        blocks.clear(); tree.clear();
    }
    int addEdge(int u,int v) {
        int id=edges.size(); edges.push_back({u,v});
        g[u].push_back({v,id}); g[v].push_back({u,id});
        return id;
    }
    void makeBlock(int lastEdge) {
        vector<int> vertices;
        while (true) {
```

```
            int id=edgeStack.back(); edgeStack.pop_back();
            vertices.push_back(edges[id].first);
            vertices.push_back(edges[id].second);
            if (id==lastEdge) break;
        }
        sort(vertices.begin(),vertices.end());
        vertices.erase(unique(vertices.begin(),vertices.end()),vertices.end());
        blocks.push_back(vertices);
    }
    void dfs(int v,int parentEdge) {
        tin[v]=low[v]=timer++;
        for (auto [u,id]:g[v]) {
            if (id==parentEdge) continue;
            if (tin[u]==-1) {
                edgeStack.push_back(id); dfs(u,id);
                low[v]=min(low[v],low[u]);
                if (low[u]>=tin[v]) makeBlock(id);
            } else if (tin[u]<tin[v]) {
                edgeStack.push_back(id);
                low[v]=min(low[v],tin[u]);
            }
        }
    }
    void build() {
        timer=0; fill(tin.begin(),tin.end(),-1); edgeStack.clear(); blocks.clear();
        for (int v=0;v<n;++v) if (tin[v]==-1) dfs(v,-1);
        tree.assign(n+blocks.size(),{});
        for (int b=0;b<(int)blocks.size();++b) for (int v:blocks[b]) {
            tree[n+b].push_back(v); tree[v].push_back(n+b);
        }
    }
};
```

4.14 Dinic Max Flow

Use: Directed max flow and min cut.
Complexity: General fast in practice; unit capacities have better listed bounds.
Pitfalls: Edges are directed; add both directions for undirected capacity. Capacities are `long long`.
Assumptions/deps: Requires `sz`, `rep`, `bits/stdc++.h`/STL.

```
// * $O(\min(E^{1/2}, V^{2/3})E)$ if $U = 1$; $O(\sqrt{V}E)$ for bipartite matching.
// After calc(s,t), leftOfMinCut(v) gives the source side; minCutEdges() lists
     crossing edges.
// Requires bits/stdc++.h and using namespace std.
using ll = long long;
using vi = vector<int>;
#ifndef sz
#define sz(x) (int)(x).size()
#endif
#ifndef rep
#define rep(i,a,b) for (int i = (a); i < (b); ++i)
#endif

struct Dinic {
  struct Edge {
    int to, rev;
    ll c, oc;
    ll flow() { return max(oc - c, 0LL); } // if you need flows
  };
  vi lvl, ptr, q;
  vector<vector<Edge>> adj;
  Dinic(int n) : lvl(n), ptr(n), q(n), adj(n) {}
  void addEdge(int a, int b, ll c, ll rcap = 0) {
    adj[a].push_back({b, sz(adj[b]), c, c});
    adj[b].push_back({a, sz(adj[a]) - 1, rcap, rcap});
  }
  ll dfs(int v, int t, ll f) {
    if (v == t || !f) return f;
    for (int& i = ptr[v]; i < sz(adj[v]); i++) {
      Edge& e = adj[v][i];
      if (lvl[e.to] == lvl[v] + 1)
        if (ll p = dfs(e.to, t, min(f, e.c))) {
          e.c -= p, adj[e.to][e.rev].c += p;
          return p;
        }
    }
    return 0;
  }
  ll calc(int s, int t) {
    ll flow = 0; q[0] = s;
    rep(L,0,31) do { // 'int L=30' maybe faster for random data
      lvl = ptr = vi(sz(q));
      int qi = 0, qe = lvl[s] = 1;
      while (qi < qe && !lvl[t]) {
        int v = q[qi++];
        for (Edge e : adj[v])
          if (!lvl[e.to] && e.c >> (30 - L))
            q[qe++] = e.to, lvl[e.to] = lvl[v] + 1;
      }
      while (ll p = dfs(s, t, LLONG_MAX)) flow += p;
    } while (lvl[t]);
    return flow;
  }
  bool leftOfMinCut(int a) { return lvl[a] != 0; }
  vector<pair<int, int>> minCutEdges() {
    vector<pair<int, int>> cut;
    rep(v,0,sz(adj)) if (leftOfMinCut(v)) {
      for (Edge& e : adj[v])
        if (e.oc > 0 && e.c == 0 && !leftOfMinCut(e.to))
```

```
            cut.push_back({v, e.to});
        }
        return cut;
    }
};
```

4.15 Maximum-Weight Closure

Use: Choose profitable vertices under directed implications.
Complexity: One max-flow on $n + 2$ vertices.
Pitfalls: INF must exceed every finite cut; implication is directed.
Assumptions/deps: Requires Dinic immediately above.

```
// Maximum-weight closure via one s-t min cut. Depends on Dinic.
// If u is chosen and (u,v) is an implication, v must also be chosen.
// Complexity: one max-flow on n+2 vertices and O(n+m) edges.
// Pitfalls: MCF_INF must exceed total absolute weight; directed implications.
// Usage: auto [best,chosen]=max_weight_closure(weight,implications);
const long long MCF_INF = (1LL<<60);
struct MaxClosureResult { long long value; vector<int> chosen; };

MaxClosureResult max_weight_closure(
    const vector<long long>& weight, const vector<pair<int,int>>& implications) {
    int n=weight.size(), s=n, t=n+1;
    Dinic flow(n+2);
    long long positive=0;
    for (int v=0;v<n;++v) {
        if (weight[v]>0) flow.addEdge(s,v,weight[v]), positive+=weight[v];
        if (weight[v]<0) flow.addEdge(v,t,-weight[v]);
    }
    for (auto [u,v]:implications) flow.addEdge(u,v,MCF_INF);
    long long best=positive-flow.calc(s,t);
    vector<int> chosen;
    for (int v=0;v<n;++v) if (flow.leftOfMinCut(v)) chosen.push_back(v);
    return {best,chosen};
}
```

4.16 Min-Cut Modeling Recipes

Use: Closure, bipartite vertex cover, and binary labeling reductions.
Complexity: One max-flow after graph construction.
Pitfalls: Cut-side meaning and INF choice are common sources of reversed answers.
Assumptions/deps: Pair with Dinic and residual source-side query.

Choose INF strictly larger than every feasible finite cut, often $1 + \sum_v |cost_v|$. After max flow, residual-reachable vertices form the source side of a minimum cut.

Maximum-weight closure

- $profit_v > 0$: edge $s \to v$ with capacity $profit_v$.
- $profit_v < 0$: edge $v \to t$ with capacity $-profit_v$.
- Choosing u requires v: edge $u \to v$ with capacity INF.
- Answer: $\sum_v \max(0, profit_v) - \text{mincut}$.

Bipartite minimum-weight vertex cover

- $s \to u$ for each left vertex; capacity is its weight.
- $v \to t$ for each right vertex; capacity is its weight.
- For every graph edge, add $u \to v$ with capacity INF.
- Cover = left vertices outside the source side + right vertices inside it.

Binary labeling

Use $s \to v$ for the cost of label 1, $v \to t$ for label 0, and undirected capacity w for disagreement penalty w.
Pitfalls: an INF edge is hard only if no optimum can cut it. Decide which side means "chosen." General pairwise rewards require submodularity and may not be graph-representable.

4.17 Min Cost Flow

Use: Successive shortest path with potentials.
Complexity: $O(FE \log V)$ plus Bellman-Ford init.
Pitfalls: Negative costs ok, but no negative cost cycles. May return less than requested flow.
Assumptions/deps: Cap/cost are `long long`; directed edges.

```
// Min-cost flow with potentials. Negative costs ok; no negative cost cycles.
// Source: Stanford/KACTL successive shortest path style.
// Complexity: O(F E log V), plus O(VE) Bellman-Ford init.
// Pitfalls: costs/caps are long long; minCost(s,t,need) may send < need flow.
// Usage: MCMF mf(n); mf.addEdge(u,v,cap,cost); auto [flow,cost]=mf.minCost(s,t,need)
    ;
struct MCMF {
    using ll = long long;
    const ll INF = (1LL << 62);
    struct Edge {
        int to, rev;
        ll cap, cost;
    };
```

```
    int n;
    vector<vector<Edge>> g;
    vector<ll> pot, dist;
    vector<int> pv, pe;

    MCMF(int n = 0) { init(n); }
    void init(int n_) {
        n = n_;
        g.assign(n, {});
        pot.assign(n, 0);
        dist.resize(n);
        pv.resize(n);
        pe.resize(n);
    }
    void addEdge(int v, int to, ll cap, ll cost) {
        if (cap <= 0) return;
        Edge a{to, (int)g[to].size(), cap, cost};
        Edge b{v, (int)g[v].size(), 0, -cost};
        g[v].push_back(a);
        g[to].push_back(b);
    }
    void setPotentials(int s) {
        fill(pot.begin(), pot.end(), INF);
        pot[s] = 0;
        for (int it = 0; it < n - 1; it++) {
            bool changed = false;
            for (int v = 0; v < n; v++) if (pot[v] != INF) {
                for (Edge& e : g[v]) if (e.cap && pot[e.to] > pot[v] + e.cost) {
                    pot[e.to] = pot[v] + e.cost;
                    changed = true;
                }
            }
            if (!changed) break;
        }
        for (ll& x : pot) if (x == INF) x = 0;
    }
    pair<ll, ll> minCost(int s, int t, ll need = (1LL << 62)) {
        setPotentials(s);
        ll flow = 0, cost = 0;
        while (flow < need) {
            fill(dist.begin(), dist.end(), INF);
            priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll, int
                >>> pq;
            dist[s] = 0;
            pq.push({0, s});
            while (!pq.empty()) {
                auto [d, v] = pq.top();
                pq.pop();
                if (d != dist[v]) continue;
                for (int i = 0; i < (int)g[v].size(); i++) {
                    Edge& e = g[v][i];
                    if (!e.cap) continue;
                    ll nd = d + e.cost + pot[v] - pot[e.to];
                    if (nd < dist[e.to]) {
                        dist[e.to] = nd;
                        pv[e.to] = v;
                        pe[e.to] = i;
                        pq.push({nd, e.to});
                    }
                }
            }
            if (dist[t] == INF) break;
            for (int v = 0; v < n; v++) if (dist[v] != INF) pot[v] += dist[v];
            ll add = need - flow;
            for (int v = t; v != s; v = pv[v]) add = min(add, g[pv[v]][pe[v]].cap);
            for (int v = t; v != s; v = pv[v]) {
                Edge& e = g[pv[v]][pe[v]];
                e.cap -= add;
                g[v][e.rev].cap += add;
                cost += add * e.cost;
            }
            flow += add;
        }
        return {flow, cost};
    }
};
```

4.18 Kuhn Bipartite Matching

Use: Maximum bipartite matching for smaller/simple cases.
Complexity: Usually $O(VE)$ worst case.
Pitfalls: Left side size `n`, right side `m`. Konig notes in header.
Assumptions/deps: Large graphs: Dinic with unit caps ($O(E\sqrt{V})$, = Hopcroft-Karp).

```
// Kuhn max bipartite matching, left size n / right size m. O(V*E) worst case
// but fast in practice with the repeat loop. matching() returns (left,right)
// pairs; match[r] = left partner of r or -1.
// Large/adversarial graphs: use Dinic (dinic_2.cpp) with unit capacities
// source->left->right->sink: O(E sqrt V), same as Hopcroft-Karp.
// Konig: min vertex cover size = max matching; complement = max independent set.
struct bipartite {
    int n, m;
    vector<vector<int>> g;
    vector<int> match;

    bipartite(int n, int m): n(n), m(m), g(n), match(m, -1) {}

    void add(int a, int b) {
        g[a].push_back(b);
```

```
    }

    vector<bool> visited;
    bool kuhn(int v) {
        if(visited[v]) {
            return false;
        }
        visited[v] = true;
        for(auto u: g[v]) {
            if(match[u] == -1 || kuhn(match[u])) {
                match[u] = v;
                return true;
            }
        }
        return false;
    }

    auto matching() {
        vector<bool> paired(n);
        for(bool repeat = true; repeat;) {
            repeat = false;
            visited.assign(n, 0);
            for(int i = 0; i < n; i++) {
                if(!paired[i]) {
                    paired[i] = kuhn(i);
                    repeat |= paired[i];
                }
            }
        }
        vector<pair<int, int>> ans;
        for(int u = 0; u < m; u++) {
            if(match[u] != -1) {
                ans.emplace_back(match[u], u);
            }
        }
        return ans;
    }
};
```

4.19 Hungarian Assignment

Use: Minimum-cost matching of every row to a distinct column.
Complexity: $O(rows^2 \cdot cols)$.
Pitfalls: Requires rows $\leq$ columns; minimizes; cost/potentials must fit `long long`.
Assumptions/deps: Rectangular matrices and negative costs supported.

```
// Minimum-cost assignment: match every row to a distinct column (rows<=cols).
// Complexity: O(rows^2 * cols), memory O(rows+cols).
// Pitfalls: minimizes cost; matrix must be rectangular and rows<=cols;
// potentials/cost total need long long. Negative costs work.
// Usage: auto [cost,column_of_row]=hungarian(cost_matrix);
pair<long long,vector<int>> hungarian(const vector<vector<long long>>& a) {
    int n=a.size();
    if(!n) return {0,{}};
    int m=a[0].size();
    assert(n<=m && m>0);
    for(auto& row:a) assert((int)row.size()==m);
    const long long INF=(1LL<<62);
    vector<long long>u(n+1),v(m+1),minv(m+1);
    vector<int>p(m+1),way(m+1);
    for(int i=1;i<=n;++i) {
        p[0]=i;
        int j0=0;
        vector<char>used(m+1);
        fill(minv.begin(),minv.end(),INF);
        do {
            used[j0]=1;
            int i0=p[j0],j1=0;
            long long delta=INF;
            for(int j=1;j<=m;++j) if(!used[j]) {
                long long cur=a[i0-1][j-1]-u[i0]-v[j];
                if(cur<minv[j]) minv[j]=cur,way[j]=j0;
                if(minv[j]<delta) delta=minv[j],j1=j;
            }
            for(int j=0;j<=m;++j) {
                if(used[j]) u[p[j]]+=delta,v[j]-=delta;
                else minv[j]-=delta;
            }
            j0=j1;
        } while(p[j0]!=0);
        do {
            int j1=way[j0];p[j0]=p[j1];j0=j1;
        } while(j0);
    }
    vector<int>column(n,-1);
    for(int j=1;j<=m;++j) if(p[j]) column[p[j]-1]=j-1;
    return {-v[0],column};
}
```

4.20 2-SAT

Use: Satisfiability + assignment for 2-CNF; implications, at-most-one.
Complexity: $O(n + clauses)$.
Pitfalls: Use `~x` for NOT x in calls; recursive SCC depth $\sim 2n$; `atMostOne` adds vars.

Assumptions/deps: `either(a,b)` = clause $(a \vee b)$; after `solve()`, `values[x]` is 0/1.

```cpp
// 2-SAT. Variables 0..n-1; use ~x (i.e. -x-1) for "NOT x" in clause calls.
// either(a,b) adds clause (a OR b). solve() = satisfiable; values[x] in {0,1}.
// Source: KACTL TwoSat. Time: O(n + clauses).
// Pitfalls: recursive SCC (stack depth ~ 2n); atMostOne adds new vars.
struct TwoSat {
    int N;
    vector<vector<int>> gr;
    vector<int> values; // 0 = false, 1 = true

    TwoSat(int n = 0) : N(n), gr(2 * n) {}
    int addVar() { gr.emplace_back(); gr.emplace_back(); return N++; }
    void either(int f, int j) {
        f = max(2 * f, -1 - 2 * f);
        j = max(2 * j, -1 - 2 * j);
        gr[f].push_back(j ^ 1);
        gr[j].push_back(f ^ 1);
    }
    void setValue(int x) { either(x, x); }
    void atMostOne(const vector<int>& li) { // at most one of li is true
        if ((int)li.size() <= 1) return;
        int cur = ~li[0];
        for (int i = 2; i < (int)li.size(); i++) {
            int next = addVar();
            either(cur, ~li[i]);
            either(cur, next);
            either(~li[i], next);
            cur = ~next;
        }
        either(cur, ~li[1]);
    }
    vector<int> val, comp, z;
    int timer = 0;
    int dfs(int i) {
        int low = val[i] = ++timer, x;
        z.push_back(i);
        for (int e : gr[i]) if (!comp[e])
            low = min(low, val[e] ? val[e] : dfs(e));
        if (low == val[i]) do {
            x = z.back(); z.pop_back();
            comp[x] = low;
            if (values[x >> 1] == -1) values[x >> 1] = x & 1;
        } while (x != i);
        return val[i] = low;
    }
    bool solve() {
        values.assign(N, -1);
        val.assign(2 * N, 0);
        comp = val;
        for (int i = 0; i < 2 * N; i++) if (!comp[i]) dfs(i);
        for (int i = 0; i < N; i++) if (comp[2 * i] == comp[2 * i + 1]) return false;
        return true;
    }
};
```

4.21 Euler Walk

Use: Eulerian path/circuit, directed or undirected.
Complexity: $O(V + E)$, iterative.
Pitfalls: Returns {} if none exists from chosen start. Undirected: push both directions with SAME edge id.
Assumptions/deps: Start vertex rules in header; ids `0..nedges-1`.

```cpp
// Eulerian path/circuit starting at src; returns node sequence (nedges+1
// nodes) or {} if none exists FROM src. Works for directed and undirected.
// gr[u] holds {v, edge_id}. Directed: push once. Undirected: push both
// directions with the SAME edge id. Ids must be 0..nedges-1.
// For a circuit pick any src with an edge; for a path start at an odd-degree
// (undirected) / out-in=1 (directed) vertex. Graph edges must be connected.
// Source: KACTL EulerWalk. Time: O(V + E). Iterative, no stack issues.
vector<int> eulerWalk(vector<vector<pair<int, int>>>& gr, int nedges, int src = 0) {
    int n = gr.size();
    vector<int> D(n), its(n), eu(nedges), ret, s = {src};
    D[src]++; // to allow Euler paths, not just cycles
    while (!s.empty()) {
        int x = s.back(), y, e, &it = its[x], end = gr[x].size();
        if (it == end) { ret.push_back(x); s.pop_back(); continue; }
        tie(y, e) = gr[x][it++];
        if (!eu[e]) {
            D[x]--, D[y]++;
            eu[e] = 1;
            s.push_back(y);
        }
    }
    for (int x : D) if (x < 0 || (int)ret.size() != nedges + 1) return {};
    return {ret.rbegin(), ret.rend()};
}
```

4.22 Heavy Light Decomposition

Use: Path/subtree update and query on tree; op comes from SegTree config.
Complexity: $O(\log^2 n)$ path operations.
Pitfalls: Roots at 0. `VALS_EDGES=true`: edge values at deeper endpoint, LCA skipped. Non-commutative merge unsafe on paths.

Assumptions/deps: Print Lazy Segment Tree first and configure its marked block.

```cpp
// HLD: path/subtree update+query in O(log^2 n) on top of SegTree from
// lazy_segment_tree.cpp (print that first; configure its marked block for
// the needed op -default range add / range sum works as-is).
// 0-indexed, rooted at 0. VALS_EDGES=false: values on vertices, path includes
// both endpoints. VALS_EDGES=true: values on edges, stored at the DEEPER
// endpoint; the LCA vertex is skipped automatically.
// Usage: HLD<false> h(adj); h.modifyPath(u,v,x); h.queryPath(u,v);
// h.modifySubtree(v,x); h.querySubtree(v); pos[v] = index in tree.
// Pitfalls: recursive dfs on ~2e5 chain; for a non-commutative SegTree::merge
// queryPath combines chain segments in arbitrary order (sum/min/max are fine).
template <bool VALS_EDGES> struct HLD {
    int N, tim = 0;
    vector<vector<int>> adj;
    vector<int> par, siz, rt, pos;
    SegTree tree;
    HLD(vector<vector<int>> adj_)
        : N(adj_.size()), adj(adj_), par(N, -1), siz(N, 1),
          rt(N), pos(N), tree(N) { dfsSz(0); dfsHld(0); }
    void dfsSz(int v) {
        if (par[v] != -1) adj[v].erase(find(adj[v].begin(), adj[v].end(), par[v]));
        for (int& u : adj[v]) {
            par[u] = v;
            dfsSz(u);
            siz[v] += siz[u];
            if (siz[u] > siz[adj[v][0]]) swap(u, adj[v][0]);
        }
    }
    void dfsHld(int v) {
        pos[v] = tim++;
        for (int u : adj[v]) {
            rt[u] = (u == adj[v][0] ? rt[v] : u);
            dfsHld(u);
        }
    }
    template <class B> void process(int u, int v, B op) { // op gets [l, r), may be
         empty
        for (; rt[u] != rt[v]; v = par[rt[v]]) {
            if (pos[rt[u]] > pos[rt[v]]) swap(u, v);
            op(pos[rt[v]], pos[v] + 1);
        }
        if (pos[u] > pos[v]) swap(u, v);
        op(pos[u] + VALS_EDGES, pos[v] + 1);
    }
    void modifyPath(int u, int v, SegTree::L x) {
        process(u, v, [&](int l, int r) { if (l < r) tree.update(l, r - 1, x); });
    }
    SegTree::V queryPath(int u, int v) {
        SegTree::V res = SegTree::VID;
        process(u, v, [&](int l, int r) {
            if (l < r) res = SegTree::merge(res, tree.query(l, r - 1));
        });
        return res;
    }
    void modifySubtree(int v, SegTree::L x) {
        int l = pos[v] + VALS_EDGES, r = pos[v] + siz[v];
        if (l < r) tree.update(l, r - 1, x);
    }
    SegTree::V querySubtree(int v) {
        int l = pos[v] + VALS_EDGES, r = pos[v] + siz[v];
        return l < r ? tree.query(l, r - 1) : SegTree::VID;
    }
};
```

4.23 Rerooting DP

Use: Compute a tree DP answer for every possible root.
Complexity: $O(n)$ for constant-time DP operations.
Pitfalls: Merge must be associative; recursion depth; input must be a tree.
Assumptions/deps: Default is sum of distances; edit only marked DP operations.

```cpp
// Rerooting DP. Configured to compute sum of distances from every vertex.
// To reuse, edit ONLY DP/ID and merge/addEdge/addRoot in the marked block.
// Complexity: O(n) time and memory for constant-time DP operations.
// Pitfalls: merge must be associative; recursive DFS can overflow; tree only.
// Usage: Rerooting rr(g); vector<DP> answer=rr.solve(0);
struct Rerooting {
    // ---- customize the DP ------------------------------------------------
    struct DP { long long size, sum; };
    static DP id() { return {0,0}; }
    static DP merge(DP a,DP b) { return {a.size+b.size,a.sum+b.sum}; }
    static DP addEdge(DP a,int /*from*/,int /*to*/) { return {a.size,a.sum+a.size}; }
    static DP addRoot(DP a,int /*v*/) { ++a.size; return a; }
    // ----------------------------------------------------------------------

    int n;
    const vector<vector<int>>& g;
    vector<DP> down, answer;
    Rerooting(const vector<vector<int>>& graph):n(graph.size()),g(graph),down(n),
         answer(n){}

    DP dfsDown(int v,int p) {
        DP cur=id();
        for (int u:g[v]) if (u!=p) cur=merge(cur,addEdge(dfsDown(u,v),u,v));
        return down[v]=addRoot(cur,v);
    }
    void dfsAll(int v,int p,DP fromParent) {
```

```
        int d=g[v].size();
        vector<DP> side(d),pre(d+1,id()),suf(d+1,id());
        for (int i=0;i<d;++i) {
            int u=g[v][i];
            side[i]=(u==p?fromParent:addEdge(down[u],u,v));
            pre[i+1]=merge(pre[i],side[i]);
        }
        for (int i=d-1;i>=0;--i) suf[i]=merge(side[i],suf[i+1]);
        answer[v]=addRoot(pre[d],v);
        for (int i=0;i<d;++i) if (g[v][i]!=p) {
            DP without=addRoot(merge(pre[i],suf[i+1]),v);
            dfsAll(g[v][i],v,addEdge(without,v,g[v][i]));
        }
    }
    vector<DP> solve(int root=0) {
        if (!n) return {};
        dfsDown(root,-1); dfsAll(root,-1,id());
        return answer;
    }
};
```

4.24 Centroid Decomposition

Use: Build centroid-parent hierarchy for distance/path query structures.
Complexity: $O(n \log n)$.
Pitfalls: Recursive DFS can overflow; `build()` resets decomposition state but keeps edges.
Assumptions/deps: Reusable tree/forest object with `addEdge/build/parent`.

```
// Centroid decomposition of a tree/forest. Parent[] is the centroid tree.
// Complexity: O(n log n) time, O(n) memory.
// Pitfalls: recursive DFS can overflow on a long chain; build() resets state,
// but keeps edges. The input must be a forest.
// Usage: CentroidDecomposition cd(n); cd.addEdge(u,v); cd.build();
struct CentroidDecomposition {
    int n;
    vector<vector<int>> g;
    vector<int> sub, parent;
    vector<char> dead;

    CentroidDecomposition(int n = 0) { init(n); }
    void init(int n_) {
        n = n_;
        g.assign(n, {});
        sub.resize(n);
        parent.assign(n, -1);
        dead.assign(n, false);
    }
    void addEdge(int u, int v) { g[u].push_back(v); g[v].push_back(u); }

    int sizes(int v, int p) {
        sub[v] = 1;
        for (int u : g[v]) if (u != p && !dead[u]) sub[v] += sizes(u, v);
        return sub[v];
    }
    int centroid(int v, int p, int total) {
        for (int u : g[v])
            if (u != p && !dead[u] && sub[u] > total / 2)
                return centroid(u, v, total);
        return v;
    }
    int decompose(int entry, int p) {
        int c = centroid(entry, -1, sizes(entry, -1));
        parent[c] = p;
        dead[c] = true;
        for (int u : g[c]) if (!dead[u]) decompose(u, c);
        return c;
    }
    void build() {
        fill(dead.begin(), dead.end(), false);
        fill(parent.begin(), parent.end(), -1);
        for (int v = 0; v < n; ++v) if (!dead[v]) decompose(v, -1);
    }
};
```

4.25 Offline Dynamic Connectivity

Use: Answer connectivity amid edge additions/removals using rollback DSU.
Complexity: $O(q \log q \log n)$.
Pitfalls: Removals must match active additions; identical edge cannot be active twice.
Assumptions/deps: Each API call advances time; query ids map into `solve()` result.

```
// Offline add/remove-edge connectivity using a rollback DSU.
// Complexity: O(q log q log n) time, O(q log q + n) memory.
// Pitfalls: every '-' must match one active '+'; the same edge cannot be
// active twice. Each add/remove/query advances time by one.
// Usage: dc.addEdge(u,v); int id=dc.connected(u,v); ...; ans=dc.solve();
struct RollbackDSU {
    vector<int> p, sz;
    vector<pair<int, int>> history; // attached root, old size of its parent

    RollbackDSU(int n = 0) { init(n); }
    void init(int n) {
        p.resize(n); iota(p.begin(), p.end(), 0);
```

```
        sz.assign(n, 1); history.clear();
    }
    int find(int v) const { while (v != p[v]) v = p[v]; return v; }
    bool same(int a, int b) const { return find(a) == find(b); }
    int snapshot() const { return history.size(); }
    bool unite(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) { history.push_back({-1, -1}); return false; }
        if (sz[a] < sz[b]) swap(a, b);
        history.push_back({b, sz[a]});
        p[b] = a; sz[a] += sz[b];
        return true;
    }
    void rollback(int snap) {
        while ((int)history.size() > snap) {
            auto [b, old] = history.back(); history.pop_back();
            if (b == -1) continue;
            int a = p[b];
            sz[a] = old; p[b] = b;
        }
    }
};

struct OfflineDynamicConnectivity {
    struct Op { char type; int u, v, id; };
    using Edge = pair<int, int>;
    int n, queries = 0;
    vector<Op> ops;
    vector<vector<Edge>> seg;

    OfflineDynamicConnectivity(int n = 0) : n(n) {}
    static Edge edge(int u, int v) { if (u > v) swap(u, v); return {u, v}; }
    void addEdge(int u, int v) { ops.push_back({'+', u, v, -1}); }
    void removeEdge(int u, int v) { ops.push_back({'-', u, v, -1}); }
    int connected(int u, int v) {
        ops.push_back({'?', u, v, queries});
        return queries++;
    }
    void addInterval(int x, int l, int r, int ql, int qr, Edge e) {
        if (qr <= l || r <= ql) return;
        if (ql <= l && r <= qr) { seg[x].push_back(e); return; }
        int m = (l + r) / 2;
        addInterval(2*x, l, m, ql, qr, e);
        addInterval(2*x+1, m, r, ql, qr, e);
    }
    void dfs(int x, int l, int r, RollbackDSU& dsu, vector<char>& ans) {
        int snap = dsu.snapshot();
        for (auto [u, v] : seg[x]) dsu.unite(u, v);
        if (r - l == 1) {
            if (ops[l].type == '?') ans[ops[l].id] = dsu.same(ops[l].u, ops[l].v);
        } else {
            int m = (l + r) / 2;
            dfs(2*x, l, m, dsu, ans);
            dfs(2*x+1, m, r, dsu, ans);
        }
        dsu.rollback(snap);
    }
    vector<char> solve() {
        int q = ops.size();
        vector<char> ans(queries);
        if (!q) return ans;
        seg.assign(4*q, {});
        map<Edge, int> active;
        for (int t = 0; t < q; ++t) {
            Edge e = edge(ops[t].u, ops[t].v);
            if (ops[t].type == '+') {
                assert(!active.count(e));
                active[e] = t;
            }
            if (ops[t].type == '-') {
                assert(active.count(e));
                addInterval(1, 0, q, active[e], t, e);
                active.erase(e);
            }
        }
        for (auto [e, start] : active) addInterval(1, 0, q, start, q, e);
        RollbackDSU dsu(n);
        dfs(1, 0, q, dsu, ans);
        return ans;
    }
};
```

4.26 CDQ 3D Dominance

Use: Count points dominated coordinate-wise by each point.
Complexity: $O(n \log^2 n)$.
Pitfalls: Exact duplicates are grouped; result excludes the point itself.
Assumptions/deps: Coordinates may be negative; includes an internal Fenwick tree.

```
// 3D dominance: for each point, count OTHER points with x<=X,y<=Y,z<=Z.
// Complexity: O(n log^2 n) time, O(n) memory (CDQ + Fenwick).
// Pitfalls: exact duplicate points are grouped; coordinates may be negative.
// Usage: vector<long long> ans = dominance_count(points);
struct Point3 { long long x, y, z; };

vector<long long> dominance_count(const vector<Point3>& input) {
    struct Node {
        long long x, y, z, weight, ans;
        int zi;
```

```
        vector<int> ids;
    };
    int n = input.size();
    vector<int> order(n); iota(order.begin(), order.end(), 0);
    sort(order.begin(), order.end(), [&](int a, int b) {
        return tie(input[a].x, input[a].y, input[a].z) <
               tie(input[b].x, input[b].y, input[b].z);
    });
    vector<Node> a;
    for (int id : order) {
        auto p = input[id];
        if (a.empty() || tie(a.back().x, a.back().y, a.back().z) != tie(p.x, p.y, p.z
            ))
            a.push_back({p.x, p.y, p.z, 0, 0, 0, {}});
        ++a.back().weight;
        a.back().ids.push_back(id);
    }
    vector<long long> zs;
    for (auto& p : a) zs.push_back(p.z);
    sort(zs.begin(), zs.end()); zs.erase(unique(zs.begin(), zs.end()), zs.end());
    for (auto& p : a) {
        p.zi = lower_bound(zs.begin(), zs.end(), p.z) - zs.begin() + 1;
        p.ans = p.weight - 1;
    }
    vector<long long> bit(zs.size() + 1);
    auto add = [&](int i, long long v) { for (; i < (int)bit.size(); i += i&-i) bit[i
        ] += v; };
    auto sum = [&](int i) { long long s=0; for (; i; i-=i&-i) s+=bit[i]; return s; };
    vector<Node> tmp(a.size());
    function<void(int,int)> cdq = [&](int l, int r) {
        if (r-l <= 1) return;
        int m=(l+r)/2; cdq(l,m); cdq(m,r);
        int i=l;
        for (int j=m; j<r; ++j) {
            while (i<m && a[i].y<=a[j].y) add(a[i].zi,a[i].weight), ++i;
            a[j].ans += sum(a[j].zi);
        }
        while (i>l) --i, add(a[i].zi,-a[i].weight);
        merge(a.begin()+l,a.begin()+m,a.begin()+m,a.begin()+r,tmp.begin()+l,
              [](const Node& p,const Node& q){ return p.y<q.y; });
        copy(tmp.begin()+l,tmp.begin()+r,a.begin()+l);
    };
    cdq(0, a.size());
    vector<long long> ans(n);
    for (auto& p : a) for (int id : p.ids) ans[id]=p.ans;
    return ans;
}
```

4.27 DSU on Tree Sack

Use: Answer all subtree queries offline.
Complexity: $O(n \log n)$ total.
Pitfalls: Reset globals per test; recursion depth on chains.
Assumptions/deps: EDIT ONLY the marked block (`add_one`, `answer`); when `answer(v)` runs, exactly subtree(v) has been added.

```
// DSU on tree / sack: answer all "subtree of v" queries offline, O(n log n).
// Fill g[0..n) and the marked block, then: dfs_sz(root,-1); dfs(root,-1,true);
// Guarantee: when answer(v) runs, add_one(u,+1) has been applied exactly once
// for every u in subtree(v) and for nothing outside it.
const int MAXN = 200000 + 5;
vector<int> g[MAXN];
int sz[MAXN], st[MAXN], ft[MAXN], ver[MAXN], timer_sack;

// ---- customize: counted state, one vertex in/out, answer hook -------------
int col[MAXN], cnt[MAXN]; // example state: color frequencies
void add_one(int v, int d) { cnt[col[v]] += d; }
void answer(int v) { (void)v; /* record answer for v from current state */ }
// ---- machinery below: do not edit -----------------------------------------

void dfs_sz(int v, int p) {
    st[v] = timer_sack;
    ver[timer_sack++] = v;
    sz[v] = 1;
    for (int u : g[v]) if (u != p) {
        dfs_sz(u, v);
        sz[v] += sz[u];
    }
    ft[v] = timer_sack;
}

void add_subtree(int v, int delta) {
    for (int i = st[v]; i < ft[v]; i++)
        add_one(ver[i], delta);
}

void dfs(int v, int p, bool keep) {
    int bigChild = -1;
    for (int u : g[v])
        if (u != p && (bigChild == -1 || sz[u] > sz[bigChild]))
            bigChild = u;

    for (int u : g[v])
        if (u != p && u != bigChild)
            dfs(u, v, false);

    if (bigChild != -1) dfs(bigChild, v, true);

    for (int u : g[v])
```

```
        if (u != p && u != bigChild)
            add_subtree(u, +1);
    add_one(v, +1);

    answer(v);

    if (!keep) add_subtree(v, -1);
}
```

4.28 Virtual Tree

Use: Build compressed tree from key vertices and LCAs.
Complexity: $O(k \log k)$ per call after LCA build.
Pitfalls: Process returned nodes in reverse for child-before-parent DP.
Assumptions/deps: Requires LCA (above) built on the tree. Returns nodes by tin (front = root); fills (parent,child) edges.

```
// Virtual (auxiliary) tree over key vertices.
// Requires LCA from lca_binary_lifting.cpp, built on the tree.
// Returns key vertices plus needed LCAs sorted by tin (front = root of the
// virtual tree) and fills edges as (parent, child) pairs. O(k log k) per call.
// Typical DP: process returned nodes in reverse (children before parents).
vector<int> vtree(vector<int> v, LCA& T, vector<pair<int, int>>& edges) {
    edges.clear();
    if (v.empty()) return {};
    auto byTin = [&](int a, int b) { return T.tin[a] < T.tin[b]; };
    sort(v.begin(), v.end(), byTin);
    int k = v.size();
    for (int i = 1; i < k; i++) v.push_back(T.lca(v[i - 1], v[i]));
    sort(v.begin(), v.end(), byTin);
    v.erase(unique(v.begin(), v.end()), v.end());
    for (int i = 1; i < (int)v.size(); i++)
        edges.push_back({T.lca(v[i - 1], v[i]), v[i]});
    return v;
}
```

4.29 Kruskal Reconstruction Tree

Use: Merge tree: max edge on path = LCA weight; threshold reachability = subtree.
Complexity: $O(m \log m \, \alpha(n))$.
Pitfalls: For disconnected graph, returned root is only last component. `MAXK` must be large enough.
Assumptions/deps: Uses global `edges`; clear between tests. Query recipes in header.

```
// Kruskal reconstruction tree for an undirected weighted graph.
// Original vertices are [0, n0); each MST-merge adds an internal node whose
// node_weight is the edge weight. Connected graph: root = total_nodes - 1.
// Time: O(m log m alpha(n)).
// Queries (weights increase toward the root):
// - max edge on u..v MST path = node_weight[LCA(u,v) in ktree].
// - "vertices reachable from u via edges <= w" = leaves under the highest
// ancestor of u with node_weight <= w (find it by binary lifting).
// For min-edge/max-bottleneck versions sort edges descending instead.
const int MAXK = 400000 + 5; // at least 2 * original_n + m_components
struct KEdge { int w, u, v; };
vector<KEdge> edges;
vector<int> ktree[MAXK];
int dsu[MAXK], node_weight[MAXK], total_nodes;

int getRoot(int u) {
    if (u == dsu[u]) return u;
    return dsu[u] = getRoot(dsu[u]);
}

int build_kruskal_tree(int n0) {
    total_nodes = n0;
    for (int i = 0; i < MAXK; i++) {
        dsu[i] = i;
        ktree[i].clear();
    }
    sort(edges.begin(), edges.end(), [](const KEdge& a, const KEdge& b) {
        return a.w < b.w;
    });
    for (auto e : edges) {
        int u = getRoot(e.u), v = getRoot(e.v);
        if (u == v) continue;
        int x = total_nodes++;
        node_weight[x] = e.w;
        ktree[x].push_back(u);
        ktree[x].push_back(v);
        dsu[u] = dsu[v] = dsu[x] = x;
    }
    return total_nodes - 1;
}
```

4.30 Global Min Cut

Use: Stoer-Wagner undirected global min cut.
Complexity: $O(V^3)$ with matrix version.

Pitfalls: Undirected symmetric matrix only; weights nonnegative; integer sums may overflow.
Assumptions/deps: Requires `vi`, `sz`, `rep`, `all`.

```
pair<int, vi> globalMinCut(vector<vi> mat) {
  pair<int, vi> best = {INT_MAX, {}};
  int n = sz(mat);
  vector<vi> co(n);
  rep(i,0,n) co[i] = {i};
  rep(ph,1,n) {
    vi w = mat[0];
    size_t s = 0, t = 0;
    rep(it,0,n-ph) { // O(V^2) -> O(E log V) with prio. queue
      w[t] = INT_MIN;
      s = t, t = max_element(all(w)) - w.begin();
      rep(i,0,n) w[i] += mat[t][i];
    }
    best = min(best, {w[t] - mat[t][t], co[t]});
    co[s].insert(co[s].end(), all(co[t]));
    rep(i,0,n) mat[s][i] += mat[t][i];
    rep(i,0,n) mat[i][s] = mat[s][i];
    mat[0][t] = INT_MIN;
  }
  return best;
}
```

4.31 Maximum Clique

Use: Branch-and-bound maximum clique / complement for independent set.
Complexity: Exponential; fast for about $n \leq 155$ in tested cases.
Pitfalls: Input must be symmetric bitset matrix, no self-edges. `bitset<200>` limits n.
Assumptions/deps: Requires `vi`, `sz`, `all`, `rep`.

```
/**
 * Author: chilli, SJTU, Janez Konc
 * Date: 2019-05-10
 * License: GPL3+
 * Source: https://en.wikipedia.org/wiki/MaxCliqueDyn_maximum_clique_algorithm, https
     ://gitlab.com/janezkonc/mcqd/blob/master/mcqd.h
 * Description: Quickly finds a maximum clique of a graph (given as symmetric bitset
 * matrix; self-edges not allowed). Can be used to find a maximum independent
 * set by finding a clique of the complement graph.
 * Time: Runs in about 1s for n=155 and worst case random graphs (p=.90). Runs
 * faster for sparse graphs.
 * Status: stress-tested
 */
typedef vector<bitset<200>> vb;
struct Maxclique {
  double limit=0.025, pk=0;
  struct Vertex { int i, d=0; };
  typedef vector<Vertex> vv;
  vb e;
  vv V;
  vector<vi> C;
  vi qmax, q, S, old;
  void init(vv& r) {
    for (auto& v : r) v.d = 0;
    for (auto& v : r) for (auto j : r) v.d += e[v.i][j.i];
    sort(all(r), [](auto a, auto b) { return a.d > b.d; });
    int mxD = r[0].d;
    rep(i,0,sz(r)) r[i].d = min(i, mxD) + 1;
  }
  void expand(vv& R, int lev = 1) {
    S[lev] += S[lev - 1] - old[lev];
    old[lev] = S[lev - 1];
    while (sz(R)) {
      if (sz(q) + R.back().d <= sz(qmax)) return;
      q.push_back(R.back().i);
      vv T;
      for(auto v:R) if (e[R.back().i][v.i]) T.push_back({v.i});
      if (sz(T)) {
        if (S[lev]++ / ++pk < limit) init(T);
        int j = 0, mxk = 1, mnk = max(sz(qmax) - sz(q) + 1, 1);
        C[1].clear(), C[2].clear();
        for (auto v : T) {
          int k = 1;
          auto f = [&](int i) { return e[v.i][i]; };
          while (any_of(all(C[k]), f)) k++;
          if (k > mxk) mxk = k, C[mxk + 1].clear();
          if (k < mnk) T[j++].i = v.i;
          C[k].push_back(v.i);
        }
        if (j > 0) T[j - 1].d = 0;
        rep(k,mnk,mxk + 1) for (int i : C[k])
          T[j].i = i, T[j++].d = k;
        expand(T, lev + 1);
      } else if (sz(q) > sz(qmax)) qmax = q;
      q.pop_back(), R.pop_back();
    }
  }
  vi maxClique() { init(V), expand(V); return qmax; }
  Maxclique(vb conn) : e(conn), C(sz(e)+1), S(sz(C)), old(S) {
    rep(i,0,sz(e)) V.push_back({i});
  }
};
```

4.32 Critical Cities Dominator

Use: Dominator-style critical vertices from source to sink in directed graph.
Complexity: About $O((n+m)\log n)$.
Pitfalls: Standalone problem solution, not a reusable library. Assumes sink reachable from source.
Assumptions/deps: Includes its own `main`; 1-indexed input converted to 0-index.

```cpp
#include <algorithm>
#include <cassert>
#include <cmath>
#include <cstdint>
#include <functional>
#include <iostream>
#include <vector>

using std::cout;
using std::endl;
using std::vector;

// BeginCodeSnip{Tree Class for rdom}
template <typename T> class Tree {
 private:
 const int root = 0;
 const vector<int> &parents;
 const vector<T> &vals;
 const int log2dist;
 vector<vector<int>> pow2ends;
 vector<vector<T>> pow2mins;
 vector<int> depth;

 public:
 Tree(const vector<int> &parents, const vector<T> &vals)
     : parents(parents), vals(vals),
       log2dist(std::ceil(std::log2(parents.size() + 1))),
       pow2ends(parents.size(), vector<int>(log2dist + 1)),
       pow2mins(parents.size(), vector<T>(log2dist + 1)) {
  assert(parents[root] == -1);

  vector<vector<int>> children(parents.size());
  for (int n = 0; n < parents.size(); n++) {
    if (parents[n] != -1) { children[parents[n]].push_back(n); }
  }
  depth = vector<int>(parents.size());
  vector<int> frontier{root};
  while (!frontier.empty()) {
    int curr = frontier.back();
    frontier.pop_back();
    for (int n : children[curr]) {
      depth[n] = depth[curr] + 1;
      frontier.push_back(n);
    }
  }

  for (int n = 0; n < parents.size(); n++) {
    pow2mins[n][0] = vals[n];
    pow2ends[n][0] = parents[n];
  }
  for (int p = 1; p <= log2dist; p++) {
    for (int n = 0; n < parents.size(); n++) {
      int halfway = pow2ends[n][p - 1];
      if (halfway == -1) {
        pow2ends[n][p] = -1;
        pow2mins[n][p] = pow2mins[n][p - 1];
      } else {
        pow2ends[n][p] = pow2ends[halfway][p - 1];
        pow2mins[n][p] =
            std::min(pow2mins[n][p - 1], pow2mins[halfway][p - 1]);
      }
    }
  }

 /**
  * @return the min value on the path from the ancestor to its descendant,
  * not including the ancestor.
  */
 T min_val(int ancestor, int desc) {
  int dist = depth[desc] - depth[ancestor];
  T ret = vals[desc];
  int at = desc;
  for (int pow = 0; pow <= log2dist; pow++) {
    if ((dist & (1 << pow)) != 0) {
      ret = std::min(ret, pow2mins[at][pow]);
      at = pow2ends[at][pow];
    }
  }
  assert(at == ancestor);
  return ret;
 }
};
// EndCodeSnip

int main() {
 int city_num;
 int flight_num;
 std::cin >> city_num >> flight_num;
 vector<vector<int>> adj(city_num);
 vector<vector<int>> rev_adj(city_num);
 for (int f = 0; f < flight_num; f++) {
  int a, b;
  std::cin >> a >> b;
  adj[--a].push_back(--b);
```

```
  rev_adj[b].push_back(a);
}

// Variables used for the Euler Tour
vector<int> visit_order;
// visit_time is initialized with INF so nodes that aren't reachable
// during the DFS won't mess up our sdom calculation
vector<int> visit_time(city_num, INT32_MAX);
vector<int> visit_parent(city_num, -1);
vector<bool> visited(city_num, false);
std::function<void(int, int)> dfs;
dfs = [&](int at, int prev) {
  if (visited[at]) { return; }
  visited[at] = true;

  visit_time[at] = visit_order.size();
  visit_order.push_back(at);
  visit_parent[at] = prev;
  for (int next : adj[at]) { dfs(next, at); }
};
dfs(0, -1);

// We use a function-based interface instead of a class-based one due to
// heavy reliance on previously calculated values
vector<int> dsu_parent(city_num, -1);
vector<int> dsu_min(city_num, INT32_MAX);
std::function<int(int)> find;
find = [&](int x) {
  if (dsu_parent[x] == -1) { return x; }
  if (dsu_parent[dsu_parent[x]] != -1) {
    int parent_res = find(dsu_parent[x]);
    if (visit_time[parent_res] < visit_time[dsu_min[x]]) {
      dsu_min[x] = parent_res;
    }
    dsu_parent[x] = dsu_parent[dsu_parent[x]];
  }
  assert(dsu_min[x] != INT32_MAX);
  return dsu_min[x];
};

vector<int> sdom(city_num, -1);
for (int i = visit_order.size() - 1; i > 0; i--) {
  int c = visit_order[i];
  // Iterate over all nodes with a directed edge to c
  for (int from : rev_adj[c]) {
    int find_res = find(from);
    // Take the node with the earliest visit time in the traversal
    if (sdom[c] == -1 || visit_time[find_res] < visit_time[sdom[c]]) {
      sdom[c] = find_res;
    }
  }

  // Link c to its parent in the DFS tree
  dsu_parent[c] = visit_parent[c];
  dsu_min[c] = sdom[c];
}

// Initialize the values in the tree
vector<std::pair<int, int>> sdom_times(city_num, {INT32_MAX, -1});
for (int c : visit_order) {
  if (c != 0) {
    // The tree takes the minimum of these pairs, but what we
    // really want is the node itself- therefore, we store both
    sdom_times[c] = {visit_time[sdom[c]], c};
  }
}
Tree tree(visit_parent, sdom_times);

vector<int> rdom(city_num, -1);
for (int c : visit_order) {
  if (c != 0) {
    // Use the definition of the rdom
    rdom[c] = tree.min_val(sdom[c], c).second;
  }
}

// Use the properties previously given to compute the idom
vector<int> idom(city_num, -1);
for (int c : visit_order) {
  if (c != 0) { idom[c] = rdom[c] == c ? sdom[c] : idom[rdom[c]]; }
}

// Trace the idom tree from the finish node to the start node,
// finding all dominators along the way
// NOTE: assumes city_num-1 is reachable from 0; otherwise idom[-1] UB.
vector<int> critical{0};
int at = city_num - 1;
while (at != 0) {
  critical.push_back(at);
  at = idom[at];
}

std::sort(critical.begin(), critical.end());

cout << critical.size() << '\n';
for (int i = 0; i < critical.size() - 1; i++) { cout << critical[i] + 1 << ' '; }
cout << critical.back() + 1 << endl;
}
```

5 Math

Checklist: prime modulus? overflow in multiplication/lcm? zero inverse? random/probabilistic? clear bases/sieves? CRT residues compatible?

5.1 Number Theory Essentials

Use: Extended gcd, modular inverse, fast power.
Complexity: $O(\log n)$.
Pitfalls: `inv` uses Fermat: only prime modulus. Use `inv_any` for coprime composite mod.
Assumptions/deps: `modpow` uses `__int128`.

```cpp
// egcd: g = gcd(a,b) and x,y with a*x + b*y = g. Needed by CRT.cpp.
// inv_any: inverse mod any modulus (or -1); inv: prime mod only (Fermat).
long long egcd(long long a, long long b, long long& x, long long& y) {
    if (!b) {
        x = 1; y = 0;
        return a;
    }
    long long x1, y1;
    long long g = egcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - y1 * (a / b);
    return g;
}

long long inv_any(long long a, long long mod) {
    long long x, y;
    long long g = egcd(a, mod, x, y);
    if (g != 1) return -1; // no inverse
    x %= mod;
    if (x < 0) x += mod;
    return x;
}

long long modpow(long long a, long long e, long long mod) {
    long long r = 1;
    while (e) {
        if (e & 1) r = (__int128)r * a % mod;
        a = (__int128)a * a % mod;
        e >>= 1;
    }
    return r;
}

long long inv(long long a, long long mod) {
    return modpow(a, mod - 2, mod);
}
```

5.2 Sieve

Use: Enumerate primes below limit.
Complexity: $O(n \log \log n)$.
Pitfalls: Returns primes `< lim`, not `<= lim`. `MAX_PR` fixed.
Assumptions/deps: Self-contained.

```cpp
// Sieve of Eratosthenes; returns primes < lim. Requires lim <= MAX_PR.
// Time: lim=1e9 ~ 1.5s if MAX_PR is raised; 5e6 is instant.
const int MAX_PR = 5'000'000;
bitset<MAX_PR> isprime;
vector<int> eratosthenesSieve(int lim) {
  isprime.set(); isprime[0] = isprime[1] = 0;
  for (int i = 4; i < lim; i += 2) isprime[i] = 0;
  for (int i = 3; i*i < lim; i += 2) if (isprime[i])
    for (int j = i*i; j < lim; j += i*2) isprime[j] = 0;
  vector<int> pr;
  for (int i = 2; i < lim; i++) if (isprime[i]) pr.push_back(i);
  return pr;
}
```

5.3 CRT

Use: Merge two congruences, non-coprime allowed.
Complexity: $O(\log \min(m_1, m_2))$.
Pitfalls: Works only if residues compatible. LCM can overflow `long long`.
Assumptions/deps: Depends on `egcd` from number theory essentials.

```cpp
// Solve x = a1 (mod m1), x = a2 (mod m2); returns {x, lcm(m1,m2)} with
// 0 <= x < lcm, or {-1,-1} if impossible. Chain calls for >2 congruences.
// Requires egcd from NT_essentials.cpp. NOTE: lcm itself must fit long long.
pair<long long,long long> crt(long long a1, long long m1,
                              long long a2, long long m2) {
    long long x, y;
    long long g = egcd(m1, m2, x, y);

    if ((a2 - a1) % g != 0) return {-1, -1};

    long long lcm = m1 / g * m2;
    __int128 t = (__int128)(a2 - a1) / g * x;
```

```
    t %= m2 / g;

    long long ans = (a1 + (__int128)m1 * t) % lcm;
    if (ans < 0) ans += lcm;

    return {ans, lcm};
}
```

5.4 Mod Inverse Table

Use: Inverses, factorials, inverse factorials modulo prime.
Complexity: $O(n)$.
Pitfalls: `MOD` must be prime and `n<MOD`. `ifact` code is cumulative inverse factorial.
Assumptions/deps: Change `MOD` before use.

```
// Precompute modular inverses and factorial inverses modulo prime MOD.
// Complexity: O(n). Use when 0 <= n < MOD.
// Pitfalls: invv[i] uses invv[MOD % i], not invv[i-1]; MOD must be prime.
// Usage: build_inv(1000000); then invv[i], fact[i], ifact[i], C(n,k).
const long long MOD = 1000000007;
vector<long long> invv, fact, ifact;

void build_inv(int n) {
    invv.assign(n + 1, 0);
    fact.assign(n + 1, 1);
    ifact.assign(n + 1, 1);
    if (n >= 1) invv[1] = 1;
    for (int i = 2; i <= n; i++)
        invv[i] = MOD - (MOD / i) * invv[MOD % i] % MOD;
    for (int i = 1; i <= n; i++) {
        fact[i] = fact[i - 1] * i % MOD;
        ifact[i] = ifact[i - 1] * invv[i] % MOD; // this one uses previous value
    }
}

long long C(int n, int k) {
    if (k < 0 || k > n) return 0;
    return fact[n] * ifact[k] % MOD * ifact[n - k] % MOD;
}
```

5.5 Binomial (auto-extend)

Use: $C(n, k)$ with lazily grown factorial tables.
Complexity: Amortized $O(1)$ per query after growth.
Pitfalls: `mod` must be prime and `n < mod`.
Assumptions/deps: Alternative to fixed-size table above; pick one.

```
// Factorials/inverses mod prime, auto-extending. C(n,k) is O(1) after extend.
using ll = long long;
const ll mod = 1e9 + 7;
struct Binom {
    vector<ll> fact{1}, inv{1}, invfact{1};

    void extend(ll n) {
        ll cur = fact.size() - 1;
        if (cur >= n) return;

        fact.resize(n + 1);
        inv.resize(n + 1);
        invfact.resize(n + 1);

        for (ll i = cur + 1; i <= n; i++) {
            fact[i] = fact[i - 1] * i % mod;

            if (i == 1) inv[i] = 1;
            else inv[i] = mod - mod / i * inv[mod % i] % mod;

            invfact[i] = invfact[i - 1] * inv[i] % mod;
        }
    }

    ll C(ll n, ll k) {
        if (k < 0 || k > n) return 0;
        extend(n);
        return fact[n] * invfact[k] % mod * invfact[n - k] % mod;
    }
} binom;
```

5.6 Lucas Theorem

Use: $C(n, k)$ mod p for large n and small prime p.
Complexity: Build $O(p)$; query $O(\log_p n)$.
Pitfalls: Prime p must be small enough for factorial tables.
Assumptions/deps: Digit products in base p; see Kummer note below.

```
// Lucas theorem: C(n,k) modulo a small prime p, even when n>=p.
// Complexity: build O(p), query O(log_p n); memory O(p).
// Pitfalls: p must be prime and small enough to allocate; n,k nonnegative.
// Usage: Lucas lucas(p); long long value=lucas.C(n,k);
struct Lucas {
    long long p;
    vector<long long>fact,ifact;
```

```
    long long power(long long a,long long e) const {
        long long r=1;
        for(;e;e>>=1,a=a*a%p) if(e&1) r=r*a%p;
        return r;
    }
    Lucas(int prime):p(prime),fact(prime),ifact(prime) {
        fact[0]=1;for(int i=1;i<p;++i)fact[i]=fact[i-1]*i%p;
        ifact[p-1]=power(fact[p-1],p-2);
        for(int i=p-1;i;--i)ifact[i-1]=ifact[i]*i%p;
    }
    long long small(int n,int k) const {
        if(k<0||k>n)return 0;
        return fact[n]*ifact[k]%p*ifact[n-k]%p;
    }
    long long C(long long n,long long k) const {
        if(k<0||k>n)return 0;
        long long ans=1;
        while(n||k) {
            int ni=n%p,ki=k%p;
            if(ki>ni)return 0;
            ans=ans*small(ni,ki)%p;n/=p;k/=p;
        }
        return ans;
    }
};
```

5.7 Lucas and Kummer

Use: Binomial digit formula and p-adic carry interpretation.
Complexity: Reference formulas.
Pitfalls: Lucas code is impractical when p is too large to precompute.
Assumptions/deps: Pairs with the implementation above.

Lucas

For prime p, write n and k in base p with digits n_i, k_i:

$$\binom{n}{k} \equiv \prod_i \binom{n_i}{k_i} \pmod{p}.$$

If any $k_i > n_i$, the result is zero.

Kummer

The exponent of prime p in $\binom{n}{k}$ equals the number of carries when adding k and $n-k$ in base p. Use this for divisibility or p-adic valuation.

The code precomputes through $p-1$, so it needs $O(p)$ memory. For large p and few queries, compute each digit-level binomial multiplicatively.

5.8 Matrix Power

Use: Linear recurrences, path counting, DP transitions.
Complexity: `mul` $O(n^3)$; `mpow` $O(n^3 \log e)$.
Pitfalls: Change `MMOD` as needed; matrices must be square for `mpow`.
Assumptions/deps: Fibonacci example in header.

```
// Matrix multiply/power mod. mul O(n^3), mpow O(n^3 log e).
// Usage (Fibonacci): mpow({{1,1},{1,0}}, n)[0][1] == F(n).
using ll = long long;
const ll MMOD = 1e9 + 7;
using Mat = vector<vector<ll>>;

Mat mul(const Mat& A, const Mat& B) {
    int n = A.size(), m = B[0].size(), K = B.size();
    Mat C(n, vector<ll>(m, 0));
    for (int i = 0; i < n; i++)
        for (int k = 0; k < K; k++) if (A[i][k])
            for (int j = 0; j < m; j++)
                C[i][j] = (C[i][j] + A[i][k] * B[k][j]) % MMOD;
    return C;
}

Mat mpow(Mat A, ll e) { // A square, e >= 0
    int n = A.size();
    Mat R(n, vector<ll>(n, 0));
    for (int i = 0; i < n; i++) R[i][i] = 1;
    for (; e; e >>= 1, A = mul(A, A))
        if (e & 1) R = mul(R, A);
    return R;
}
```

5.9 Primitive Root

Use: Find generator modulo prime.
Complexity: $O(\sqrt{p} \log p)$ roughly.
Pitfalls: Prime modulus only (factors `p-1`). 998244353 has root 3.
Assumptions/deps: Requires `modpow` from NT Essentials. Used by kth roots below.

```
// Smallest primitive root modulo PRIME p (factors phi = p-1).
// Requires modpow from NT_essentials.cpp. O(sqrt p + ans * log^2 p).
// For NTT: root of mod 998244353 is 3. Used by discrete_mod.cpp.
long long generator(long long p) {
    if (p == 2) return 1;
```

```
    vector<long long> fact;
    long long phi = p - 1, n = phi;
    for (long long i = 2; i * i <= n; i++)
        if (n % i == 0) {
            fact.push_back(i);
            while (n % i == 0) n /= i;
        }
    if (n > 1) fact.push_back(n);
    for (long long res = 2; res <= p; res++) {
        bool ok = true;
        for (long long f : fact)
            ok &= modpow(res, phi / f, p) != 1;
        if (ok) return res;
    }
    return -1;
}
```

5.10 Miller Rabin and Pollard Rho

Use: 64-bit primality and factorization.
Complexity: Primality $O(\log n)$; factorization expected about $O(n^{1/4})$.
Pitfalls: Probabilistic factor search but deterministic bases for 64-bit primality. Factors unsorted.
Assumptions/deps: Uses unsigned 64-bit and `__uint128_t`.

```
// Deterministic Miller-Rabin and Pollard Rho factorization for unsigned 64-bit.
// Source: KACTL MillerRabin/Factor, using __int128 mod multiplication.
// Complexity: primality O(7 log n); factorization expected about O(n^(1/4)).
// Pitfalls: call factor(n) only for n >= 1; returns unsorted prime factors; factor(1) is empty.
// Usage: isPrime(n); auto f = factor(n); sort(f.begin(), f.end());
using ull = unsigned long long;
using u128 = __uint128_t;

ull mod_mul(ull a, ull b, ull mod) {
    return (ull)((u128)a * b % mod);
}

ull mod_pow(ull a, ull e, ull mod) {
    ull r = 1;
    while (e) {
        if (e & 1) r = mod_mul(r, a, mod);
        a = mod_mul(a, a, mod);
        e >>= 1;
    }
    return r;
}

bool isPrime(ull n) {
    if (n < 2 || n % 6 % 4 != 1) return (n | 1) == 3;
    ull d = n - 1, s = 0;
    while ((d & 1) == 0) d >>= 1, s++;
    for (ull a : {2ULL, 325ULL, 9375ULL, 28178ULL, 450775ULL, 9780504ULL, 1795265022ULL}) {
        ull x = mod_pow(a % n, d, n), i = s;
        while (x != 1 && x != n - 1 && a % n && i--) x = mod_mul(x, x, n);
        if (x != n - 1 && i != s) return false;
    }
    return true;
}

ull pollard(ull n) {
    if (n % 2 == 0) return 2;
    static mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
    while (true) {
        ull c = uniform_int_distribution<ull>(1, n - 1)(rng);
        ull x = uniform_int_distribution<ull>(0, n - 1)(rng), y = x, d = 1;
        auto f = [&](ull v) { return (mod_mul(v, v, n) + c) % n; };
        while (d == 1) {
            x = f(x);
            y = f(f(y));
            ull diff = x > y ? x - y : y - x;
            d = gcd(diff, n);
        }
        if (d != n) return d;
    }
}

vector<ull> factor(ull n) {
    if (n == 1) return {};
    if (isPrime(n)) return {n};
    ull d = pollard(n);
    auto a = factor(d), b = factor(n / d);
    a.insert(a.end(), b.begin(), b.end());
    return a;
}
```

5.11 Gaussian Elimination mod p

Use: Solve $Ax = b$ over prime field; get one solution + solution count.
Complexity: $O(nm \min(n, m))$.
Pitfalls: Returns -1 if inconsistent, else #free variables (p^{free} solutions); free vars set to 0. Prime modulus only.
Assumptions/deps: Self-contained (includes `gpow`).

```
// Solve A x = b mod PRIME p. a = n rows of m+1 entries (last column is b).
```

```cpp
// Returns -1 if no solution, else the number of FREE variables (0 = unique).
// sol gets one valid solution (free variables set to 0). O(n m min(n,m)).
// Rank of A alone: pass b = 0 column; rank = m - free. Self-contained.
long long gpow(long long a, long long e, long long mod) {
    long long r = 1;
    for (a %= mod; e; e >>= 1, a = a * a % mod)
        if (e & 1) r = r * a % mod;
    return r;
}

int gauss(vector<vector<long long>> a, long long mod, vector<long long>& sol) {
    int n = a.size(), m = (int)a[0].size() - 1, row = 0;
    for (auto& r : a) for (auto& x : r) x = (x % mod + mod) % mod;
    vector<int> where(m, -1);
    for (int col = 0; col < m && row < n; col++) {
        int sel = row;
        while (sel < n && !a[sel][col]) sel++;
        if (sel == n) continue;
        swap(a[sel], a[row]);
        long long invp = gpow(a[row][col], mod - 2, mod);
        for (int j = col; j <= m; j++) a[row][j] = a[row][j] * invp % mod;
        for (int i = 0; i < n; i++) if (i != row && a[i][col]) {
            long long f = a[i][col];
            for (int j = col; j <= m; j++)
                a[i][j] = ((a[i][j] - f * a[row][j]) % mod + mod) % mod;
        }
        where[col] = row++;
    }
    for (int i = row; i < n; i++) // leftover rows are all-zero except maybe b
        if (a[i][m]) return -1;
    sol.assign(m, 0);
    for (int c = 0; c < m; c++)
        if (where[c] != -1) sol[c] = a[where[c]][m];
    return m - row; // row = rank
}
```

5.12 Floating Simplex

Use: Maximize $c^T x$ subject to $Ax \le b$, $x \ge 0$.
Complexity: $O(nm \cdot pivots)$; exponential worst case.
Pitfalls: Numerical stability is not guaranteed; distinguish infeasible and unbounded.
Assumptions/deps: Two-phase solver with explicit status; high-risk fallback only.

```cpp
// Two-phase simplex: maximize c^T x subject to A*x<=b and x>=0.
// Adapted to an explicit-result API from the Stanford/KACTL simplex (MIT).
// Complexity: O(nm * pivots), exponential worst case; memory O(nm).
// Pitfalls: floating numerical stability is NOT guaranteed; prefer x=0 feasible
// formulations. Status: -1 infeasible, 0 optimum, +1 unbounded.
// Usage: auto r=Simplex(A,b,c).solve(); r.status,r.value,r.x.
struct LPResult {int status;long double value;vector<long double>x;};
struct Simplex {
    using T=long double;
    static constexpr T EPS=1e-12L;
    int m,n;
    vector<int>N,B;
    vector<vector<T>>D;
    Simplex(const vector<vector<T>>&A,const vector<T>&b,const vector<T>&c):
        m(b.size()),n(c.size()),N(n+1),B(m),D(m+2,vector<T>(n+2)) {
        for(int i=0;i<m;++i)for(int j=0;j<n;++j)D[i][j]=A[i][j];
        for(int i=0;i<m;++i)B[i]=n+i,D[i][n]=-1,D[i][n+1]=b[i];
        for(int j=0;j<n;++j)N[j]=j,D[m][j]=-c[j];
        N[n]=-1;D[m+1][n]=1;
    }
    void pivot(int r,int s){
        T inv=1/D[r][s];
        for(int i=0;i<m+2;++i)if(i!=r&&fabsl(D[i][s])>EPS){T factor=D[i][s]*inv;for(
            int j=0;j<n+2;++j)D[i][j]-=D[r][j]*factor;D[i][s]=D[r][s]*factor;}
        for(int j=0;j<n+2;++j)if(j!=s)D[r][j]*=inv;
        for(int i=0;i<m+2;++i)if(i!=r)D[i][s]*=-inv;
        D[r][s]=inv;swap(B[r],N[s]);
    }
    bool phase(int which){
        int row=m+which-1;
        while(true){
            int s=-1;
            for(int j=0;j<=n;++j)if(N[j]!=-which&&(s==-1||make_pair(D[row][j],N[j])<
                make_pair(D[row][s],N[s])))s=j;
            if(s==-1||D[row][s]>=-EPS)return true;
            int r=-1;
            for(int i=0;i<m;++i)if(D[i][s]>EPS&&(r==-1||make_pair(D[i][n+1]/D[i][s],B[
                i])<make_pair(D[r][n+1]/D[r][s],B[r])))r=i;
            if(r==-1)return false;
            pivot(r,s);
        }
    }
    LPResult solve(){
        if(!n){for(int i=0;i<m;++i)if(D[i][n+1]<-EPS)return {-1,0,{}};return
            {0,0,{}};}
        if(m){int r=min_element(D.begin(),D.begin()+m,[&](auto&a,auto&b){return a[n
            +1]<b[n+1];})-D.begin();
            if(D[r][n+1]<-EPS){pivot(r,n);if(!phase(2)||D[m+1][n+1]<-EPS)return
                {-1,0,{}};for(int i=0;i<m;++i)if(B[i]==-1){int s=0;for(int j=1;j<=n
                ;++j)if(make_pair(D[i][j],N[j])<make_pair(D[i][s],N[s]))s=j;pivot(i,s
                );}}
        }
        bool bounded=phase(1);vector<T>x(n);for(int i=0;i<m;++i)if(B[i]<n)x[B[i]]=D[i
            ][n+1];
        return {bounded?0:1,bounded?D[m][n+1]:numeric_limits<T>::infinity(),x};
```

```
    }
};
```

5.13 Simplex Modeling

Use: Canonical-form conversions and numerical checklist.
Complexity: Reference notes.
Pitfalls: Prefer combinatorial formulations when available.
Assumptions/deps: Verify every returned constraint.

Canonical input

$$\max c^T x \quad \text{subject to } Ax \le b,\ x \ge 0.$$

Phase I adds an artificial variable to find a feasible basis; failure means infeasible. Phase II uses the real objective; an entering column with no leaving row means unbounded.

Modeling conversions

- Equality $a^T x = b$: add both $a^T x \le b$ and $-a^T x \le -b$.
- Free variable: $y = y^+ - y^-$ with $y^+, y^- \ge 0$.
- Lower bound $y \ge L$: substitute $y = L + z$, $z \ge 0$.
- Minimization: maximize the negated objective.

Floating simplex is risky: scale coefficients, avoid nearly dependent constraints, and verify every returned constraint. Prefer a combinatorial flow/matching formulation when one exists.

5.14 Catalan Numbers

Use: Quick formulas for Catalan counts.
Complexity: $O(1)$ formula after binomial precompute.
Pitfalls: Division requires modular inverse; modulus must allow it.
Assumptions/deps: Use factorial/binomial table separately.

Counts balanced-parenthesis strings, rooted ordered trees, and triangulations:

$$C_n = \frac{1}{n+1}\binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1}.$$

In modular arithmetic, the division needs an inverse of $n+1$. The difference form can be safer when that inverse does not exist.

5.15 Mobius Sieve

Use: Linear sieve for Mobius function.
Complexity: $O(n)$.
Pitfalls: `N` fixed; reset arrays before each build. `mu[1]=1`.
Assumptions/deps: Globals `mu,primes,is_comp`.

```
const int N = 1e6;
int mu[N + 1], primes[N + 1], pcnt;
bool is_comp[N + 1];

void mobius_sieve(int n) {
    pcnt = 0;
    for (int i = 0; i <= n; i++) {
        mu[i] = 0;
        is_comp[i] = false;
    }
    mu[1] = 1;
    for (int i = 2; i <= n; i++) {
        if (!is_comp[i]) {
            primes[pcnt++] = i;
            mu[i] = -1;
        }
        for (int j = 0; j < pcnt && i * primes[j] <= n; j++) {
            int x = i * primes[j];
            is_comp[x] = true;
            if (i % primes[j] == 0) {
                mu[x] = 0;
                break;
            } else {
                mu[x] = -mu[i];
            }
        }
    }
}
```

5.16 Mobius Inversion Notes

Use: Formula reminder for divisor-sum inversion.
Complexity: Reference notes.
Pitfalls: Apply only when the divisor relation really matches.
Assumptions/deps: Pair with the sieve below.

Möbius function

The Möbius function $\mu(n)$ is defined as

$$\mu(n) = \begin{cases} 1 & \text{if } n = 1, \\ (-1)^k & \text{if } n \text{ is a product of } k \text{ distinct primes}, \\ 0 & \text{if } n \text{ has a squared prime factor.} \end{cases}$$

Key identity:

$$\sum_{d|n} \mu(d) = \begin{cases} 1 & \text{if } n = 1, \\ 0 & \text{if } n > 1. \end{cases}$$

Inversion formula

If

$$F(n) = \sum_{d|n} f(d),$$

then

$$f(n) = \sum_{d|n} \mu(d)\, F\left(\frac{n}{d}\right).$$

Equivalently, in Dirichlet convolution form:

$$F = f * 1 \quad \Longleftrightarrow \quad f = F * \mu.$$

Common application

The number of ordered pairs $1 \le i, j \le n$ with $\gcd(i, j) = 1$ is

$$\sum_{d=1}^{n} \mu(d) \left\lfloor \frac{n}{d} \right\rfloor^2$$

Use Möbius inversion to:

- remove overcounting over divisors;
- transform sums constrained by gcd;
- invert divisor summations.

Implementation

`mobius_sieve(n)` fills `mu[1..n]` in $O(n)$.

5.17 XOR Linear Basis

Use: Max subset xor, membership, count of distinct xors (2^{rank}).
Complexity: $O(B)$ per insert, $B = 60$.
Pitfalls: Reset basis per test (`b = Basis()`); raise `B` for values $\ge 2^{60}$.
Assumptions/deps: 64-bit safe; `add` returns false if already representable.

```cpp
// XOR linear basis over 64-bit values. add O(B), B = bits.
// rank() = basis size; number of DISTINCT subset xors = 2^rank
// (each achieved by exactly 2^(n-rank) subsets, empty subset gives 0).
// Pitfalls: reset (b = Basis()) between test cases; raise B for values >= 2^60.
struct Basis {
    static const int B = 60;
    long long b[B] = {};
    bool add(long long x) { // false if x already representable
        for (int i = B - 1; i >= 0; i--) if (x >> i & 1) {
            if (!b[i]) { b[i] = x; return true; }
            x ^= b[i];
        }
        return false;
    }
    bool contains(long long x) {
        for (int i = B - 1; i >= 0; i--) if (x >> i & 1) x ^= b[i];
        return x == 0;
    }
    long long maxXor(long long start = 0) {
        long long r = start;
        for (int i = B - 1; i >= 0; i--) r = max(r, r ^ b[i]);
        return r;
    }
    int rank() {
        int r = 0;
        for (int i = 0; i < B; i++) r += b[i] != 0;
        return r;
    }
};
```

5.18 XOR FWHT

Use: XOR convolution.
Complexity: $O(n \log n)$.
Pitfalls: Size must be power of two. Modular inverse needed instead of division for modular values.
Assumptions/deps: Pointwise multiply between forward and inverse transforms.

```cpp
// XOR Fast Walsh-Hadamard Transform.
// n must be a power of two. Use inverse=true after pointwise multiply.
// For modular T, replace division by n with multiplication by inv(n).
// AND/OR convolutions: use superset/subset zeta+mobius (zeta_and_mobius.cpp).
// Time: O(n log n).
template<class T>
void fwht(vector<T>& a, bool inverse = false) {
    int n = (int)a.size();
    for (int len = 1; 2 * len <= n; len <<= 1) {
        for (int i = 0; i < n; i += 2 * len) {
```

```
            for (int j = 0; j < len; j++) {
                T u = a[i + j], v = a[i + len + j];
                a[i + j] = u + v;
                a[i + len + j] = u - v;
            }
        }
    }
    if (inverse) {
        for (T& x : a) x /= n;
    }
}

template<class T>
vector<T> xor_convolution(vector<T> a, vector<T> b) {
    fwht(a);
    fwht(b);
    for (int i = 0; i < (int)a.size(); i++) a[i] *= b[i];
    fwht(a, true);
    return a;
}
```

5.19 Discrete Log

Use: Smallest $x \geq 0$ with $a^x = b \pmod{m}$, any a, b, m.
Complexity: $O(\sqrt{m})$ time/memory.
Pitfalls: Returns -1 if none. Non-coprime handled by factor stripping.
Assumptions/deps: Requires `chash` (misc/custom_hash).

```
// Discrete log: smallest x >= 0 with a^x = b (mod m), or -1 if none.
// Works for ANY a, b, m >= 1 (no coprimality/primality needed).
// O(sqrt(m)) time and memory. Requires chash from misc/custom_hash.cpp
// (identity-hash unordered_map can be degraded to O(m) by chosen m).
// Source: cp-algorithms extended BSGS.
long long discrete_log(long long a, long long b, long long m) {
    a %= m; b %= m;
    if (a < 0) a += m;
    if (b < 0) b += m;
    long long k = 1 % m, add = 0, g;
    while ((g = __gcd(a, m)) > 1) {
        if (b == k) return add;
        if (b % g) return -1;
        b /= g; m /= g; add++;
        k = (__int128)k * (a / g) % m;
    }
    // now gcd(a, m) = 1: find a^(n*p) * k = b * a^q
    long long n = (long long)sqrtl(m) + 1;
    unordered_map<long long, long long, chash> vals;
    long long cur = b;
    for (long long q = 0; q <= n; q++) {
        vals[cur] = q; // keep the LARGEST q per value -> smallest answer
        cur = (__int128)cur * a % m;
    }
    long long an = 1;
    for (long long i = 0; i < n; i++) an = (__int128)an * a % m;
    cur = k;
    for (long long p = 1; p <= n; p++) {
        cur = (__int128)cur * an % m;
        auto it = vals.find(cur);
        if (it != vals.end()) return n * p - it->second + add;
    }
    return -1;
}
```

5.20 Discrete kth Roots mod Prime

Use: Solve $x^k = a \pmod{p}$.
Complexity: $O(\sqrt{p} \log p)$.
Pitfalls: `p` must be prime. Returns distinct sorted roots; `a=0` returns `{0}`.
Assumptions/deps: Requires NT Essentials, Primitive Root, and `chash`.

```
// Solve x^k = a (mod p) for prime p, via primitive root + BSGS index log.
// Requires NT_essentials.cpp (modpow, inv_any), primitive_root.cpp, and
// chash from misc/custom_hash.cpp.
// Complexity: O(sqrt(p) log p). Returns sorted distinct roots; a=0 gives {0}.
// Usage: vector<long long> xs = kth_roots_mod_prime(a, k, p);
long long discrete_log_prime_root(long long g, long long a, long long p) {
    // g must be a primitive root modulo prime p, and a != 0.
    long long m = (long long)sqrtl(p - 1) + 1;
    unordered_map<long long, long long, chash> baby;
    long long cur = 1;
    for (long long j = 0; j < m; j++) {
        if (!baby.count(cur)) baby[cur] = j;
        cur = (__int128)cur * g % p;
    }
    long long step = modpow(modpow(g, m, p), p - 2, p);
    cur = a % p;
    for (long long i = 0; i <= m; i++) {
        auto it = baby.find(cur);
        if (it != baby.end()) return i * m + it->second;
        cur = (__int128)cur * step % p;
    }
    return -1;
}

vector<long long> kth_roots_mod_prime(long long a, long long k, long long p) {
    a %= p;
```

```
    if (a < 0) a += p;
    if (a == 0) return {0};
    if (p == 2) return {1};

    long long g = generator(p);
    long long ind = discrete_log_prime_root(g, a, p); // a = g^ind
    if (ind == -1) return {};

    long long mod = p - 1, d = __gcd(k, mod);
    if (ind % d) return {};

    long long kk = ((k / d) % (mod / d) + (mod / d)) % (mod / d);
    long long b = ind / d, md = mod / d;
    long long y0 = (__int128)b * inv_any(kk, md) % md;

    vector<long long> ans;
    for (long long t = 0; t < d; t++) ans.push_back(modpow(g, y0 + t * md, p));
    sort(ans.begin(), ans.end());
    ans.erase(unique(ans.begin(), ans.end()), ans.end());
    return ans;
}
```

5.21 FFT

Use: Polynomial convolution over doubles.
Complexity: $O(n \log n)$.
Pitfalls: Rounding fails for large coefficient energy; n rounded to power of two.
Assumptions/deps: Requires complex literals and helper macros `sz/all/rep`.

```
/**
 * Description: fft(a) computes $\hat f(k) = \sum_x a[x] \exp(2\pi i \cdot k x / N)$
     for all $k$. N must be a power of 2.
   Useful for convolution:
   \texttt{conv(a, b) = c}, where $c[x] = \sum a[i]b[x-i]$.
   For convolution of complex numbers or more than two vectors: FFT, multiply
   pointwise, divide by n, reverse(start+1, end), FFT back.
   Rounding is safe if $(\sum a_i^2 + \sum b_i^2)\log_2{N} < 9\cdot10^{14}$
   (in practice $10^{16}$; higher for random inputs).
   Otherwise, use NTT/FFTMod.
 * Time: O(N \log N) with $N = |A|+|B|$ ($\tilde 1s$ for $N=2^{22}$)
 * Status: somewhat tested
 */
using ll = long long;
using vi = vector<int>;
#ifndef sz
#define sz(x) (int)(x).size()
#endif
#ifndef all
#define all(x) begin(x), end(x)
#endif
#ifndef rep
#define rep(i,a,b) for (int i = (a); i < (b); ++i)
#endif
using namespace std::complex_literals;

typedef complex<double> C;
typedef vector<double> vd;
void fft(vector<C>& a) {
  int n = sz(a), L = 31 - __builtin_clz(n);
  static vector<complex<long double>> R(2, 1);
  static vector<C> rt(2, 1); // (^ 10% faster if double)
  for (static int k = 2; k < n; k *= 2) {
    R.resize(n); rt.resize(n);
    auto x = polar(1.0L, acos(-1.0L) / k);
    rep(i,k,2*k) rt[i] = R[i] = i&1 ? R[i/2] * x : R[i/2];
  }
  vi rev(n);
  rep(i,0,n) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
  rep(i,0,n) if (i < rev[i]) swap(a[i], a[rev[i]]);
  for (int k = 1; k < n; k *= 2)
    for (int i = 0; i < n; i += 2 * k) rep(j,0,k) {
      // hand-rolled z = rt[j+k] * a[i+j+k], ~25% faster than complex*
      auto x = (double *)&rt[j+k], y = (double *)&a[i+j+k];
      C z(x[0]*y[0] - x[1]*y[1], x[0]*y[1] + x[1]*y[0]);
      a[i + j + k] = a[i + j] - z;
      a[i + j] += z;
    }
}
vd conv(const vd& a, const vd& b) {
  if (a.empty() || b.empty()) return {};
  vd res(sz(a) + sz(b) - 1);
  int L = 32 - __builtin_clz(sz(res)), n = 1 << L;
  vector<C> in(n), out(n);
  copy(all(a), begin(in));
  rep(i,0,sz(b)) in[i].imag(b[i]);
  fft(in);
  for (C& x : in) x *= x;
  rep(i,0,n) out[i] = in[-i & (n - 1)] - conj(in[i]);
  fft(out);
  rep(i,0,sz(res)) res[i] = imag(out[i]) / (4 * n);
  return res;
}
```

5.22 FFT Mod

Use: Convolution modulo arbitrary int.
Complexity: $O(n \log n)$.
Pitfalls: Inputs must be in `[0,M)`; precision bound matters.
Assumptions/deps: Requires `fft_v2.cpp` before this snippet.

```cpp
/**
 * Description: Higher precision FFT, can be used for convolutions modulo arbitrary integers
 * as long as $N\log_2N\cdot \text{mod} < 8.6 \cdot 10^{14}$ (in practice $10^{16}$ or higher).
 * Inputs must be in $[0, \text{mod})$.
 * Time: O(N \log N), where $N = |A|+|B|$ (twice as slow as NTT or FFT)
 * Status: stress-tested
 */
// Requires fft_v2.cpp printed above this snippet (defines C, fft, ll, rep, sz).
typedef vector<ll> vl;
template<int M> vl convMod(const vl &a, const vl &b) {
  if (a.empty() || b.empty()) return {};
  vl res(sz(a) + sz(b) - 1);
  int B=32-__builtin_clz(sz(res)), n=1<<B, cut=int(sqrt(M));
  vector<C> L(n), R(n), outs(n), outl(n);
  rep(i,0,sz(a)) L[i] = C((int)a[i] / cut, (int)a[i] % cut);
  rep(i,0,sz(b)) R[i] = C((int)b[i] / cut, (int)b[i] % cut);
  fft(L), fft(R);
  rep(i,0,n) {
    int j = -i & (n - 1);
    outl[j] = (L[i] + conj(L[j])) * R[i] / (2.0 * n);
    outs[j] = (L[i] - conj(L[j])) * R[i] / (2.0 * n) / 1i;
  }
  fft(outl), fft(outs);
  rep(i,0,sz(res)) {
    ll av = ll(real(outl[i])+.5), cv = ll(imag(outs[i])+.5);
    ll bv = ll(imag(outl[i])+.5) + ll(real(outs[i])+.5);
    res[i] = ((av % M * cut + bv) % M * cut + cv) % M;
  }
  return res;
}
```

5.23 NTT

Use: Exact convolution mod 998244353.
Complexity: $O(n \log n)$, sizes to 2^{23}.
Pitfalls: Inputs in `[0,mod)`; only NTT-friendly primes (list in header).
Assumptions/deps: Self-contained; prefer over FFT when the modulus fits.

```cpp
// NTT convolution mod 998244353 (primitive root 3). Exact, no precision issues.
// conv_ntt(a,b)[k] = sum a[i]*b[k-i] mod p. O(n log n), sizes up to 2^23.
// Inputs must be in [0, mod). For arbitrary mod use fft_mod.cpp.
// Other NTT primes (root 3): 167772161, 469762049, 1004535809.
using ll = long long;
const ll NTT_MOD = 998244353, NTT_G = 3;

ll ntt_pow(ll a, ll e) {
    ll r = 1;
    for (a %= NTT_MOD; e; e >>= 1, a = a * a % NTT_MOD)
        if (e & 1) r = r * a % NTT_MOD;
    return r;
}

void ntt(vector<ll>& a, bool inv) {
    int n = a.size(); // power of 2
    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1) j ^= bit;
        j ^= bit;
        if (i < j) swap(a[i], a[j]);
    }
    for (int len = 2; len <= n; len <<= 1) {
        ll w = ntt_pow(NTT_G, (NTT_MOD - 1) / len);
        if (inv) w = ntt_pow(w, NTT_MOD - 2);
        for (int i = 0; i < n; i += len) {
            ll wn = 1;
            for (int j = 0; j < len / 2; j++, wn = wn * w % NTT_MOD) {
                ll u = a[i + j], v = a[i + j + len / 2] * wn % NTT_MOD;
                a[i + j] = (u + v) % NTT_MOD;
                a[i + j + len / 2] = (u - v + NTT_MOD) % NTT_MOD;
            }
        }
    }
    if (inv) {
        ll ninv = ntt_pow(n, NTT_MOD - 2);
        for (ll& x : a) x = x * ninv % NTT_MOD;
    }
}

vector<ll> conv_ntt(vector<ll> a, vector<ll> b) {
    if (a.empty() || b.empty()) return {};
    int res = a.size() + b.size() - 1, n = 1;
    while (n < res) n <<= 1;
    assert(n <= (1 << 23)); // beyond 2^23 the roots silently break for this mod
    a.resize(n); b.resize(n);
    ntt(a, false); ntt(b, false);
    for (int i = 0; i < n; i++) a[i] = a[i] * b[i] % NTT_MOD;
    ntt(a, true);
    a.resize(res);
    return a;
}
```

5.24 Formal Power Series

Use: Polynomial inverse, logarithm, and exponential modulo 998244353.
Complexity: $O(n \log^2 n)$.
Pitfalls: Constants must satisfy inverse $a_0 \neq 0$, $\log a_0 = 1$, $\exp a_0 = 0$.
Assumptions/deps: Requires the NTT immediately above; respects its size limit.

```cpp
// Formal power series inverse/log/exp modulo 998244353. Depends on ntt.cpp.
// Returns first n coefficients. Complexity O(n log^2 n), memory O(n).
// Pitfalls: inverse needs a[0]!=0; log needs a[0]=1; exp needs a[0]=0;
// n and inputs must fit the NTT modulus/2^23 limits.
// Usage: fps_inv(a,n), fps_log(a,n), fps_exp(a,n).
vector<ll> fps_inv(const vector<ll>& a,int n) {
    if(!n)return {};
    assert(!a.empty() && a[0]!=0);
    vector<ll>r={ntt_pow(a[0],NTT_MOD-2)};
    while((int)r.size()<n) {
        int next=min(n,2*(int)r.size());
        vector<ll>f(next);
        for(int i=0;i<next&&i<(int)a.size();++i)f[i]=a[i];
        auto prod=conv_ntt(f,r);prod.resize(next);
        for(ll&x:prod)x=(NTT_MOD-x)%NTT_MOD;
        prod[0]=(prod[0]+2)%NTT_MOD;
        r=conv_ntt(r,prod);r.resize(next);
    }
    return r;
}
vector<ll> fps_derivative(const vector<ll>& a) {
    vector<ll>d(max(0,(int)a.size()-1));
    for(int i=1;i<(int)a.size();++i)d[i-1]=a[i]*i%NTT_MOD;
    return d;
}
vector<ll> fps_integral(const vector<ll>& a) {
    vector<ll>inverses(a.size()+1),r(a.size()+1);if(!a.empty())inverses[1]=1;
    for(int i=2;i<=(int)a.size();++i)inverses[i]=NTT_MOD-NTT_MOD/i*inverses[NTT_MOD%
        i]%NTT_MOD;
    for(int i=0;i<(int)a.size();++i)r[i+1]=a[i]*inverses[i+1]%NTT_MOD;
    return r;
}
vector<ll> fps_log(const vector<ll>& a,int n) {
    if(!n)return {};
    assert(!a.empty() && a[0]==1);
    auto r=conv_ntt(fps_derivative(a),fps_inv(a,n));
    if((int)r.size()>n-1)r.resize(n-1);
    r=fps_integral(r);r.resize(n);
    return r;
}
vector<ll> fps_exp(const vector<ll>& a,int n) {
    if(!n)return {};
    assert(a.empty() || a[0]==0);
    vector<ll>r={1};
    while((int)r.size()<n) {
        int next=min(n,2*(int)r.size());
        auto logarithm=fps_log(r,next);
        vector<ll>delta(next);
        for(int i=0;i<next;++i) {
            ll value=i<(int)a.size()?a[i]:0;
            delta[i]=(value-logarithm[i]+NTT_MOD)%NTT_MOD;
        }
        delta[0]=(delta[0]+1)%NTT_MOD;
        r=conv_ntt(r,delta);r.resize(next);
    }
    return r;
}
```

5.25 FPS Newton Formulas

Use: Precision-doubling identities for inverse/log/exp.
Complexity: $O(n \log^2 n)$ implementation.
Pitfalls: Formal series are truncated modulo x^n.
Assumptions/deps: Pairs with NTT and FPS code.

All identities are modulo x^n over $\mathbb{F}_{998244353}$.

Inverse

Given $a_0 \neq 0$, start with $b = a_0^{-1}$ and double precision using

$$b \leftarrow b(2 - ab).$$

If $ab \equiv 1 \pmod{x^m}$, the new b is correct modulo x^{2m}.

Logarithm and exponential

For $a_0 = 1$,

$$\log a = \int \frac{a'}{a}.$$

For $a_0 = 0$, start with $b = 1$ and repeatedly apply

$$b \leftarrow b(1 + a - \log b).$$

The implementation uses the notebook NTT and the simple $O(n \log^2 n)$ doubling form. NTT root limits and coefficient normalization still apply.

5.26 Fraction Finder

Use: Binary search over rationals with bounded numerator/denominator.
Complexity: $O(\log N)$ calls.
Pitfalls: Predicate must be monotone and true at `hi`; overflow possible in mediants.
Assumptions/deps: Uses `ll`.

```
/**
 * Description: Given $f$ and $N$, finds the smallest fraction $p/q \in [0, 1]$
 * such that $f(p/q)$ is true, and $p, q \le N$.
 * You may want to throw an exception from $f$ if it finds an exact solution,
 * in which case $N$ can be removed.
 * Usage: fracBS([](Frac f) { return f.p>=3*f.q; }, 10); // {1,3}
 * Time: O(\log(N))
 * Status: stress-tested for n <= 300
 */

using ll = long long;
struct Frac { ll p, q; };

template<class F>
Frac fracBS(F f, ll N) {
  bool dir = 1, A = 1, B = 1;
  Frac lo{0, 1}, hi{1, 1}; // Set hi to 1/0 to search (0, N]
  if (f(lo)) return lo;
  assert(f(hi));
  while (A || B) {
    ll adv = 0, step = 1; // move hi if dir, else lo
    for (int si = 0; step; (step *= 2) >>= si) {
      adv += step;
      Frac mid{lo.p * adv + hi.p, lo.q * adv + hi.q};
      if (abs(mid.p) > N || mid.q > N || dir == !f(mid)) {
        adv -= step; si = 2;
      }
    }
    hi.p += lo.p * adv;
    hi.q += lo.q * adv;
    dir = !dir;
    swap(lo, hi);
    A = B; B = !!adv;
  }
  return dir ? hi : lo;
}
```

5.27 Zeta and Mobius Transforms

Use: Subset/superset/divisor/multiple sums and AND/OR/gcd/lcm convolutions.
Complexity: Subset/superset $O(n \log n)$; divisor/multiple $O(n \log \log n)$.
Pitfalls: Subset versions need power-of-two size; divisor versions are indexed `1..n`.
Assumptions/deps: Self-contained (inline prime sieve). Header lists convolution recipes.

```
// Zeta/Mobius transforms. Subset/superset: v.size() must be a power of 2,
// O(n log n). AND-convolution: supersetZeta both, multiply, supersetMobius.
// OR-convolution: subsetZeta both, multiply, subsetMobius.
// Divisor/multiple: v indexed 1..n (v[0] unused), O(n log log n).
// divisorZeta = F(i) = sum_{d|i} f(d); multipleZeta = F(i) = sum_{i|m} f(m).
// Mobius versions invert their zeta. gcd-convolution: multipleZeta both,
// multiply, multipleMobius (lcm-conv: divisor versions).
template<class T> void subsetZeta(vector<T>& v) {
    int n = v.size();
    for (int j = 1; j < n; j <<= 1)
        for (int i = 0; i < n; i++) if (i & j) v[i] += v[i ^ j];
}
template<class T> void subsetMobius(vector<T>& v) {
    int n = v.size();
    for (int j = 1; j < n; j <<= 1)
        for (int i = 0; i < n; i++) if (i & j) v[i] -= v[i ^ j];
}
template<class T> void supersetZeta(vector<T>& v) {
    int n = v.size();
    for (int j = 1; j < n; j <<= 1)
        for (int i = 0; i < n; i++) if (i & j) v[i ^ j] += v[i];
}
template<class T> void supersetMobius(vector<T>& v) {
    int n = v.size();
    for (int j = 1; j < n; j <<= 1)
        for (int i = 0; i < n; i++) if (i & j) v[i ^ j] -= v[i];
}
vector<int> primesUpTo(int n) {
    vector<int> pr; vector<bool> comp(n + 1);
    for (int p = 2; p <= n; p++) if (!comp[p]) {
        pr.push_back(p);
        for (int j = 2 * p; j <= n; j += p) comp[j] = true;
    }
    return pr;
}
template<class T> void divisorZeta(vector<T>& v) {
    int n = (int)v.size() - 1;
    for (int p : primesUpTo(n))
        for (int i = 1; i * p <= n; i++) v[i * p] += v[i];
}
template<class T> void divisorMobius(vector<T>& v) {
    int n = (int)v.size() - 1;
```

```
    for (int p : primesUpTo(n))
        for (int i = n / p; i; i--) v[i * p] -= v[i];
}
template<class T> void multipleZeta(vector<T>& v) {
    int n = (int)v.size() - 1;
    for (int p : primesUpTo(n))
        for (int i = n / p; i; i--) v[i] += v[i * p];
}
template<class T> void multipleMobius(vector<T>& v) {
    int n = (int)v.size() - 1;
    for (int p : primesUpTo(n))
        for (int i = 1; i * p <= n; i++) v[i] -= v[i * p];
}
```

5.28 Inclusion–Exclusion

Use: Union sizes and onto-function counting helpers.
Complexity: $O(2^k)$ or $O(k^2 + k \log n)$.
Pitfalls: Intersection values must have exact mask meaning; union totals can overflow.
Assumptions/deps: Also view as subset-lattice Mobius inversion.

```
// Two concrete inclusion-exclusion helpers: union size from all intersections,
// and onto functions from n labeled items to k labeled boxes modulo mod.
// Complexity: O(2^k) / O(k^2+k log n). Memory O(k).
// Pitfalls: intersection[mask] must mean intersection of every set in mask;
// union answer may overflow long long. mod need not be prime.
// Usage: union_size_ie(intersections); surjections_ie(n,k,mod).
__extension__ typedef __int128 IEInt;
long long union_size_ie(const vector<long long>& intersection) {
    long long ans=0;
    for(int mask=1;mask<(int)intersection.size();++mask)
        ans+=(__builtin_popcount((unsigned)mask)&1?1:-1)*intersection[mask];
    return ans;
}
long long surjections_ie(long long n,int k,long long mod) {
    assert(mod>0);
    vector<long long>C(k+1);C[0]=1;
    for(int i=1;i<=k;++i) for(int j=i;j;--j) C[j]=(C[j]+C[j-1])%mod;
    auto power=[&](long long a,long long e){long long r=1%mod;for(;e;e>>=1,a=(IEInt)a
        *a%mod)if(e&1)r=(IEInt)r*a%mod;return r;};
    long long ans=0;
    for(int i=0;i<=k;++i) {
        long long term=(IEInt)C[i]*power(k-i,n)%mod;
        ans=((i&1)?ans-term:ans+term)%mod;
    }
    return (ans+mod)%mod;
}
```

5.29 Inclusion–Exclusion Formulas

Use: Union, complement, surjection, and derangement reminders.
Complexity: Formula-dependent.
Pitfalls: Define the universe and empty intersection explicitly.
Assumptions/deps: Use subset transforms when all masks are required.

For finite sets $A_1, \ldots, A_k$,

$$\left|\bigcup_i A_i\right| = \sum_{\emptyset \neq S \subseteq [k]} (-1)^{|S|+1} \left|\bigcap_{i \in S} A_i\right|.$$

To count objects avoiding bad properties B_i, apply this to the complement: total minus single bad sets, plus pair intersections, and so on.

Standard formulas

Onto functions $[n] \to [k]$:

$$\sum_{i=0}^{k} (-1)^i \binom{k}{i} (k-i)^n.$$

Derangements:

$$!n = n! \sum_{i=0}^{n} \frac{(-1)^i}{i!}.$$

Inclusion-exclusion is Möbius inversion on the subset lattice. Use subset transforms when all 2^k masks are needed.

Pitfalls: define the universe and empty intersection; do not double-count inside an intersection; use a safe type or modulus.

5.30 Sprague–Grundy

Use: Solve finite impartial normal-play games on a DAG.
Complexity: $O(n + m)$ plus mex work.
Pitfalls: Returns empty on a cycle; game moves must be finite and impartial.
Assumptions/deps: Xor Grundy values of independent starting positions.

```
// Grundy values of a finite DAG; xor values to combine impartial games.
// Complexity: O(n+m) plus mex work bounded by total outdegree.
// Pitfalls: returns {} on a directed cycle; moves must define a finite DAG.
// Usage: auto sg=grundy_dag(moves); answer xor of starting sg values.
```

```cpp
vector<int> grundy_dag(const vector<vector<int>>& g) {
    int n = (int)g.size();
    vector<int> sg(n), color(n);
    bool ok = true;
    function<void(int)> dfs = [&](int v) {
        color[v] = 1;
        for (int to : g[v]) {
            if (color[to] == 0) dfs(to);
            else if (color[to] == 1) ok = false;
        }
        vector<int> seen(g[v].size() + 1);
        for (int to : g[v]) if (sg[to] < (int)seen.size()) seen[sg[to]] = 1;
        while (sg[v] < (int)seen.size() && seen[sg[v]]) ++sg[v];
        color[v] = 2;
    };
    for (int v = 0; v < n; ++v) if (!color[v]) dfs(v);
    return ok ? sg : vector<int>{};
}
```

5.31 Floor Sum

Use: Evaluate $\sum_{i=0}^{n-1} \lfloor (ai+b)/m \rfloor$.
Complexity: $O(\log \max(a, m))$.
Pitfalls: Nonnegative arguments only, $m > 0$; returns `__int128`.
Assumptions/deps: Euclidean recurrence; see derivation below.

```cpp
// Sum floor((a*i+b)/m) for i=0..n-1, all arguments nonnegative, m>0.
// Complexity: O(log(max(a,m))) time, O(1) memory (Euclidean recurrence).
// Pitfalls: this is the nonnegative version; answer/products need i128.
// Usage: i128 ans=floor_sum(n,m,a,b);
__int128 floor_sum(long long n,long long m,long long a,long long b) {
    __int128 ans=0;
    while(true) {
        if(a>=m) {
            ans+=(__int128)(n-1)*n*(a/m)/2;
            a%=m;
        }
        if(b>=m) {
            ans+=(__int128)n*(b/m);
            b%=m;
        }
        __int128 y=(__int128)a*n+b;
        if(y<m) return ans;
        n=y/m;
        b=y%m;
        swap(a,m);
    }
}
```

5.32 Floor-Sum Notes

Use: Derivation and signed-input warning.
Complexity: $O(\log \max(a, m))$.
Pitfalls: C++ division is not mathematical floor for negative inputs.
Assumptions/deps: Pairs with the nonnegative implementation above.

For $n, a, b \geq 0$ and $m > 0$,

$$F(n, m, a, b) = \sum_{0 \leq i < n} \left\lfloor \frac{ai+b}{m} \right\rfloor.$$

First remove whole multiples:

- $a = q_a m + r_a$ contributes $q_a n(n-1)/2$;
- $b = q_b m + r_b$ contributes $q_b n$.

Now $0 \leq a, b < m$. Let $y = an + b$. If $y < m$, the remainder is zero. Otherwise reflect the lattice points and swap a, m:

$$n' = \left\lfloor \frac{y}{m} \right\rfloor, \qquad b' = y \bmod m,$$

then continue with (n', a, m, b'). This is Euclidean-algorithm-like, hence logarithmic. For signed inputs, normalize with mathematical floor division; C++ integer division truncates toward zero.

5.33 Lagrange Interpolation

Use: Evaluate a polynomial from values at $0, 1, \ldots, n-1$.
Complexity: $O(n + \log MOD)$ per evaluation.
Pitfalls: Prime modulus, $n < MOD$; degree must be below n.
Assumptions/deps: Change `LAGRANGE_MOD`; consecutive-point optimized form.

```cpp
// Evaluate degree < n polynomial from y[i]=f(i), modulo prime LAGRANGE_MOD.
// Complexity: O(n + log MOD) time, O(n) memory per evaluation.
// Pitfalls: n<MOD; values/x are interpreted modulo MOD; MOD must be prime.
// Usage: long long value=lagrangeConsecutive(y,x);
const long long LAGRANGE_MOD=1000000007;
long long lagPow(long long a,long long e) {
    long long r=1;
    for(;e;e>>=1,a=a*a%LAGRANGE_MOD) if(e&1) r=r*a%LAGRANGE_MOD;
    return r;
}
long long lagrangeConsecutive(vector<long long> y,long long x) {
```

```cpp
    int n=y.size(); if(!n) return 0;
    x=(x%LAGRANGE_MOD+LAGRANGE_MOD)%LAGRANGE_MOD;
    for(auto& v:y) v=(v%LAGRANGE_MOD+LAGRANGE_MOD)%LAGRANGE_MOD;
    if(x<n) return y[x];
    vector<long long> pre(n+1,1),suf(n+1,1),fact(n),ifact(n);
    for(int i=0;i<n;++i) pre[i+1]=pre[i]*(x-i+LAGRANGE_MOD)%LAGRANGE_MOD;
    for(int i=n-1;i>=0;--i) suf[i]=suf[i+1]*(x-i+LAGRANGE_MOD)%LAGRANGE_MOD;
    fact[0]=1;for(int i=1;i<n;++i)fact[i]=fact[i-1]*i%LAGRANGE_MOD;
    ifact[n-1]=lagPow(fact[n-1],LAGRANGE_MOD-2);
    for(int i=n-1;i;--i)ifact[i-1]=ifact[i]*i%LAGRANGE_MOD;
    long long ans=0;
    for(int i=0;i<n;++i) {
        long long term=y[i]*pre[i]%LAGRANGE_MOD*suf[i+1]%LAGRANGE_MOD;
        term=term*ifact[i]%LAGRANGE_MOD*ifact[n-1-i]%LAGRANGE_MOD;
        if((n-1-i)&1) ans=(ans-term+LAGRANGE_MOD)%LAGRANGE_MOD;
        else ans=(ans+term)%LAGRANGE_MOD;
    }
    return ans;
}
```

5.34 Lagrange Formula

Use: Consecutive-point denominator and prefix/suffix products.
Complexity: $O(n)$ after one inverse.
Pitfalls: Return samples directly when x is a known point.
Assumptions/deps: Pairs with interpolation implementation above.

Given $y_i = f(i)$ for $0 \le i < n$ and $\deg f < n$,

$$f(x) = \sum_{i=0}^{n-1} y_i \frac{\prod_{j \neq i}(x-j)}{\prod_{j \neq i}(i-j)}.$$

At consecutive sample points,

$$\prod_{j \neq i}(i-j) = i!\,(-1)^{n-1-i}(n-1-i)!.$$

Prefix/suffix products of $(x-j)$ plus factorials and inverse factorials make each term $O(1)$, hence $O(n)$ total after one modular inverse.

Pitfalls: return y_x immediately when x is a sample index; otherwise the general numerator contains zero. Modular division normally needs prime MOD and $n < MOD$. This reconstructs only polynomials of degree below n.

6 Geometry

Checklist: integer or floating? EPS? collinear kept or removed? boundary included? polygon order? coordinates up to 10^9 may need `__int128`. Circles need tangent/same-center cases.

6.1 Geometry Base

Use: Shared floating 2D point/vector primitives.
Complexity: Each primitive $O(1)$.
Pitfalls: Exact `operator<` but EPS equality; avoid mixing with integer exact logic.
Assumptions/deps: Copy before other floating geometry snippets.

```cpp
/*
GEOMETRY CHECKLIST
1. int or floating?
2. Need EPS?
3. Coordinates up to 1e9 => integer cross may need __int128.
4. Segment intersection: endpoint? overlap? collinear?
5. Convex hull: keep or remove collinear points?
6. Polygon vertices clockwise or counterclockwise?
7. Area is signed; use abs(area2)/2.
8. For angle/sqrt, clamp values if needed.
9. For circles, handle tangent and same-center cases.
*/

// Shared floating-point geometry base.
// Source: KACTL Point style, simplified for notebook use.
// Complexity: each primitive is O(1).
// Pitfalls: operator< is exact for sort/set; geometric equality uses EPS.
// Usage: copy this before other floating geometry snippets.
using T = long double;
const T EPS = 1e-12;

int sgn(T x) { return (x > EPS) - (x < -EPS); }

struct P {
    T x, y;
    bool operator<(P p) const {
        if (x != p.x) return x < p.x;
        return y < p.y;
    }
    bool operator==(P p) const {
        return sgn(x - p.x) == 0 && sgn(y - p.y) == 0;
    }
```

```
    P operator+(P p) const { return {x + p.x, y + p.y}; }
    P operator-(P p) const { return {x - p.x, y - p.y}; }
    P operator*(T k) const { return {x * k, y * k}; }
    P operator/(T k) const { return {x / k, y / k}; }
};

T dot(P a, P b) { return a.x * b.x + a.y * b.y; }
T cross(P a, P b) { return a.x * b.y - a.y * b.x; }
T cross(P a, P b, P c) { return cross(b - a, c - a); }
T norm2(P a) { return dot(a, a); }
T norm(P a) { return sqrtl(norm2(a)); }
P perp(P a) { return {-a.y, a.x}; }

int ori(P a, P b, P c) { return sgn(cross(a, b, c)); }

bool onSegment(P a, P b, P p) {
    return ori(a, b, p) == 0 && sgn(dot(p - a, p - b)) <= 0;
}
```

6.2 Integer Geometry

Use: Exact orientation for large integer coordinates.
Complexity: $O(1)$.
Pitfalls: Use `__int128` for products near 10^{18}; return only sign.
Assumptions/deps: Defines `IP`, `i128`.

```
// Integer geometry orientation helper for large coordinates.
// Complexity: O(1).
// Pitfalls: use __int128 for cross when coordinates/products may overflow long long.
// Usage: int o = ori_i(a, b, c); // -1, 0, +1
using ll = long long;
__extension__ typedef __int128 i128;

struct IP {
    ll x, y;
};

i128 cross_i(IP a, IP b, IP c) {
    return (i128)(b.x - a.x) * (c.y - a.y) -
           (i128)(b.y - a.y) * (c.x - a.x);
}

int ori_i(IP a, IP b, IP c) {
    i128 v = cross_i(a, b, c);
    return (v > 0) - (v < 0);
}
```

6.3 Convex Hull and Geometry Routines

Use: Segments, polygon tests, hull, diameter, circle intersections.
Complexity: Most $O(1)$; polygon/hull $O(n)$ or $O(n \log n)$.
Pitfalls: Hull removes collinear boundary points with `<=`. Line intersection assumes nonparallel lines.
Assumptions/deps: Depends on `geometry_base.cpp`; hull output has no duplicate last point.

```
// Core 2D geometry routines. Depends on geometry_base.cpp.
// Sources: KACTL OnSegment/LineIntersection/InsidePolygon/ConvexHull/HullDiameter/
    CircleLine/CircleIntersection style.
// Complexity: listed per function below. Uses 0-indexed vectors.
// Usage: copy geometry_base.cpp first.

// segment intersection. O(1).
// Pitfalls: segInter only says yes/no; overlapping intersection needs separate
    handling.
bool segInter(P a, P b, P c, P d) {
    int oa = ori(c, d, a), ob = ori(c, d, b);
    int oc = ori(a, b, c), od = ori(a, b, d);
    if (oa == 0 && onSegment(c, d, a)) return true;
    if (ob == 0 && onSegment(c, d, b)) return true;
    if (oc == 0 && onSegment(a, b, c)) return true;
    if (od == 0 && onSegment(a, b, d)) return true;
    return oa * ob < 0 && oc * od < 0;
}

// line intersection/projection/distances. O(1).
// Pitfalls: lineInter assumes non-parallel non-zero-length lines.
P lineInter(P a, P b, P c, P d) {
    P r = b - a, s = d - c;
    return a + r * (cross(c - a, s) / cross(r, s));
}

P projLine(P a, P b, P p) {
    P ab = b - a;
    return a + ab * (dot(p - a, ab) / norm2(ab));
}

T distLine(P a, P b, P p) {
    return abs(cross(b - a, p - a)) / norm(b - a);
}

T distSegment(P a, P b, P p) {
    if (a == b) return norm(p - a);
    if (sgn(dot(p - a, b - a)) <= 0) return norm(p - a);
    if (sgn(dot(p - b, a - b)) <= 0) return norm(p - b);
    return distLine(a, b, p);
}
```

```cpp
// polygon signed double-area. O(n).
// Pitfalls: clockwise order gives negative area.
T area2(const vector<P>& p) {
    T s = 0;
    for (int i = 0, n = (int)p.size(); i < n; i++) s += cross(p[i], p[(i + 1) % n]);
    return s;
}

// point in polygon: 0 outside, 1 inside, 2 boundary. O(n).
// Pitfalls: works for simple polygons; boundary is checked first.
int inPoly(const vector<P>& poly, P q) {
    bool in = false;
    int n = (int)poly.size();
    for (int i = 0, j = n - 1; i < n; j = i++) {
        P a = poly[i], b = poly[j];
        if (onSegment(a, b, q)) return 2;
        bool cond = (a.y > q.y) != (b.y > q.y);
        if (cond) {
            T x = a.x + (b.x - a.x) * (q.y - a.y) / (b.y - a.y);
            if (sgn(x - q.x) > 0) in = !in;
        }
    }
    return in;
}

// convex hull in CCW order. O(n log n).
// Pitfalls: <= removes collinear boundary points; change to < to keep them.
vector<P> convexHull(vector<P> p) {
    sort(p.begin(), p.end());
    p.erase(unique(p.begin(), p.end()), p.end());
    if ((int)p.size() <= 1) return p;
    vector<P> h;
    for (P pt : p) {
        while ((int)h.size() >= 2 && sgn(cross(h[(int)h.size() - 2], h.back(), pt))
             <= 0)
            h.pop_back();
        h.push_back(pt);
    }
    int lower = (int)h.size();
    for (int i = (int)p.size() - 2; i >= 0; i--) {
        P pt = p[i];
        while ((int)h.size() > lower && sgn(cross(h[(int)h.size() - 2], h.back(), pt)
             ) <= 0)
            h.pop_back();
        h.push_back(pt);
    }
    h.pop_back();
    return h;
}

// squared diameter of a convex polygon in CCW order. O(n).
// Pitfalls: run convexHull first; hull should have no duplicate last point.
T diameter2(const vector<P>& p) {
    int n = (int)p.size();
    if (n <= 1) return 0;
    if (n == 2) return norm2(p[0] - p[1]);
    T ans = 0;
    int j = 1;
    for (int i = 0; i < n; i++) {
        int ni = (i + 1) % n;
        while (abs(cross(p[ni] - p[i], p[(j + 1) % n] - p[i])) >
               abs(cross(p[ni] - p[i], p[j] - p[i])))
            j = (j + 1) % n;
        ans = max(ans, norm2(p[i] - p[j]));
    }
    return ans;
}

// circle-line intersection. O(1).
// Pitfalls: a and b must be distinct; returns 0/1/2 points.
vector<P> circleLine(P c, T r, P a, P b) {
    P ab = b - a;
    P h = a + ab * (dot(c - a, ab) / norm2(ab));
    T d2 = norm2(h - c);
    if (sgn(d2 - r * r) > 0) return {};
    P dir = ab / norm(ab);
    if (sgn(d2 - r * r) == 0) return {h};
    T len = sqrtl(max((T)0, r * r - d2));
    return {h - dir * len, h + dir * len};
}

// circle-circle intersection. O(1).
// Pitfalls: same center returns empty for both infinite and no intersections.
vector<P> circleCircle(P a, T r, P b, T R) {
    P ab = b - a;
    T d2 = norm2(ab), d = sqrtl(d2);
    if (sgn(d) == 0) return {};
    T x = (d2 + r * r - R * R) / (2 * d);
    T h2 = r * r - x * x;
    if (sgn(h2) < 0) return {};
    P v = ab / d, p = a + v * x, q = perp(v);
    if (sgn(h2) == 0) return {p};
    T h = sqrtl(max((T)0, h2));
    return {p - q * h, p + q * h};
}
```

6.4 Point in Convex Polygon

Use: Fast containment query in a convex polygon.
Complexity: $O(\log n)$ per query.

Pitfalls: Polygon must be CCW and no duplicate last point; `strict` excludes boundary.
Assumptions/deps: Depends on `geometry_base.cpp`.

```cpp
// Point in convex polygon in O(log n). Depends on geometry base.
// Source: KACTL PointInsideHull style.
// Complexity: O(log n) per query after CCW convex hull is built.
// Pitfalls: polygon must be CCW and have no duplicate last point; strict excludes boundary.
// Usage: bool ok = inConvex(poly, q, false);
bool inConvex(const vector<P>& p, P q, bool strict = false) {
    int n = (int)p.size();
    if (n == 0) return false;
    if (n == 1) return !strict && p[0] == q;
    if (n == 2) return !strict && onSegment(p[0], p[1], q);
    int a = ori(p[0], p[1], q), b = ori(p[0], p[n - 1], q);
    if (a < 0 || b > 0) return false;
    if (strict && (a == 0 || b == 0)) return false;
    int l = 1, r = n - 1;
    while (r - l > 1) {
        int m = (l + r) / 2;
        if (ori(p[0], p[m], q) >= 0) l = m;
        else r = m;
    }
    int o = ori(p[l], p[(l + 1) % n], q);
    return strict ? o > 0 : o >= 0;
}
```

6.5 Closest Pair

Use: Minimum squared distance among points.
Complexity: $O(n \log n)$.
Pitfalls: Returns squared distance; needs at least 2 points; duplicates return 0.
Assumptions/deps: Depends on `geometry_base.cpp`.

```cpp
// Closest pair of points. Depends on geometry base.
// Source: standard divide-and-conquer closest pair.
// Complexity: O(n log n).
// Pitfalls: returns squared distance; duplicate points give 0; needs n >= 2.
// Usage: auto [d2, pr] = closestPair(points);
pair<T, pair<P, P>> closestPair(vector<P> p) {
    assert((int)p.size() >= 2);
    sort(p.begin(), p.end(), [](P a, P b) {
        if (a.x != b.x) return a.x < b.x;
        return a.y < b.y;
    });
    vector<P> tmp(p.size());
    pair<T, pair<P, P>> ans = {norm2(p[0] - p[1]), {p[0], p[1]}};

    auto upd = [&](P a, P b) {
        T d = norm2(a - b);
        if (d < ans.first) ans = {d, {a, b}};
    };

    function<void(int, int)> solve = [&](int l, int r) {
        if (r - l <= 3) {
            for (int i = l; i < r; i++)
                for (int j = i + 1; j < r; j++) upd(p[i], p[j]);
            sort(p.begin() + l, p.begin() + r, [](P a, P b) { return a.y < b.y; });
            return;
        }
        int m = (l + r) / 2;
        T mx = p[m].x;
        solve(l, m);
        solve(m, r);
        merge(p.begin() + l, p.begin() + m, p.begin() + m, p.begin() + r,
              tmp.begin(), [](P a, P b) { return a.y < b.y; });
        copy(tmp.begin(), tmp.begin() + (r - l), p.begin() + l);

        int sz = 0;
        T d = sqrtl(ans.first);
        for (int i = l; i < r; i++) if (fabsl(p[i].x - mx) <= d + EPS) {
            for (int j = sz - 1; j >= 0 && p[i].y - tmp[j].y <= d + EPS; j--)
                upd(p[i], tmp[j]);
            tmp[sz++] = p[i];
        }
    };
    solve(0, (int)p.size());
    return ans;
}
```

6.6 Circle Tangents

Use: External/internal tangent points between two circles or point-circle.
Complexity: $O(1)$.
Pitfalls: Use `r2=0` for point tangents; negate radius for internal tangents. Same center returns empty.
Assumptions/deps: Depends on `geometry_base.cpp`.

```cpp
// Tangent points between two circles. Depends on geometry base.
// Source: KACTL CircleTangents style.
// Complexity: O(1).
// Pitfalls: use r2 = 0 for tangents from point c2 to circle c1; negate r2 for internal tangents.
// Usage: auto t = circleTangents(c, r, p, 0); // pairs {on circle, p}
vector<pair<P, P>> circleTangents(P c1, T r1, P c2, T r2) {
```

```
    P d = c2 - c1;
    T dr = r1 - r2, d2 = norm2(d), h2 = d2 - dr * dr;
    if (sgn(d2) == 0 || sgn(h2) < 0) return {};
    vector<pair<P, P>> out;
    for (T sign : {-1.0L, 1.0L}) {
        P v = (d * dr + perp(d) * sqrtl(max((T)0, h2)) * sign) / d2;
        out.push_back({c1 + v * r1, c2 + v * r2});
    }
    if (sgn(h2) == 0) out.pop_back();
    return out;
}
```

6.7 Minimum Enclosing Circle

Use: Smallest circle covering all points.
Complexity: Expected $O(n)$.
Pitfalls: Randomized; circumcenter fails for collinear triples but algorithm should call only when needed. EPS matters.
Assumptions/deps: Depends on `geometry_base.cpp`; uses `shuffle`, `mt19937`.

```
// Minimum enclosing circle for points in the plane.
// Source: KACTL MinEnclosingCircle, adapted to the shared geometry P.
// Complexity: expected O(n).
// Pitfalls: randomized order; copy geometry base first; use EPS when checking
    containment.
// Usage: auto [c, r] = minimum_enclosing_circle(points);
P circumcenter(P a, P b, P c) {
    b = b - a, c = c - a;
    T d = 2 * cross(b, c);
    return a + P{
        (c.y * (b.x * b.x + b.y * b.y) - b.y * (c.x * c.x + c.y * c.y)) / d,
        (b.x * (c.x * c.x + c.y * c.y) - c.x * (b.x * b.x + b.y * b.y)) / d
    };
}

pair<P, T> minimum_enclosing_circle(vector<P> p) {
    if (p.empty()) return {{0, 0}, 0};
    shuffle(p.begin(), p.end(), mt19937(chrono::steady_clock::now().time_since_epoch
        ().count()));
    P c = p[0];
    T r = 0, mult = 1 + 1e-10;
    for (int i = 0; i < (int)p.size(); i++) if (norm(c - p[i]) > r * mult) {
        c = p[i], r = 0;
        for (int j = 0; j < i; j++) if (norm(c - p[j]) > r * mult) {
            c = (p[i] + p[j]) / 2;
            r = norm(c - p[i]);
            for (int k = 0; k < j; k++) if (norm(c - p[k]) > r * mult) {
                c = circumcenter(p[i], p[j], p[k]);
                r = norm(c - p[i]);
            }
        }
    }
    return {c, r};
}
```

6.8 Half-Plane Intersection

Use: Intersect directed left half-planes into a bounded polygon.
Complexity: $O(n \log n)$.
Pitfalls: Returns empty for unbounded or empty result; add a bounding box when required; EPS matters.
Assumptions/deps: Depends on Geometry Base; zero direction invalid.

```
// Intersection of left half-planes p + t*v. Depends on geometry_base.cpp.
// Returns the bounded convex polygon in CCW order; unbounded/empty -> {}.
// Complexity: O(n log n) time, O(n) memory.
// Pitfalls: add a large bounding box if the answer may be unbounded; zero
// direction vectors are invalid; EPS controls parallel/outside tests.
// Usage: halfPlaneIntersection({{{0,0},{1,0}}, ...});
struct HalfPlane {
    P p,v;
    bool outside(P q) const { return sgn(cross(v,q-p))<0; }
};
bool hpSameDirection(const HalfPlane& a,const HalfPlane& b) {
    return sgn(cross(a.v,b.v))==0 && sgn(dot(a.v,b.v))>0;
}
P hpIntersection(const HalfPlane& a,const HalfPlane& b) {
    return a.p+a.v*(cross(b.p-a.p,b.v)/cross(a.v,b.v));
}
int hpHalf(P v) { return v.y>0 || (v.y==0 && v.x>=0) ? 0 : 1; }

vector<P> halfPlaneIntersection(vector<HalfPlane> h) {
    sort(h.begin(),h.end(),[](const HalfPlane& a,const HalfPlane& b) {
        int ha=hpHalf(a.v),hb=hpHalf(b.v);
        if(ha!=hb) return ha<hb;
        return cross(a.v,b.v)>0;
    });
    vector<HalfPlane> unique;
    for(auto line:h) {
        if(!unique.empty() && hpSameDirection(unique.back(),line)) {
            if(line.outside(unique.back().p)) unique.back()=line;
        } else unique.push_back(line);
    }
    deque<HalfPlane> q;
    for(auto line:unique) {
```

```
        while(q.size()>1 && line.outside(hpIntersection(q[q.size()-2],q.back()))) q.pop_back();
        while(q.size()>1 && line.outside(hpIntersection(q[0],q[1]))) q.pop_front();
        q.push_back(line);
    }
    while(q.size()>2 && q.front().outside(hpIntersection(q[q.size()-2],q.back()))) q.pop_back();
    while(q.size()>2 && q.back().outside(hpIntersection(q[0],q[1]))) q.pop_front();
    if(q.size()<3) return {};
    vector<P> poly;
    for(int i=0;i<(int)q.size();++i) {
        if(sgn(cross(q[i].v,q[(i+1)%q.size()].v))==0) return {};
        poly.push_back(hpIntersection(q[i],q[(i+1)%q.size()]));
    }
    return poly;
}
```

6.9 Minkowski Sum

Use: Sum two strictly convex integer polygons.
Complexity: $O(n+m)$ after normalization.
Pitfalls: Remove collinear hull points first; coordinate sums must fit `long long`.
Assumptions/deps: Depends on Integer Geometry; accepts CW/rotated input.

```
// Minkowski sum of two nonempty strictly-convex integer polygons.
// Input polygons may be CW/rotated; output is CCW, without repeated endpoint.
// Complexity: O(n+m) after normalization, O(n+m) memory.
// Pitfalls: remove collinear hull points first; products use i128, coordinates
// of the resulting sums must still fit long long. Depends on IP/i128.
// Usage: vector<IP> c=minkowskiSum(a,b);
i128 mpCross(IP a,IP b) { return (i128)a.x*b.y-(i128)a.y*b.x; }
i128 mpDot(IP a,IP b) { return (i128)a.x*b.x+(i128)a.y*b.y; }
IP mpAdd(IP a,IP b) { return {a.x+b.x,a.y+b.y}; }
IP mpSub(IP a,IP b) { return {a.x-b.x,a.y-b.y}; }
int mpHalf(IP a) { return a.y>0 || (a.y==0 && a.x>=0) ? 0 : 1; }
bool mpAngleLess(IP a,IP b) {
    if(mpHalf(a)!=mpHalf(b)) return mpHalf(a)<mpHalf(b);
    return mpCross(a,b)>0;
}

vector<IP> mpNormalize(vector<IP> p) {
    if(p.size()>1 && p.front().x==p.back().x && p.front().y==p.back().y) p.pop_back();
    i128 area=0;
    for(int i=0;i<(int)p.size();++i) area+=mpCross(p[i],p[(i+1)%p.size()]);
    if(area<0) reverse(p.begin(),p.end());
    int at=min_element(p.begin(),p.end(),[](IP a,IP b){return tie(a.y,a.x)<tie(b.y,b.x);})-p.begin();
    rotate(p.begin(),p.begin()+at,p.end());
    return p;
}
vector<IP> minkowskiSum(vector<IP> a,vector<IP> b) {
    a=mpNormalize(a); b=mpNormalize(b);
    if(a.size()==1) { for(auto& p:b)p=mpAdd(p,a[0]); return b; }
    if(b.size()==1) { for(auto& p:a)p=mpAdd(p,b[0]); return a; }
    vector<IP> ea(a.size()),eb(b.size());
    for(int i=0;i<(int)a.size();++i) ea[i]=mpSub(a[(i+1)%a.size()],a[i]);
    for(int i=0;i<(int)b.size();++i) eb[i]=mpSub(b[(i+1)%b.size()],b[i]);
    vector<IP> ans={mpAdd(a[0],b[0])};
    int i=0,j=0;
    while(i<(int)ea.size() || j<(int)eb.size()) {
        i128 cr=(i<(int)ea.size() && j<(int)eb.size())?mpCross(ea[i],eb[j]):0;
        IP step;
        if(i<(int)ea.size() && j<(int)eb.size() && cr==0 && mpDot(ea[i],eb[j])>0)
            step=mpAdd(ea[i++],eb[j++]);
        else if(j==(int)eb.size() || (i<(int)ea.size() && mpAngleLess(ea[i],eb[j])))
            step=ea[i++];
        else step=eb[j++];
        ans.push_back(mpAdd(ans.back(),step));
    }
    ans.pop_back();
    return ans;
}
```

6.10 Rectangle Union Area

Use: Sweep-line union area of axis-aligned rectangles.
Complexity: $O(n \log n)$.
Pitfalls: Half-open rectangles; degenerate rectangles ignored; answer is `__int128`.
Assumptions/deps: Internal compressed cover tree; equal-x events grouped.

```
// Sweep-line area of the union of axis-aligned half-open rectangles.
// Complexity: O(n log n) time, O(n) memory.
// Pitfalls: rectangles are [x1,x2)x[y1,y2); degenerate ones are ignored;
// answer is i128 because coordinate products may overflow long long.
// Usage: i128 area=rectangleUnionArea(rectangles);
struct Rectangle { long long x1,y1,x2,y2; };

__int128 rectangleUnionArea(const vector<Rectangle>& rectangles) {
    struct Event { long long x,y1,y2; int delta; };
    vector<Event> events;
    vector<long long> ys;
    for(auto r:rectangles) if(r.x1<r.x2 && r.y1<r.y2) {
        events.push_back({r.x1,r.y1,r.y2,1});
        events.push_back({r.x2,r.y1,r.y2,-1});
        ys.push_back(r.y1);ys.push_back(r.y2);
```

```
    }
    if(events.empty()) return 0;
    sort(events.begin(),events.end(),[](auto a,auto b){return a.x<b.x;});
    sort(ys.begin(),ys.end());ys.erase(unique(ys.begin(),ys.end()),ys.end());
    int m=ys.size()-1;
    vector<int> cover(4*max(1,m));
    vector<long long> length(4*max(1,m));
    function<void(int,int,int)> pull=[&](int x,int l,int r){
        if(cover[x]) length[x]=ys[r]-ys[l];
        else if(r-l==1) length[x]=0;
        else length[x]=length[2*x]+length[2*x+1];
    };
    function<void(int,int,int,int,int,int)> add=[&](int x,int l,int r,int ql,int qr,
         int d){
        if(qr<=l || r<=ql) return;
        if(ql<=l && r<=qr) cover[x]+=d;
        else {int mid=(l+r)/2;add(2*x,l,mid,ql,qr,d);add(2*x+1,mid,r,ql,qr,d);}
        pull(x,l,r);
    };
    __int128 area=0;
    long long prev=events[0].x;
    for(int i=0;i<(int)events.size();) {
        long long x=events[i].x;
        area+=(__int128)(x-prev)*length[1];
        while(i<(int)events.size() && events[i].x==x) {
            int l=lower_bound(ys.begin(),ys.end(),events[i].y1)-ys.begin();
            int r=lower_bound(ys.begin(),ys.end(),events[i].y2)-ys.begin();
            add(1,0,m,l,r,events[i].delta);++i;
        }
        prev=x;
    }
    return area;
}
```

6.11 Sweep-Line Event Ordering

Use: Covered-length invariant and tie-order checklist.
Complexity: $O(n \log n)$ for rectangle union.
Pitfalls: Event order changes boundary semantics; group equal coordinates where possible.
Assumptions/deps: Pairs with rectangle-union implementation above.

Maintain a data structure for the active cross-section while events are visited in coordinate order. Compute the contribution from x_{prev} to x using the **old** active state, then apply all events at x as one group.

Rectangle union

$$area \mathrel{+}= covered_y \, (x - x_{prev}).$$

- Start at x_1: add $[y_1, y_2)$.
- End at x_2: remove $[y_1, y_2)$.

Coordinate-compress all y endpoints. A segment-tree node stores a cover count:

- `cover > 0`: covered length is $ys_r - ys_l$.
- `cover == 0`: sum its children, or use zero at a leaf.

Grouping equal x avoids arbitrary start/end tie order. In sweeps with point queries, tie order is semantic: decide the open/closed boundaries before placing add, query, and remove events.

Skip zero-area rectangles, keep half-open intervals consistent, and use `__int128` if coordinate difference times covered length may overflow.

7 Dynamic Programming

7.1 Fast Knapsack and Subset Sum

Use: Bitset reachability, compressed bounded sums/min-items, bounded max-value.
Complexity: Bitset $O(nS/64)$; compressed $O(S\sqrt{S})$; bounded $O(types \cdot C)$.
Pitfalls: Weights must be nonnegative; zero weights/values need the documented handling.
Assumptions/deps: Codeforces square-root residue-window method; fixed-size bitset needs compile-time sum.

```
// Fast bounded knapsack and subset-sum variants.
// Complexity: bitset O(items*SUM/64); compressed DP O(S*D), D <= sqrt(2S).
// Pitfalls: all weights are nonnegative; zero weights need special handling.
// Usage: subset_sum_sqrt(a), subset_sum_min_items_sqrt(a), bounded_knapsack(C,it).
const int SUBSET_INF = 1e9;

template<size_t MAX_SUM>
bitset<MAX_SUM + 1> subset_sum_bitset(const vector<int>& a) {
    bitset<MAX_SUM + 1> can;
    can[0] = 1;
    for (int x : a) if (0 <= x && x <= (int)MAX_SUM) can |= can << x;
    return can;
}
```

```cpp

vector<pair<int, int>> compress_positive(const vector<int>& a, int& sum) {
    sum = 0;
    for (int x : a) if (x > 0) sum += x;
    vector<int> freq(sum + 1);
    for (int x : a) if (x > 0) ++freq[x];
    vector<pair<int, int>> items;
    for (int w = 1; w <= sum; ++w) if (freq[w]) items.push_back({w, freq[w]});
    return items;
}

vector<char> subset_sum_sqrt(const vector<int>& a) {
    int S;
    auto items = compress_positive(a, S);
    vector<char> dp(S + 1), ndp(S + 1);
    dp[0] = 1;
    for (auto [w, cnt] : items) {
        fill(ndp.begin(), ndp.end(), 0);
        for (int r = 0; r < w && r <= S; ++r) {
            int window = 0;
            for (int s = r, t = 0; s <= S; s += w, ++t) {
                window += dp[s];
                if (t > cnt) window -= dp[s - (cnt + 1) * w];
                ndp[s] = window > 0;
            }
        }
        dp.swap(ndp);
    }
    return dp;
}

vector<int> subset_sum_min_items_sqrt(const vector<int>& a) {
    int S;
    auto items = compress_positive(a, S);
    vector<int> dp(S + 1, SUBSET_INF), ndp(S + 1);
    dp[0] = 0;
    for (auto [w, cnt] : items) {
        fill(ndp.begin(), ndp.end(), SUBSET_INF);
        for (int r = 0; r < w && r <= S; ++r) {
            deque<pair<int, int>> q; // (dp[s]-t, t)
            for (int s = r, t = 0; s <= S; s += w, ++t) {
                while (!q.empty() && q.front().second < t - cnt) q.pop_front();
                if (dp[s] < SUBSET_INF) {
                    int val = dp[s] - t;
                    while (!q.empty() && q.back().first >= val) q.pop_back();
                    q.push_back({val, t});
                }
                if (!q.empty()) ndp[s] = q.front().first + t;
            }
        }
        dp.swap(ndp);
    }
    return dp;
}

struct BoundedItem { int weight, count; long long value; };

long long bounded_knapsack(int cap, const vector<BoundedItem>& items) {
    vector<long long> dp(cap + 1), ndp(cap + 1);
    for (auto [w, cnt, val] : items) {
        if (w == 0) {
            if (val > 0) for (long long& x : dp) x += val * cnt;
            continue;
        }
        ndp = dp;
        for (int r = 0; r < w && r <= cap; ++r) {
            deque<pair<long long, int>> q;
            for (int c = r, t = 0; c <= cap; c += w, ++t) {
                while (!q.empty() && q.front().second < t - cnt) q.pop_front();
                long long cur = dp[c] - 1LL * t * val;
                while (!q.empty() && q.back().first <= cur) q.pop_back();
                q.push_back({cur, t});
                ndp[c] = q.front().first + 1LL * t * val;
            }
        }
        dp.swap(ndp);
    }
    return dp[cap];
}
```

7.2 Fast Knapsack Formulas

Use: Residue-class sliding-window recurrences.
Complexity: As above.
Pitfalls: Compress equal positive values; deque direction differs for min-items and max-value.
Assumptions/deps: Read before adapting the code.

Bitset reachability

The 0/1 transition is `can |= can << weight`, taking $O(items \cdot SUM/wordsize)$.

Few distinct positive weights

If positive weights total S and there are D distinct values, then

$$1 + 2 + \cdots + D \le S, \qquad D = O(\sqrt{S}).$$

Compress equal weights into (w, k). Within each residue modulo w, a window of length $k + 1$ processes one weight in $O(S)$, hence $O(S\sqrt{S})$ total.

Minimum item count for sum $r + tw$:

$$new_t = t + \min_{\max(0,t-k)\le j\le t} (old_j - j).$$

For k copies of an item with weight w and value v:

$$new_t = tv + \max_{\max(0,t-k)\le j\le t} (old_j - jv).$$

Maintain the min/max with a deque. Bounded max-value knapsack is $O(types \cdot capacity)$. Zero subset-sum weights do not help; zero-weight positive-value items contribute every available copy.

7.3 Longest Increasing Subsequence

Use: Reconstruct one strict LIS as original indices.
Complexity: $O(n \log n)$.
Pitfalls: `lower_bound` is strict; use `upper_bound` for nondecreasing.
Assumptions/deps: 0-indexed; empty input returns empty.

```cpp
// Reconstruct one strict longest increasing subsequence (indices).
// Complexity: O(n log n).
// Pitfalls: lower_bound gives strict LIS; use upper_bound for nondecreasing LIS.
// Usage: vector<int> ids = lis_indices(a);
vector<int> lis_indices(const vector<long long>& a) {
    vector<long long> tail;
    vector<int> at, parent(a.size(), -1);
    for (int i = 0; i < (int)a.size(); ++i) {
        int p = lower_bound(tail.begin(), tail.end(), a[i]) - tail.begin();
        if (p == (int)tail.size()) tail.push_back(a[i]), at.push_back(i);
        else tail[p] = a[i], at[p] = i;
        if (p) parent[i] = at[p - 1];
    }
    vector<int> ans;
    if (!at.empty()) for (int v = at.back(); v != -1; v = parent[v]) ans.push_back(v
        );
    reverse(ans.begin(), ans.end());
    return ans;
}
```

7.4 Meet in the Middle

Use: Maximum subset sum not exceeding a limit.
Complexity: $O(2^{n/2} \log 2^{n/2})$.
Pitfalls: Returns `LLONG_MIN` if no subset fits; additions can overflow.
Assumptions/deps: Useful around $n \approx 40$–44.

```cpp
// Maximum subset sum <= limit by meet in the middle.
// Complexity: O(2^(n/2) log 2^(n/2)); practical for n about 40-44.
// Pitfalls: returns LLONG_MIN if no subset fits (possible when limit < 0).
// Usage: long long best=max_subset_sum_leq(a,limit).
vector<long long> half_sums(const vector<long long>& a, int l, int r) {
    vector<long long> sums(1, 0);
    for (int i = l; i < r; ++i) {
        int old = sums.size();
        sums.resize(2 * old);
        for (int j = 0; j < old; ++j) sums[old + j] = sums[j] + a[i];
    }
    return sums;
}

long long max_subset_sum_leq(const vector<long long>& a, long long limit) {
    int m = a.size() / 2;
    auto left = half_sums(a, 0, m), right = half_sums(a, m, a.size());
    sort(right.begin(), right.end());
    long long ans = LLONG_MIN;
    for (long long x : left) {
        auto it = upper_bound(right.begin(), right.end(), limit - x);
        if (it != right.begin()) ans = max(ans, x + *prev(it));
    }
    return ans;
}
```

7.5 Bitmask DP

Use: Submask enumeration and Held–Karp Hamiltonian path.
Complexity: Submasks $O(2^{popcount})$; path $O(n^2 2^n)$.
Pitfalls: Loop includes zero; memory is often limiting; example starts at vertex 0.
Assumptions/deps: Usually $n \le 22$; costs use `long long`.

```cpp
// Bitmask DP helpers: submask enumeration and Held-Karp Hamiltonian path.
// Complexity: all submasks of one mask O(2^popcount); Held-Karp O(n^2 2^n).
// Pitfalls: the submask loop includes 0; n must be small (usually <= 20-22);
// this path starts at 0, visits every vertex, and may end anywhere.
// Usage: for(int s=m;;s=(s-1)&m){... if(!s)break;} tsp_path(cost);
vector<int> submasks(int mask) {
    vector<int> out;
    for(int s=mask;;s=(s-1)&mask) {
        out.push_back(s);
        if(!s) break;
    }
    return out;
```

```cpp
}

long long tsp_path(const vector<vector<long long>>& cost) {
    int n=cost.size();
    if(!n) return 0;
    const long long INF=(1LL<<62);
    vector<vector<long long>> dp(1<<n,vector<long long>(n,INF));
    dp[1][0]=0;
    for(int mask=1;mask<(1<<n);++mask) for(int v=0;v<n;++v) if(mask>>v&1) {
        if(dp[mask][v]==INF) continue;
        for(int u=0;u<n;++u) if(!(mask>>u&1))
            dp[mask|1<<u][u]=min(dp[mask|1<<u][u],dp[mask][v]+cost[v][u]);
    }
    return *min_element(dp.back().begin(),dp.back().end());
}
```

7.6 Bitmask DP Formulas

Use: Submask $O(3^n)$ proof and Held–Karp recurrence.
Complexity: Reference notes.
Pitfalls: Confirm path versus cycle and mask integer width.
Assumptions/deps: Pairs with the example implementation above.

Bit i belongs to subset S when `(S >> i) & 1`. Add, remove, or toggle it with `|`, `&`, or `^` using `1 << i`.

Enumerate all submasks

```cpp
for (int T = S; ; T = (T - 1) & S) {
    use(T);
    if (T == 0) break;
}
```

Across every S, this is $O(3^n)$: each bit is outside S, in $S \setminus T$, or in T. For nonempty proper submasks, start at `(S - 1) & S` and stop before zero.

Held-Karp path DP

$dp[S][v]$ is the minimum cost of a path starting at 0, visiting exactly S, and ending at v:

$$dp[S \cup \{u\}][u] \leftarrow \min\big(dp[S \cup \{u\}][u],\ dp[S][v] + cost[v][u]\big).$$

Pitfalls: `1 << n` overflows `int` near $n = 31$; use `1LL << n` when needed. Memory $O(n2^n)$ is often the real limit. Decide path vs. cycle and whether the start is fixed.

7.7 Digit DP

Use: Count values up to a bound with a target digit sum.
Complexity: $O(digits^2 \cdot 10)$.
Pitfalls: Leading zeros represent shorter values; zero is counted for target zero; reset memo per bound.
Assumptions/deps: Canonical bound function for range subtraction.

```cpp
// Digit DP example: count x in [0,limit] with digit sum exactly target.
// Complexity: O(digits^2 * 10) time, O(digits^2) memory per call.
// Pitfalls: leading zeroes represent shorter numbers; 0 is counted when
// target=0. This version assumes -1 <= limit <= 1e18.
// Usage: count_digit_sum(r,s)-count_digit_sum(l-1,s);
long long count_digit_sum(long long limit,int target) {
    if(limit<0 || target<0 || target>171) return 0;
    string digits=to_string(limit);
    long long memo[20][172];
    memset(memo,-1,sizeof memo);
    function<long long(int,int,bool)> dfs=[&](int pos,int sum,bool tight)->long long
        {
        if(sum>target) return 0;
        if(pos==(int)digits.size()) return sum==target;
        if(!tight && memo[pos][sum]!=-1) return memo[pos][sum];
        int hi=tight?digits[pos]-'0':9;
        long long ans=0;
        for(int d=0;d<=hi;++d) ans+=dfs(pos+1,sum+d,tight && d==hi);
        if(!tight) memo[pos][sum]=ans;
        return ans;
    };
    return dfs(0,0,true);
}
```

7.8 Digit DP State Design

Use: Tight/started flags and common property states.
Complexity: State-dependent.
Pitfalls: Decide zero semantics and answer width before coding.
Assumptions/deps: Use the code above as a small concrete example.

Canonical state

Scan most-significant digit first with `(position, problem_state, tight, started)`.

- `tight`: the prefix still equals the bound, so the next digit is bounded.
- `started`: separates leading zeroes from real zero digits when length, adjacency, or the number zero matters.

For $[L, R]$, compute $solve(R) - solve(L-1)$. Memoize only states with `tight == false`; tight states depend on the bound prefix. Omit `started` when leading zeroes cannot change the property, as with digit sum.

Common problem states

- digit sum or sum modulo k;
- previous digit for adjacency constraints;
- automaton node for forbidden/required substrings;
- used-digit mask;
- remainder modulo m.

Decide whether zero is included, reset memo between bounds/test cases, and estimate answer overflow before choosing `long long`.

7.9 Slope Trick

Use: Maintain convex piecewise-linear DP functions with two heaps.
Complexity: $O(\log n)$ per hinge; $O(1)$ shift/minimum.
Pitfalls: Convex functions only; offsets and function values can overflow.
Assumptions/deps: Supports constants, one-sided hinges, absolute value, and sliding minimum.

```
// Slope trick for convex piecewise-linear functions with integer breakpoints.
// Supports adding constants, max(a-x,0), max(x-a,0), |x-a|, and sliding min.
// Complexity: O(log n) per hinge, O(1) shift/minimum; memory O(operations).
// Pitfalls: values/offsets can overflow long long; shift(a,b) requires a<=b.
// Usage: SlopeTrick f; f.addAbs(a); f.shift(left,right); f.minimum().
struct SlopeTrick {
    using ll=long long;
    const ll INF=(1LL<<62);
    priority_queue<ll>L;
    priority_queue<ll,vector<ll>,greater<ll>>R;
    ll addL=0,addR=0,minF=0;

    ll topL() const{return L.top()+addL;}
    ll topR() const{return R.top()+addR;}
    void pushL(ll x){L.push(x-addL);}
    void pushR(ll x){R.push(x-addR);}
    void addConstant(ll c){minF+=c;}
    void addAMinusX(ll a) { // max(a-x,0)
        if(R.empty()){pushL(a);return;}
        ll r=topR();if(a>r)minF+=a-r;
        pushR(a);r=topR();R.pop();pushL(r);
    }
    void addXMinusA(ll a) { // max(x-a,0)
        if(L.empty()){pushR(a);return;}
        ll l=topL();if(l>a)minF+=l-a;
        pushL(a);l=topL();L.pop();pushR(l);
    }
    void addAbs(ll a){addAMinusX(a);addXMinusA(a);}
    // f_new(x)=min_{x-b <= y <= x-a} f(y)
    void shift(ll a,ll b){assert(a<=b);addL+=a;addR+=b;}
    ll minimum() const{return minF;}
    pair<ll,ll> argmin() const{return {L.empty()?-INF:topL(),R.empty()?INF:topR()};}
    ll value(ll x) const { // O(number of breakpoints), mainly for debugging
        ll ans=minF;auto l=L;auto r=R;
        while(!l.empty()){ans+=max(0LL,l.top()+addL-x);l.pop();}
        while(!r.empty()){ans+=max(0LL,x-(r.top()+addR));r.pop();}
        return ans;
    }
};
```

7.10 Slope Trick Formulas

Use: Breakpoint representation and sliding-min transform.
Complexity: As above.
Pitfalls: Derive the DP transition as supported convex operations first.
Assumptions/deps: Pairs with the two-heap implementation.

Represent a convex piecewise-linear f by its minimum and two heaps of breakpoints. The left max-heap contributes $\max(l - x, 0)$; the right min-heap contributes $\max(x - r, 0)$.

- Add $\max(a - x, 0)$: insert a on the left, rebalancing through the right.
- Add $\max(x - a, 0)$: insert a on the right, rebalancing through the left.
- Add $|x - a|$: perform both operations.

The minimum is stored directly. Its minimizer interval is $[\max(left), \min(right)]$.

Sliding-window minimum

For $a \le b$,

$$g(x) = \min_{x-b \le y \le x-a} f(y).$$

Shift every left breakpoint by a and every right breakpoint by b; lazy offsets avoid touching the heaps.

This structure represents **convex functions only**. Derive the DP transition as supported convex operations before copying code.

7.11 Lagrangian Relaxation (Aliens/WQS)

Use: Replace an exact-count dimension by a per-component penalty.
Complexity: $O(\log range \cdot oracle)$.
Pitfalls: Monotone count alone is insufficient; prove discrete convexity/strong duality and tie rules.
Assumptions/deps: Generic exact-k minimum wrapper around a pure penalized oracle.

```cpp
// Lagrangian relaxation (WQS/Aliens trick) for an exact-k penalized oracle.
// solve(lambda) -> {min(base_cost + lambda*count), chosen_count}.
// Complexity: O(log(hi-lo) * oracle cost).
// Pitfalls: count must be nonincreasing in lambda and the exact-count cost must
// have the required discrete-convex/strong-duality property.
// Usage: answer=lagrangian_exact_k(k,lo,hi,solve);
template<class Solve>
long long lagrangian_exact_k(int k,long long lo,long long hi,Solve solve) {
    long long originalLo=lo;
    assert(solve(lo).second>=k && solve(hi).second<=k);
    while(lo<hi) {
        long long mid=lo+(hi-lo)/2;
        if(solve(mid).second<=k)hi=mid;
        else lo=mid+1;
    }
    auto adjusted=[&](long long lambda) {
        auto [penalized,count]=solve(lambda);(void)count;
        return penalized-lambda*k;
    };
    long long ans=adjusted(lo);
    if(lo>originalLo)ans=max(ans,adjusted(lo-1));
    return ans;
}
template<class Solve>
long long aliens_exact_k(int k,long long lo,long long hi,Solve solve) {
    return lagrangian_exact_k(k,lo,hi,solve);
}
```

7.12 Lagrangian Relaxation Proof

Use: Penalty monotonicity, dual recovery, and oracle checklist.
Complexity: As above.
Pitfalls: Penalty bounds must straddle k; cost arithmetic can overflow.
Assumptions/deps: Also known as Aliens trick or WQS binary search.

Let $C(k)$ be the optimum base cost using exactly k components. Add penalty λ per component and ask an oracle for

$$P(\lambda) = \min_t \big(C(t) + \lambda t\big).$$

As λ increases, the chosen component count is nonincreasing. For optima a at λ_1 and b at $\lambda_2 > \lambda_1$, add the two optimality inequalities to get

$$(\lambda_2 - \lambda_1)(b - a) \le 0 \quad \Longrightarrow \quad b \le a.$$

Binary-search where the oracle count crosses k. Under the required discrete-convex/strong-duality condition,

$$C(k) = \max_\lambda \big(P(\lambda) - \lambda k\big).$$

Oracle checklist

- Return both penalized cost and chosen count.
- Break equal-cost ties consistently to preserve monotonicity.
- Prove penalty bounds whose endpoint counts straddle k.
- Monotone count is not enough: justify exact-k dual recovery.

Also called Aliens trick or WQS binary search.

7.13 Knuth Optimization

Use: Interval DP with monotone split bounds.
Complexity: $O(n^2)$ after prefix/cost setup.
Pitfalls: Only valid when Knuth quadrangle/monotone conditions hold. Snippet is 1-indexed and uses `#define int long long`.
Assumptions/deps: Standalone example with `main`; globals sized by `N`.

```cpp
// Knuth optimization: dp[i][j] = min_{i<=k<j}(dp[i][k]+dp[k+1][j]) + C(i,j).
// Valid when opt[i][j-1] <= opt[i][j] <= opt[i+1][j], e.g. C satisfies the
// quadrangle inequality and is monotone on inclusion. O(n^2) total.
// 1-indexed. Example: merge-stones cost C(i,j) = sum a[i..j].
#include <bits/stdc++.h>
using namespace std;
#define int long long
const int N = 5e3 + 5;
int a[N], pref[N];
int dp[N][N], opt[N][N];
// EDIT ONLY C(): the interval cost. The loops below are the optimization.
int C(int l, int r) { return pref[r] - pref[l - 1]; }
int32_t main() {
    ios::sync_with_stdio(0); cin.tie(NULL);
    int n; cin >> n;
```

```
    for (int i = 1; i <= n; i++) {
        cin >> a[i];
        pref[i] = pref[i - 1] + a[i];
    }
    for (int i = 1; i <= n; i++) opt[i][i] = i;
    for (int len = 2; len <= n; len++) {
        for (int i = 1; i + len - 1 <= n; i++) {
            int j = i + len - 1;
            pair<int, int> best = {(int)1e18, -1};
            for (int k = max(i, opt[i][j - 1]); k <= min(opt[i + 1][j], j - 1); k++)
                best = min(best, make_pair(dp[i][k] + dp[k + 1][j], k));
            dp[i][j] = best.first + C(i, j);
            opt[i][j] = best.second;
        }
    }
    cout << dp[1][n] << "\n";
}
```

7.14 Divide and Conquer DP

Use: Optimize DP with monotone optimal split.
Complexity: Typically $O(mn \log n)$ here; tighter variants can be $O(mn)$.
Pitfalls: Do not use unless monotonicity of `opt` is proven/tested. Cost `C(k,j)` must be fast.
Assumptions/deps: Implement `C`; recurrence assumes previous row and 0-indexed intervals.

```
// Divide & conquer DP optimization: dp_cur[j] = min_k(dp_bef[k-1] + cost(k,j)).
// Valid when opt(j) is non-decreasing in j (e.g. cost satisfies quadrangle
// inequality). O(m n log n) instead of O(m n^2). 1-indexed.
// Example: split a[1..n] into m groups, cost of a group = (sum)^2.
// WARNING: (sum)^2 cost needs a[i] >= 0 (monotone prefix sums), else QI fails.
#include <bits/stdc++.h>
using namespace std;
#define int long long
const int N = 3e5 + 5;
int n, m;
vector<int> dp_bef, dp_cur;
int a[N], pref[N];
// EDIT ONLY cost(): comp() below is the optimization machinery.
int cost(int l, int r) {
    int sum = pref[r] - pref[l - 1];
    return sum * sum;
}
void comp(int l, int r, int lo, int hi) { // fill dp_cur[l..r]; opt is in [lo,hi]
    if (l > r) return;
    int mid = (l + r) / 2;
    pair<int, int> best = {(int)1e18, -1};
    for (int k = lo; k <= min(hi, mid); k++)
        best = min(best, make_pair((k ? dp_bef[k - 1] : 0) + cost(k, mid), k));
    dp_cur[mid] = best.first;
    int opt = best.second;
    comp(l, mid - 1, lo, opt);
    comp(mid + 1, r, opt, hi);
}
int32_t main() {
    ios::sync_with_stdio(0); cin.tie(NULL);
    cin >> n >> m;
    for (int i = 1; i <= n; i++) {
        cin >> a[i];
        pref[i] = pref[i - 1] + a[i];
    }
    dp_cur.assign(n + 1, 0);
    for (int j = 1; j <= n; j++) dp_cur[j] = cost(1, j);
    dp_bef = dp_cur;
    for (int i = 2; i <= m; i++) {
        comp(1, n, 1, n);
        dp_bef = dp_cur;
    }
    cout << dp_cur[n] << '\n';
}
```

7.15 Divide and Conquer DP Notes

Use: Proof checklist for monotone DP optimization.
Complexity: Reference notes.
Pitfalls: The note is a guide, not a proof for a specific problem.
Assumptions/deps: Read before using the template.

Recurrence and requirement

For one DP layer,

$$dp_i(j) = \min_{0 \le k \le j} \left(dp_{i-1}(k-1) + C(k,j) \right).$$

Let $opt_i(j)$ be a minimizing k. The optimization is valid only after proving

$$opt_i(j) \le opt_i(j+1).$$

A common sufficient condition is the quadrangle inequality

$$C(a,c) + C(b,d) \le C(a,d) + C(b,c) \quad (a \le b \le c \le d),$$

but this must be checked for the actual cost function.

Divide-and-conquer step

Compute the midpoint mid while searching only $[opt_L, \min(mid, opt_R)]$. If its optimum is $best$, recurse on:

- the left half with bounds $[opt_L, best]$;
- the right half with bounds $[best, opt_R]$.

With $O(1)$ cost evaluation, one layer is $O(n \log n)$ and m layers are $O(mn \log n)$, versus $O(mn^2)$ naively.

Usage and pitfalls

The printed implementation uses 1-indexed intervals and calls `comp(1, n, 1, n)` once per layer.

- Define impossible previous states as $+\infty$; only the empty prefix has cost 0.
- Use a wide type for both C and `dp + C`.
- Tie-breaking must preserve a valid monotone optimum.
- Monotonicity is the proof obligation, not a property of the template.

8 Misc

8.1 Parallel Binary Search

Use: Find each query's first true event prefix by batched replays.
Complexity: $O(\log E)$ complete event sweeps.
Pitfalls: Predicate must be prefix-monotone; reset must fully clear replay state.
Assumptions/deps: Callbacks for reset/apply/check; returns $E + 1$ when never true.

```cpp
// Parallel binary search for predicates monotone over an event prefix.
// Returns smallest prefix length t in [0,events], or events+1 if never true.
// Complexity: O(log(events+1)) full replays; memory O(q+events).
// Pitfalls: apply events in fixed order; reset must fully clear state; check(q)
// must stay true after becoming true.
// Usage: ans=parallel_bs(q,event_count,reset,apply_event,check_query);
template<class Reset,class Apply,class Check>
vector<int> parallel_bs(int q,int events,Reset reset,Apply apply,Check check) {
    vector<int>lo(q,-1),hi(q,events+1);
    while(true) {
        bool active=false;
        vector<vector<int>>bucket(events+1);
        for(int id=0;id<q;++id) if(lo[id]+1<hi[id]) {
            int mid=(lo[id]+hi[id])/2;
            if(mid<=events) bucket[mid].push_back(id);
            active=true;
        }
        if(!active) return hi;
        reset();
        for(int t=0;t<=events;++t) {
            for(int id:bucket[t]) {
                if(check(id)) hi[id]=t;
                else lo[id]=t;
            }
            if(t<events) apply(t);
        }
    }
}
```

8.2 Quick Select

Use: Get kth element / partition order statistic.
Complexity: Average/guaranteed linear for `nth_element`.
Pitfalls: After call, only kth is fixed; sides are partitioned, not sorted.
Assumptions/deps: `k` is 0-indexed iterator offset.

```cpp
nth_element(a.begin(), a.begin()+k, a.end());
```

8.3 mt19937 RNG

Use: Random 32-bit generator, correct usage demo.
Complexity: $O(1)$ per value.
Pitfalls: `int x = rng()` can be negative – use `unsigned`. Never `rand()`/`random_shuffle`.
Assumptions/deps: Shows `uniform_int_distribution` and `shuffle`.

```cpp
#include <bits/stdc++.h>
using namespace std;

mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());

int main() {
    unsigned x = rng(); // random 32-bit (int x = rng() can be negative!)
    int y = rng() % 100; // 0..99, slight modulo bias (fine for CP)
    int z = uniform_int_distribution<int>(5, 10)(rng); // exact uniform in [5,10]
    long long big = ((long long)rng() << 31) ^ rng(); // or use mt19937_64
    vector<int> v = {1, 2, 3};
    shuffle(v.begin(), v.end(), rng); // NEVER random_shuffle (uses rand())
```

```
    cout << x << " " << y << " " << z << " " << big << "\n";
}
```

8.4 mt19937_64 RNG

Use: Random 64-bit generator.
Complexity: $O(1)$ per value.
Pitfalls: Seed once; do not reseed inside tight loops.
Assumptions/deps: Requires `chrono`, `mt19937_64`.

```
mt19937_64 rng64(chrono::steady_clock::now().time_since_epoch().count());
unsigned long long x = rng64();
```

8.5 Random Hashing

Use: Assign random xor keys to values.
Complexity: Expected $O(1)$ map operations.
Pitfalls: Hash is probabilistic. Requires stable rng and clearing map when state universe changes.
Assumptions/deps: Depends on `rng`; `uint64_t`.

```
// Set-equality hashing: hash of a set = xor of random keys of its elements.
// Requires: mt19937_64 rng (rand_mt19937_faster.cpp). Compare multisets with
// sums instead of xor (xor cancels duplicate pairs).
uint64_t rnd() {
    return rng();
}
unordered_map<int, uint64_t> h;
uint64_t getHash(int x) {
    if (!h.count(x)) h[x] = rnd();
    return h[x];
}
cur ^= getHash(x); // toggle x
```

8.6 Zobrist Hashing

Use: Incremental xor hash for states.
Complexity: Build $O(positions \cdot values)$, update/hash $O(1)$ or $O(n)$.
Pitfalls: Probabilistic collisions; state values must be valid indices.
Assumptions/deps: Uses `mt19937_64`; `uint64_t`.

```
// Zobrist hashing: xor random keys for incremental state hashes.
// Source: standard Zobrist/tabulation hashing.
// Complexity: build O(positions * values), update/query O(1).
// Pitfalls: probabilistic collisions; state values must be in range.
// Usage: h ^= key[pos][old]; h ^= key[pos][now];
struct Zobrist {
    vector<vector<uint64_t>> key;

    Zobrist(int positions, int values, uint64_t seed = chrono::steady_clock::now().
        time_since_epoch().count()) {
        mt19937_64 rng(seed);
        key.assign(positions, vector<uint64_t>(values));
        for (auto& row : key)
            for (uint64_t& x : row) x = rng();
    }

    uint64_t hash(const vector<int>& state) const {
        uint64_t h = 0;
        for (int i = 0; i < (int)state.size(); i++)
            if (state[i] >= 0) h ^= key[i][state[i]];
        return h;
    }
};
```

8.7 Custom Hash for unordered_map

Use: Defeat anti-hash tests against gcc's identity hash.
Complexity: $O(1)$ per op restored.
Pitfalls: Default integer hash is identity: chosen keys collide into one bucket ($O(n)$ per op).
Assumptions/deps: `unordered_map<ll,int,chash>`; random offset at runtime.

```
// Safe hash for unordered_map/set. libstdc++'s default hash for integers is
// the IDENTITY: a setter can pick keys colliding mod the bucket prime and
// degrade the map to O(n) per op (same anti-test class as mod-2^64 string
// hashing). splitmix64 + runtime-random offset defeats fixed test data.
// Usage: unordered_map<long long, int, chash> mp;
struct chash {
    static uint64_t splitmix64(uint64_t x) {
        x += 0x9e3779b97f4a7c15;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
        x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
        return x ^ (x >> 31);
    }
    size_t operator()(uint64_t x) const {
        static const uint64_t R =
            chrono::steady_clock::now().time_since_epoch().count();
        return splitmix64(x + R);
```

```
    }
};
```

8.8 Deep Recursion Stack

Use: Raise stack to 256MB for 1e6-deep DFS.
Complexity: $O(1)$ setup.
Pitfalls: Call FIRST in `main`; returns false if hard limit forbids (then go iterative).
Assumptions/deps: Linux `setrlimit`; covers all recursive snippets here.

```cpp
// Raise the stack limit for deep recursion (Linux). Call FIRST in main.
// Default stack is often 8MB: a DFS on a 1e6-vertex chain overflows it
// (each frame is ~50-200B, more with std::function). 256MB fixes all
// recursive snippets in this notebook (SCC, HLD, LCA, sack, tree DPs).
// Returns false if the hard limit forbids it (then convert to iterative).
#include <sys/resource.h>
bool raise_stack(rlim_t bytes = 256 << 20) {
    rlimit rl;
    getrlimit(RLIMIT_STACK, &rl);
    if (rl.rlim_max != RLIM_INFINITY && rl.rlim_max < bytes) return false;
    rl.rlim_cur = bytes;
    return setrlimit(RLIMIT_STACK, &rl) == 0;
}
```

8.9 GNU Extensions

Use: PBDS ordered sets/maps and rope snippets.
Complexity: $O(\log n)$ usual PBDS/rope ops.
Pitfalls: Examples are not one compilable file. Never use `less_equal` PBDS multiset; use the pair trick.
Assumptions/deps: GNU C++ only.

```cpp
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
// NOTE: examples below are not one compilable file; paste only the needed block.
using namespace __gnu_pbds;

template<typename T>
using ordered_set = tree<
    T,
    null_type,
    less<T>,
    rb_tree_tag,
    tree_order_statistics_node_update>;

template<typename K, typename V>
using ordered_map = tree<
    K,
    V,
    less<K>,
    rb_tree_tag,
    tree_order_statistics_node_update>;

ordered_set<int> s;

s.insert(x);
s.erase(x);

// number of elements strictly smaller than x
int k = s.order_of_key(x);

// iterator to k-th (0-indexed) element
auto it = s.find_by_order(k);

// multiset trick: store (value, unique id) pairs. Do NOT use less_equal<> as
// comparator instead: it silently breaks erase(key) and lower_bound.
using pii = pair<int,int>;

ordered_set<pii> ms;
int id = 0;

ms.insert({value, id++});

// erase one occurrence
ms.erase(ms.lower_bound({value, 0}));

// order_of_key({value, 0}) = # elements < value

// rope: O(log n) insert/erase of chunks (editor-style problems)
#include <ext/rope>
using namespace __gnu_cxx;

rope<char> r;

r.insert(pos, "abc");
r.erase(pos, len);
r.replace(pos, len, "xyz");

char c = r[i];
```

8.10 Simulated Annealing Skeleton

Use: Approximate optimization when exact method is hard.
Complexity: Heuristic.
Pitfalls: Risky in ICPC: tune step, time limit, random direction, and score. No correctness guarantee.
Assumptions/deps: Depends on geometry point `P` and user functions.

```
// Local search with shrinking step (hill climbing / pattern search).
// Good for: approximate geometric optimization (e.g. point minimizing max
// distance), unimodal-ish continuous functions. Tune step count to time limit.
// For true simulated annealing also accept worse moves with probability
// exp((score(cur)-score(nxt))/temp), temp decaying like step.
P cur = start;
double step = initial_step; // e.g. max coordinate range

while (step > 1e-9) {
    P nxt = cur + random_direction() * step;
    if (score(nxt) < score(cur))
        cur = nxt;
    step *= 0.98; // slower decay (0.999) = better quality, more time
}
```

8.11 Debug Checklist

Use: Contest debugging prompts for WA/RE/TLE/MLE.
Complexity: Checklist.
Pitfalls: Use before rewriting blindly; check output format and resets.
Assumptions/deps: Plain text notes.

Before submit

- Re-read constraints and output format; submit the right file.
- Test smallest, degenerate, duplicate, and maximum cases.
- Check overflow, memory, recursion depth, and per-test reset.
- Estimate time from the actual worst-case operation count.

Wrong answer

- Print the computed state, not only the final answer.
- Hand-simulate a tiny counterexample and add assertions.
- Check off-by-one bounds, N/M mixups, ties, duplicates, and unreachable states.
- Verify STL/API semantics and strict comparator ordering.
- Explain the proof and code to a teammate; let fresh eyes inspect it.
- If the model is wrong, rewrite from the invariant instead of patching symptoms.

Runtime, time, and memory

- **RE:** out-of-bounds, uninitialized data, zero division, invalid iterators, failed assertions, or recursion depth.
- **TLE:** hidden nested work, infinite loops, copying large containers, slow I/O, or an adversarial hash/table case.
- **MLE:** actual peak allocation, recursion stack, retained globals, or failure to clear between tests.

Known killers

- Hashing modulo 2^{64} by overflow: Thue-Morse pairs collide for every base. Use the notebook's modulo-$2^{61}-1$ hash.
- Default `unordered_map/set`: adversarial keys may share one GCC bucket. Use `chash` from `misc/custom_hash.cpp`.
- A chain of 10^6 recursive calls overflows an 8 MB stack. Use `raise_stack()` or an iterative traversal.
- Kuhn matching is $O(VE)$ and fails on dense adversarial graphs. Use unit-capacity Dinic when needed.
- Large FFT coefficient energy loses precision. Prefer NTT or FFT-mod.
- A non-strict comparator such as `<=` causes undefined behavior.
- `rand()` and `random_shuffle` are banned here; use `mt19937`.